
Auditoría Integral del Repositorio Observatorio MinCiencias — Investigadores Reconocidos

Documento maestro · Entendimiento, datos, código, estadística, lógica de construcción
y línea de tiempo

Consultor Estadístico Senior · Auditoría de Proyectos de Datos

Universidad Santo Tomás · Ustadistica · 2026-I

Repositorio auditado: `Observatorio_Ministerio_de_Ciencias_Grupo8`

Versión 1.0 · 29 de agosto de 2026

Documento generado de forma reproducible a partir de la auditoría del repositorio. La construcción de
este documento está documentada en `documentacion_auditoria`.

Índice

1. Resumen ejecutivo	4
1.1. Objeto de la auditoría	4
1.2. Veredicto general	4
1.3. Principales fortalezas verificadas	4
1.4. Principales brechas detectadas	4
1.5. Recomendaciones de máximo impacto	5
2. Entendimiento del objetivo del repositorio	5
2.1. Objetivo declarado	5
2.2. Preguntas de investigación que el repositorio responde	6
2.3. Evaluación del planteamiento del objetivo	6
2.4. Conclusión de la sección	7
3. Datos utilizados: suficiencia y corrección	7
3.1. Inventario de fuentes de datos	7
3.2. ¿Los datos eran suficientes para el objetivo?	8
3.3. ¿Los datos eran los correctos para el objetivo?	8
3.4. Evaluación de la calidad de datos por dimensión (Wang & Strong)	9
3.5. Conclusión de la sección	9
4. Documentación y crítica por archivo de código	9
4.1. Ingesta de datos	10
4.1.1. src/_init_.py	10
4.1.2. src/ingesta/_init_.py	10
4.1.3. src/ingesta/minciencias.py	12
4.1.4. src/ingesta/produccion.py	13
4.2. Transformación y limpieza	15
4.2.1. src/Transformacion.py	15
4.3. Modelado dimensional	17
4.3.1. src/modelo/_init_.py	17
4.3.2. src/modelo/dimensional.py	18
4.4. Análisis estadístico	19
4.4.1. src/analisis/calidad.py	20
4.4.2. src/analisis/diversidad.py	21
4.4.3. src/analisis/genero.py	23
4.4.4. src/analisis/longitudinal.py	24
4.4.5. src/analisis/produccion.py	26
4.4.6. src/analisis/redes.py	28
4.4.7. src/analisis/territorial.py	30
4.5. Orquestación de análisis (scripts por sprint)	31
4.5.1. scripts/sprint2_duckdb.py	32
4.5.2. scripts/sprint2_genero_ocde.py	34

4.5.3.	scripts/sprint2_matrices_transicion.py	36
4.5.4.	scripts/sprint2_panel_longitudinal.py	38
4.5.5.	scripts/sprint2_territorial.py	39
4.5.6.	scripts/sprint3_grafo_interactivo.py	41
4.5.7.	scripts/sprint3_redes.py	43
4.5.8.	scripts/sprint4_diversidad.py	45
4.5.9.	scripts/sprint5_duckdb.py	48
4.5.10.	scripts/sprint5_produccion.py	50
4.5.11.	scripts/sprint6_geografia_institucional.py	52
4.5.12.	scripts/sprint6_ocde_composicion.py	54
4.5.13.	scripts/sprint6_sankey_categoria.py	56
4.5.14.	scripts/sprint6_sankey_territorial.py	58
4.5.15.	scripts/sprint6_tabla_maestra_ies.py	60
4.5.16.	scripts/sprint6_validacion_calidad.py	62
4.5.17.	src/analisis/_init_.py	64
4.6.	Visualización y dashboard	65
4.6.1.	app/streamlit_app.py	65
4.6.2.	streamlit_app.py	67
4.7.	Documentación ejecutable	69
4.7.1.	docs/manual.ipynb	70
4.7.2.	scripts/generar_manual.py	72
4.8.	Análisis exploratorio de datos	75
4.8.1.	notebooks/01_eda.ipynb	75
4.9.	Pruebas automatizadas	78
4.9.1.	tests/_init_.py	78
4.9.2.	tests/test_ingesta.py	78
4.10.	Código legacy (etapa académica inicial)	79
4.10.1.	notebooks/tarea_analisis_bases_de_datos/2019_codigo.py	79
4.10.2.	notebooks/tarea_analisis_bases_de_datos/datos_2021.ipynb	81
4.10.3.	notebooks/tarea_analisis_bases_de_datos/inv_17_codigo_graficas.R	83
4.10.4.	notebooks/tarea_dimensiones_y_hechos/codigo_dimensiones_C.R	85
4.10.5.	notebooks/tarea_dimensiones_y_hechos/dimensiones.ipynb	86
4.10.6.	notebooks/tarea_dimensiones_y_hechos/tabla_hechos_codigo.R	88
4.10.7.	notebooks/tarea_gran_tabla/analisis2.ipynb	90
4.10.8.	notebooks/tarea_gran_tabla/analisis_de_clusters.R	92
4.10.9.	notebooks/tarea_gran_tabla/analisis_multivariado_base_gran_table.R	94
4.10.10.	notebooks/tarea_gran_tabla/codigo_analisis_univariado.py	96
4.10.11.	notebooks/tarea_gran_tabla/codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py	97
4.10.12.	notebooks/tarea_join/codigo_join.py	99
4.10.13.	notebooks/tarea_tabla/arreglar_base_consultoria_7.R	101
5.	Validación de la calidad de pruebas, tratamientos y lógica estadística	103
5.1.	Tratamiento de datos (preprocesamiento)	103

5.2. Análisis longitudinal	103
5.3. Análisis territorial	103
5.4. Análisis de género	104
5.5. Análisis de diversidad	104
5.6. Análisis de redes	104
5.7. Análisis de productividad (cruce con producción)	104
5.8. Clustering y análisis multivariado (legacy R)	104
5.9. Veredicto estadístico	105
6. Lógica de construcción del repositorio y cumplimiento del objetivo	105
6.1. Cómo se construyó el repositorio	105
6.2. ¿La lógica de construcción es correcta?	105
6.3. Flujos, herramientas y pasos incorrectos o no óptimos	106
6.4. ¿La ejecución cumplió el objetivo principal?	106
6.5. Verificación empírica de ejecutabilidad (auditoría 2026)	106
6.6. Recomendaciones de mejora de la lógica	107
7. Línea de tiempo de la construcción del repositorio	108
7.1. Resumen general	108
7.2. Distribución de autoría	108
7.3. Actividad por mes	108
7.4. Tabla completa de commits	108
7.5. Lectura de la línea de tiempo	112
A. Apéndice A: Inventario de archivos de código	112

1 Resumen ejecutivo

1.1 Objeto de la auditoría

Este documento presenta la auditoría integral del repositorio `Observatorio_Ministerio_de_Ciencias_Grupo8` (https://github.com/ustadistica/Observatorio_Ministerio_de_Ciencias_Grupo8), que materializa un **observatorio crítico del sistema de reconocimiento de investigadores del Ministerio de Ciencia, Tecnología e Innovación de Colombia (MinCiencias)**. El análisis cubre 236 archivos (51 de código), 133 commits (2025-08-24 a 2026-05-14), 12 autores y un pipeline que procesa 77 237 registros del padrón de investigadores (6 convocatorias, 2013–2021) y 3 166 629 filas de producción de grupos de investigación.

1.2 Veredicto general

Hallazgo

El repositorio cumple su objetivo declarado: consolidar, validar y analizar el padrón de investigadores reconocidos y producir un observatorio con 14 hallazgos críticos documentados, dashboard de 7 capítulos, informe LaTeX y presentación. La **arquitectura y la lógica estadística de nivel descriptivo-exploratorio son sólidas**; la **deuda técnica** (CI/CD rota, Dockerfile desalineado, codificación mojibake en el CSV canónico, tests mínimos, legacy R no reproducible, versionado de datos incompleto) compromete la **reproducibilidad end-to-end**, pero no la validez de los hallazgos ya generados.

1.3 Principales fortalezas verificadas

- **Documentación de nivel profesional**: README exhaustivo, catálogo YAML con diccionario de datos y llaves de cruce, decisiones metodológicas documentadas en el propio código (p. ej. tratamiento de atípicos en `src/analisis/calidad.py`).
- **Modularización correcta**: separación ingesta / transformación / análisis / modelado / orquestación con lógica reutilizable en `src/`.
- **Modelo dimensional adecuado**: esquema estrella en DuckDB con tabla puente N:N para la relación investigador–institución.
- **Evidencias trazables**: 79 CSVs y 56 PNG/HTML con convención de nombres consistente por sprint.
- **Enfoque crítico alineado con la evaluación responsable de la ciencia** (patrón *Pregunta* → *Hallazgo* → *Caveat*).

1.4 Principales brechas detectadas

1. **CI/CD rota**: `.github/workflows/etl_update.yml` ejecuta `src.ingesta.main` y `src.transformacion.main`, módulos que no existen (los reales son `src.ingesta.minciencias/produccion` y `src/Transformacion.py`).

2. **Dockerfile desalineado**: ejecuta `app/streamlit_app.py` (placeholder de 17 líneas) en lugar de `streamlit_app.py` (dashboard real).
3. **Mojibake en el CSV canónico**: `investigadores consolidado.csv` presenta caracteres corruptos (p. ej. *Física* por *Física*), indicativo de un desajuste UTF-8/Latin-1 en la exportación.
4. **Testing insuficiente**: 1 test funcional; `pandera` declarado pero sin uso; sin contratos de esquema.
5. **Legacy no reproducible**: scripts R/Python con rutas hardcodeadas (`Downloads/...`) y entradas no versionadas.
6. **Versionado de datos incompleto**: `datos/raw` y `datos/processed` gitignored sin DVC; la ingesta de producción (`33dq-ab5a`, 1.2 GB) no es reproducible desde el repositorio.
7. **Componente ML vacío**: `models/` sin contenido; el clustering (k-prototypes) vive en R legacy fuera del pipeline.
8. **Duplicidad de artefactos**: variantes `matriz_*`, `matriz*_ext` y `matriz*_obs` sin documentación de diferencias.
9. **Inconsistencia documental**: atípicos de edad reportados como 10 en un docstring y 22 en el README (el conteo real es 22).
10. **Nomenclatura desactualizada**: referencias a Grupo7, ramas y repos antiguos en el README.

1.5 Recomendaciones de máximo impacto

1. Corregir el workflow de GitHub Actions y el Dockerfile (reproducibilidad de la cadena de despliegue).
2. Normalizar la codificación del CSV consolidado y añadir un contrato de esquema (`pandera`) en ingesta y transformación.
3. Ampliar la cobertura de tests a las funciones analíticas y de transformación.
4. Migrar los análisis legacy (R) al pipeline Python en `src/` o documentarlos formalmente como material de referencia histórica.
5. Complementar la inferencia estadística: intervalos de confianza en índices de concentración, modelos de supervivencia para retención, modelos nulos para redes y validación formal de clusters.

2 Entendimiento del objetivo del repositorio

2.1 Objetivo declarado

El repositorio declara su objetivo en dos documentos con alcances distintos:

- `informe_final.md` (visión temprana): *“Consolidar, validar y analizar los datos de investigadores reconocidos de las convocatorias 2017, 2019 y 2021 del Minciencias,*

y producir un modelo dimensional y un conjunto de análisis descriptivos y multivariados para facilitar su uso en el observatorio institucional”.

- **README.md** (visión final): observatorio crítico del sistema de reconocimiento con análisis longitudinal de **6 convocatorias (2013–2021)**, con foco en calidad de los datos, concentración territorial, brecha de género, redes de co-filiación, diversidad, productividad cruzada con el dataset de producción de grupos y tabla maestra de IES.

Nota metodológica

El objetivo **evolucionó durante la construcción del proyecto**: de 3 convocatorias (2017/2019/2021) se pasó a las 6 históricas (2013–2021) y se incorporaron ejes nuevos (producción, diversidad, redes). Esta ampliación es coherente con la naturaleza exploratoria de un observatorio, pero implica que las métricas de las primeras fases (p. ej. matrices de transición 2017–2019) deben reinterpretarse a la luz del alcance ampliado.

2.2 Preguntas de investigación que el repositorio responde

1. ¿Qué tan confiable es la información que MinCiencias publica? ¿Permite analizar el sistema de manera comparable entre convocatorias?
2. ¿El sistema retiene a los investigadores reconocidos entre convocatorias? ¿Qué porcentaje sube, baja o desaparece?
3. ¿La capacidad investigativa está distribuida equitativamente o concentrada? (comparación per cápita con población DANE 2018)
4. ¿Hay paridad de género por gran área OCDE? ¿La brecha en STEM se ha cerrado?
5. ¿Cómo se conectan las instituciones a través de investigadores con doble afiliación? ¿Hay universidades-puente?
6. ¿La composición del padrón refleja la diversidad poblacional de Colombia (etnia, discapacidad, víctimas del conflicto)?
7. ¿Quién firma realmente la producción que MinCiencias mide? ¿La brecha de género se replica en outputs?

2.3 Evaluación del planteamiento del objetivo

- **Claridad**: el objetivo es comprensible y medible a través de las preguntas de investigación; cada pregunta mapea a un módulo de análisis (ver Tabla 1).
- **Idoneidad del diseño**: el proyecto adopta el ciclo CRISP-DM y el patrón “Pregunta → Hallazgo → Caveat”, alineado con la evaluación responsable de la ciencia (Principios de Leiden; Hicks et al., 2015).
- **Debilidad**: no existe un documento único de “teoría de cambio” o “marco de indicadores” que defina formalmente cada indicador, su fórmula, su umbral de alerta y su fuente de comparación; la formalización se recomienda como entregable de esta consultoría.

Tabla 1. Mapa objetivo → módulos del repositorio

Pregunta de investigación	Módulo que la responde
Fiabilidad y comparabilidad de los datos	src/analisis/calidad.py, src/Transformacion.py, tab 1 del dashboard
Retención y transición entre convocatorias	src/analisis/longitudinal.py, scripts/sprint2-matrices-transicion.py
Concentración territorial	src/analisis/territorial.py (HHI, cuotas), sprint2.territorial.py
Brecha de género por área OCDE	src/analisis/genero.py, sprint2.genero_ocde.py
Redes de co-filiación institucional	src/analisis/redes.py, sprint3.redes.py, sprint3-grafo_interactivo.py
Diversidad vs DANE/RUV	src/analisis/diversidad.py, sprint4-diversidad.py
Productividad cruzada	src/analisis/produccion.py, sprint5-produccion.py, sprint5_duckdb.py
Ajustes de calidad, IES y geografía	sprint6-*, src/analisis/calidad.py

2.4 Conclusión de la sección

El objetivo del repositorio está **bien entendido y correctamente descompuesto** en preguntas de investigación que los módulos efectivamente responden. La principal recomendación es **formalizar el marco de indicadores** (definición operativa, fórmula, fuente y límites de cada métrica) para garantizar que la entrega final sea auditable y reproducible.

3 Datos utilizados: suficiencia y corrección

3.1 Inventario de fuentes de datos

Tabla 2. Inventario de fuentes de datos del repositorio

Fuente	Descripción	Volumen	Estado en el clon
Padrón de investigadores	Dataset Socrata bqtm-4y2h: investigadores reconocidos por convocatoria, 6 convocatorias (2013–2021), 30 variables	77 237 filas · 30 086 únicos	Consolidado XLSX/CSV versionado en datos/tarea_ join/
Producción de grupos	Dataset Socrata 33dq-ab5a: una fila por producto-autor	3 166 629 filas · 77 401 autores	No presente (gitignored, 1.2 GB)
Catálogo	datos/catalogo.yaml: metadatos, diccionario de datos, llaves de cruce	308 líneas	Versionado
Gran tabla (sin ID)	Tabla analítica sin identificadores personales	— (mencionada en informe_final.md)	No versionada (solo referencias en notebooks legacy)

3.2 ¿Los datos eran suficientes para el objetivo?

Hallazgo

Sí, los datos son suficientes para el objetivo descriptivo-exploratorio planteado: 6 convocatorias permiten análisis longitudinal de retención y transición; 30 086 investigadores únicos dan potencia suficiente para desagregaciones por área, género y región; y el cruce con 3 166 629 productos permite analizar productividad. La cobertura temporal (2013–2021) es adecuada para estudiar una década de reconocimiento científico en Colombia.

Limitaciones de suficiencia (detalladas en la sección de alcance y riesgos):

- **Subgrupos pequeños:** raizales, palenqueros, Rrom y personas con discapacidad tienen conteos reducidos → estimaciones inestables, intervalos amplios.
- **Ventanas irregulares:** los intervalos entre convocatorias no son constantes (2013, 2014, 2015, 2017, 2019, 2021), lo que complica la comparación de tasas de transición entre periodos.
- **Ausencia de convocatorias recientes:** no se incluye 2023; el README lo señala como pendiente (Sprint 1).
- **Producción no reproducible localmente:** el dataset de producción no está versionado; su ingesta depende de la API Socrata (1.2 GB).

3.3 ¿Los datos eran los correctos para el objetivo?

Hallazgo

Los datos son los correctos para el objetivo planteado: el padrón de investigadores reconocidos es la fuente oficial del sistema de reconocimiento de MinCiencias y el dataset de producción es el complemento natural para medir outputs. La elección de fuentes de datos abiertos (datos.gov.co, Socrata) es apropiada y verificable.

Problemas de corrección detectados en la auditoría:

1. **Codificación (mojibake):** el CSV canónico `investigadores consolidado.csv` presenta caracteres corruptos (Física por Física, Bogotá; por Bogotá), un desajuste UTF-8/Latin-1 en la cadena de exportación Socrata → CSV. El XLSX no se ve afectado.
2. **Atípicos de edad:** 22 registros con edad > 100 años (máximo 956), tratados por eliminación documentada; inconsistencia documental 10 vs. 22.
3. **Edad fraccionaria:** la variable `EDAD_ANOS_PR` contiene valores decimales (p. ej. 54.48), lo que sugiere edad calculada con precisión de días y posterior redondeo inconsistente.
4. **Faltantes estructurales:** `NME_DEPARTAMENTO_*` “NO DISPONIBLE” 5.2%; género “NO REPORTADO” 2.7%; cobertura de etnia/discapacidad/conflicto 0% antes de 2021.
5. **Etiquetado de convocatoria:** la convocatoria 833 (2018) registra `ANO_CONV0` = 2019 (fecha de publicación de resultados); está documentado en el catálogo, pero puede inducir a error si no se controla.

6. **Consolidado versionado incompleto (3 de 6 convocatorias)**: tanto `datos/tarea_join/investigadores_consolidado.xlsx` como el CSV paralelo contienen **solo 50 891 filas y 3 convocatorias** (2017: 13 001, 2019: 16 796, 2021: 21 094), frente a las 77 237 filas y 6 convocatorias (2013–2021) declaradas en el README y el catálogo. Las evidencias (p. ej. matrices de transición 2013–2014) sí cubren las 6 convocatorias, lo que indica que el artefacto versionado es un *snapshot* de la fase inicial de 3 convocatorias que nunca se actualizó. Consecuencia: **un clon fresco que ejecute el pipeline no reproduce los hallazgos del README** (faltan 2013, 2014 y 2015); la fuente completa vive solo en `datos/raw` (gitignored). Hallazgo verificado con conteo real (pandas 3.0) sobre ambos archivos.
7. **Captura selectiva de co-filiación**: el campo `INST_FILIA` solo capturó múltiples instituciones en 2019 (569 casos), por lo que el análisis de redes es un corte transversal, no una serie.
8. **Identificadores**: se asumió la unicidad de `ID_PERSONA_PR` con una heurística de coherencia de edad (± 4 años) y género; no hay validación formal contra fuentes externas.

3.4 Evaluación de la calidad de datos por dimensión (Wang & Strong)

Tabla 3. Estado de la calidad de datos por dimensión (Wang & Strong, 1996)

Dimensión	Observación	Estado
Exactitud	Cruce con fuentes oficiales no realizado; mojibake en CSV	Parcial
Complejidad	Mapeada por convocatoria (heatmaps); faltantes estructurales declarados	Adecuada
Consistencia	Encodings inconsistentes; duplicidad de artefactos; etiquetas normalizadas en transformación	Parcial
Unicidad	Verificación de IDs por coherencia de edad y género; sin formalizar	Adecuada
Validez	Atípicos de edad documentados y tratados con justificación	Adecuada
Oportunidad	Datos de 2021 con pull 2026; convocatoria 2023 pendiente	Parcial

3.5 Conclusión de la sección

Los datos son **correctos y suficientes** para el objetivo; sin embargo, la auditoría detecta **problemas de calidad corregibles** (codificación, faltantes, atípicos, captura selectiva) que deben documentarse explícitamente en cada entregable y corregirse en el pipeline de transformación (normalización UTF-8, contrato de esquema, imputaciones documentadas).

4 Documentación y crítica por archivo de código

Esta sección documenta, para cada archivo de código del repositorio: qué hace (descripción), cómo está construido (lógica), una crítica de calidad (fortalezas, debilidades, mejoras), su

ejecutabilidad real y si produce los resultados que los investigadores buscaban.

Archivos documentados: **51**.

4.1 Ingesta de datos

Archivos: 4.

4.1.1 `src/__init__.py`

Rol: ingesta · **Líneas:** 2

Descripción funcional Marcador de paquete Python para `src/`. Convierte la carpeta `src/` en un paquete importable (los módulos se importan como `'from ingesta import ...'` vía `sys.path` en los scripts).

Análisis de la lógica Sin lógica: archivo vacío/de marcador (2 líneas).

Fortalezas

- Correcto por convención: permite imports del paquete.

Debilidades

- Ninguna relevante; es boilerplate estándar.

Ejecutabilidad N/A (marcador de paquete).

¿Produce lo esperado? N/A.

Recomendación de auditoría **mantener** — estándar de Python.

4.1.2 `src/ingesta/__init__.py`

Rol: ingesta · **Líneas:** 53

Descripción funcional Punto de entrada de carga de datos del paquete ingesta. Expone `cargar_consolidado()` y `cargar_produccion()`, con prioridad de rutas (CSV crudo Socrata -> XLSX local para investigadores; CSV crudo obligatorio para producción), normaliza nombres de columnas a MAYÚSCULAS y reporta conteos cargados.

Análisis de la lógica Define rutas absolutas ROOT/datos/raw y ROOT/datos/tarea_join. cargar_consolidado() prefiere datos/raw/investigadores_consolidado.csv (Socrata) y cae a datos/tarea_join/investigadores_consolidado.xlsx; cargar_produccion() exige datos/raw/produccion_grupos.csv y lanza FileNotFoundError si falta, sugiriendo python -m src.ingesta.produccion. Es el contrato de entrada que usan todos los scripts (from ingest import cargar_consolidado, cargar_produccion).

Fortalezas

- Mecanismo de fallback raw ->tarea_join permite ejecutar análisis de investigadores sin datos crudos.
- Uppercase centralizado de columnas: garantiza el contrato MAYÚSCULAS que exigen analisis/*.py y Transformacion.py.
- Mensajes de error accionables que indican el comando de ingesta a ejecutar.

Debilidades

- cargar_consolidado() no valida el shape esperado (77.237 x 30) ni los tipos: con low_memory=True, id_persona_pr con ceros a la izquierda puede leerse de forma inconsistente (int vs str).
- cargar_produccion() solo acepta el CSV crudo: no hay fallback ni lectura desde el modelo DuckDB; bloquea todo el análisis de producción en el clon.
- Depende de openpyxl para read_excel, declarado solo en requirements.txt (no en pyproject.toml): una instalación vía poetry pura fallaría.
- El comando sugerido python -m src.ingesta.minciencias no pagina (hace una sola llamada con limit=100.000) y puede truncar silenciosamente los 77.237 registros si Socrata limita la respuesta por petición (contraste con ingest/produccion.py que sí pagina).

Mejoras propuestas

- Añadir validación de shape y conteo contra datos/catalogo.yaml (tamano_registros).
- Especificar dtypes en read_csv (id_persona_pr como str) para preservar ceros a la izquierda y garantizar el cruce con producción.
- Declarar openpyxl y pyyaml en pyproject.toml.
- Añadir fallback para cargar_produccion desde parquet o desde el modelo DuckDB si el CSV crudo no está.

Ejecutabilidad cargar_consolidado(): SÍ en el clon — cae al XLSX datos/tarea_join/investigadores_consolidado.xlsx (existe, 8 MB) porque datos/raw/investigadores_consolidado.csv NO existe (solo .gitkeep). cargar_produccion(): NO — datos/raw/produccion_

grupos.csv no existe (gitignored; requiere descarga de ~1,2 GB). El módulo en sí es importable sin datos.

¿Produce lo esperado? No produce archivos; es capa de lectura. Sus salidas (DataFrames) alimentan transformar() y analisis.*. Cumple el contrato documentado en README (ingesta/cargar_consolidado + cargar_produccion).

Riesgo / Limitación

Bugs / problemas detectados:

- src/ingesta/_init_.py:44-48: cargar_produccion() lanza FileNotFoundError sin alternativa; en el clon (sin datos/raw) todos los scripts sprint5/sprint6 de producción mueren en el arranque.
- src/ingesta/_init_.py:26: read_csv(low_memory=False) + read_excel sin dtype explícito -> ID_PERSONA_PR puede inferirse como int en un lado y str en otro, rompiendo el cruce id_persona_pd <-> id_persona_pr.
- src/ingesta/_init_.py:18-36: no verifica duplicados por (ID_PERSONA_PR, ID_CONVOCATORIA) pese a que el README define esa granularidad para el dataset.

Recomendación de auditoría mejorar — diseño sólido de fallback, pero requiere validación de tipos/duplicados y dependencias declaradas (openpyxl/pyyaml) para que la instalación reproducible funcione.

4.1.3 src/ingesta/minciencias.py

Rol: ingesta · **Líneas:** 70

Descripción funcional Descarga el dataset consolidado de investigadores reconocidos de MinCiencias desde datos.gov.co (API Socrata, sodapy; recurso bqtm-4y2h) y lo guarda en datos/raw/investigadores_consolidado.csv. Actualiza el campo ultimo_pull del catálogo YAML. CLI con -limit (muestra de pruebas) y -token (app token Socrata).

Análisis de la lógica descargar() crea el cliente Socrata, trae hasta limit (por defecto 100.000) registros y construye un DataFrame. guardar() persiste a CSV UTF-8 en datos/raw. actualizar_catalogo() reescribe el YAML con la fecha de hoy. main() orquesta y, si no hay límite, actualiza el catálogo. Logging con formato estándar.

Fortalezas

- CLI limpia y documentada (-limit para pruebas evita descargas completas).
- Separación descargar/guardar/actualizar_catalogo permite reutilización y tests.
- Maneja app token Socrata para evitar throttling.

Debilidades

- Sin paginación para datasets grandes (el límite por defecto 100.000 cubre el padrón de 77k, pero es frágil si el dataset crece).
- Escribe en datos/raw (gitignored): el artefacto no se versiona ni se valida.
- actualizar_catalogo reescribe el YAML completo con yaml.dump, perdiendo comentarios y formato original del catálogo.
- Sin control de calidad post-descarga (conteo de filas, codificación UTF-8) — el mojibake del CSV canónico no se detecta aquí.

Mejoras propuestas

- Añadir paginación genérica (offset) para datasets >100k.
- Validar el resultado (filas esperadas, codificación) tras la descarga.
- Actualizar el catálogo conservando comentarios o con un campo separado.

Ejecutabilidad Parcial: requiere red y API Socrata (no ejecutable offline); en la auditoría no se ejecutó porque los datos ya están en el clon (el XLSX versionado proviene de esta fuente). datos/raw está gitignored, por lo que el resultado no es visible en el repositorio.

¿Produce lo esperado? Sí, en su momento: los 77.237 registros (según README/catálogo) se descargaron de esta fuente; el XLSX consolidado versionado es el derivado. No verificable en clon por gitignore.

Riesgo / Limitación

Bugs / problemas detectados:

- yaml.dump reescribe el catálogo y puede alterar su estructura/orden (riesgo de formato).

Recomendación de auditoría mantener — simple y funcional; mejorar la validación de codificación y el manejo del catálogo.

4.1.4 src/ingesta/produccion.py

Rol: ingest · **Líneas:** 106

Descripción funcional Descarga paginada del dataset de Producción de Grupos (Socrata 33dq-ab5a, 3.166.629 filas) a datos/raw/produccion_grupos.csv, con modo -limit para pruebas y actualización de metadatos en datos/catalogo.yaml. Es la única ingesta del proyecto con paginación real.

Análisis de la lógica Usa `sodapy.Socrata` con `timeout=300`; `descargar()` pagina con `$offset/$limit` de 50.000 y `order='id'` hasta que un chunk venga vacío o con menos de `PAGE_SIZE`; con `-limit` hace una sola llamada. `guardar()` escribe CSV UTF-8; `actualizar_catalogo()` actualiza `ultimo_pull` y `tamano_registros` de la fuente 33dq-ab5a en `catalogo.yaml`; `main()` orquesta y omite el catálogo en modo prueba.

Fortalezas

- Paginación robusta con orden estable (`:id`) y condición de corte doble (chunk vacío o `<PAGE_SIZE`).
- CLI argparse con modo `limit` para pruebas sin descargar 1,2 GB.
- Cierra el cliente `Socrata` explícitamente y loguea progreso por página.
- Actualiza el catálogo de metadatos automáticamente tras la descarga completa.

Debilidades

- Importa `yaml` en el módulo pero `PyYAML` NO está declarado en `pyproject.toml` ni `requirements.txt` (`sodapy` no lo trae) -> `ImportError` en un entorno limpio.
- Descarga todas las columnas sin proyección (`select`), incluyendo títulos largos (`nme_producto_pd`) -> `payload` y tiempo innecesarios en 3,2 M de filas.
- Sin reintentos/`backoff` ante errores de red ni validación del conteo final contra el catálogo (3.166.629).
- `yaml.dump` reescribe `catalogo.yaml` reformateándolo (indentación y comillas), generando diffs ruidosos en un archivo versionado con formato manual.
- `if limit:` con `limit=0` ejecutaría la descarga completa (valor `falsy` no contemplado).

Mejoras propuestas

- Añadir `select` de columnas analíticas (`id_persona_pd`, `id_convocatoria`, `ano_convo`, `nme_tipo_medicion_pd`, `cod_grupo_gr`, etc.).
- Declarar `pyyaml` en `pyproject.toml` y `requirements.txt`.
- Verificar `len(df)` contra `catalogo['tamano_registros']` y reintentar páginas fallidas con `backoff` exponencial.
- Guardar en `parquet` con tipos explícitos (`id_persona_pd` como `str` de 10 caracteres).

Ejecutabilidad Ejecutable con `red: python -m src.ingesta.produccion` (completa) o `-limit N`. Requiere `sodapy`, `pandas` y `PyYAML` (este último no declarado en el proyecto). Escribe en `datos/raw/` (`gitignore`; en el clon el directorio existe con `.gitkeep` y es escribible localmente, pero la salida NO está versionada ni presente en el clon).

¿Produce lo esperado? Produce datos/raw/produccion_grupos.csv (gitignored, ~1,2 GB), el input que exige cargar_produccion(). No genera evidencias/ ni artifacts/ directamente; coincide con lo documentado en README (ingesta por API Socrata paginada, Sprint 5).

Riesgo / Limitación

Bugs / problemas detectados:

- src/ingesta/produccion.py:26: import yaml no cubierto por pyproject.toml -> falla en instalación reproducible (probablemente ejecutado ad-hoc con PyYAML instalado manualmente).
- src/ingesta/produccion.py:47-53: en modo limit main() igualmente guarda el DataFrame como CSV 'completo' en SALIDA: si alguien corre -limit sin querer, sobrescribe el archivo crudo con una muestra.
- src/ingesta/produccion.py:59-65: si la última página devuelve exactamente PAGE_SIZE y la siguiente viene vacía, el loop hace una petición extra vacía (ineficiencia menor, no error).

Recomendación de auditoría mantener/mejorar — única ingesta con paginación correcta del proyecto; corregir la dependencia PyYAML declarada y el sobrescrito en modo limit para producción segura.

4.2 Transformación y limpieza

Archivos: 1.

4.2.1 src/Transformacion.py

Rol: transformacion · **Líneas:** 222

Descripción funcional Núcleo de limpieza y normalización del dataset consolidado de investigadores (77.237 registros). Aplica limpieza de texto (strip + mayúsculas), parseo robusto del año de convocatoria (fecha, serial de Excel o regex de 4 dígitos), estandarización de género y tratamiento de faltantes (mediana para numéricas, 'NO REPORTADO' para categóricas). Entrada: DataFrame crudo de ingesta.cargar_consolidado(); salida: DataFrame transformado listo para análisis.

Análisis de la lógica Flujo en transformar(): limpiar_texto -> parsear_ano_convocatoria -> estandarizar_genero -> tratar_faltantes. Usa constantes COLS_NUMERICAS y COLS_CATEGORICAS; funciones puras que copian el df. En __main__ inserta ROOT/src en sys.path y llama cargar_consolidado() + transformar(). Se conecta con el resto del pipeline porque todos los scripts sprint* ejecutan df = transformar(cargar_consolidado()) antes de llamar a analisis.*.

Fortalezas

- Funciones puras con docstrings claros y operaciones idempotentes (strip + upper).
- Parseo de año multiestrategia (fecha, serial Excel, regex) — robusto frente a formatos heterogéneos de ANO_CONVO.
- Centraliza la lógica de limpieza en un solo módulo consumido por todos los scripts.

Debilidades

- Imputa con mediana global sin crear columna indicadora de imputación: para un observatorio de calidad de datos, destruye la señal de 'missingness' (el propio proyecto reporta faltantes como hallazgo).
- No corrige la anomalía documentada de la convocatoria 833 (ANO_CONVO=2019-12-06, año nominal 2018): ANO_CONVO_INT quedará en 2019 para esa convocatoria.
- Imputar mediana en columnas ordinales (ORDEN_CLAS_PR, NRO_ORDEN_FORM_PR) puede producir valores no enteros que contaminan agrupaciones posteriores.
- Docstring dice 'Grupo 7' y 'generado por ingesta.py' mientras el repo es Grupo 8 y el módulo de carga es ingesta/__init__.py (desactualización cosmética).

Mejoras propuestas

- Añadir columna flag (ej. *_imputado) por variable imputada y comparar análisis con/sin imputación.
- Aplicar corrección de año nominal para la conv. 833 según datos/catalogo.yaml (campo ano_convo_campo).
- Validar shape/nulos contra catálogo (77.237 x 30) y loguear filas descartadas.
- Escribir tests para parsear_ano_convocatoria con casos: '2013', serial 41739, '2019-12-06', NaN.

Ejecutabilidad Sí, ejecutable tal cual en el clon para el dataset de investigadores: __main__ carga vía ingesta.cargar_consolidado(), que cae al fallback datos/tarea_join/investigadores_consolidado.xlsx (existe, 8 MB) porque datos/raw/ solo tiene .gitkeep (sin CSV crudo). Requiere pandas, numpy (declarados) y openpyxl (solo en requirements.txt, no en pyproject.toml). NO depende de datos/raw ni de .duckdb.

¿Produce lo esperado? No escribe archivos (solo devuelve DataFrame y print de shape); el volcado a CSV lo hacen los scripts sprint*. Su output alimenta todos los análisis del README; consistente con lo documentado.

Riesgo / Limitación**Bugs / problemas detectados:**

- Transformacion.py:107-109: la regex `r'\{\}b(20[0-9]{2})\{\}b'` solo captura años 2000-2099; correcto para el dominio, pero un año fuera de rango quedaría como NaT sin aviso.
- Transformacion.py:99-100: `pd.to_datetime(s, dayfirst=True)` sobre un texto numérico tipo '640' (id de convocatoria) produciría NaT; depende del contenido real de ANO_CONVO, que el catálogo documenta como año o timestamp.
- Transformacion.py:140-141: si una columna numérica fuera 100 % NaN, `mediana=NaN` y `fillna(NaN)` no corrige nada (caso límite no manejado).

Recomendación de auditoría mejorar — núcleo correcto y bien estructurado; debe mejorarse la trazabilidad de imputaciones y la corrección del año nominal 2018 antes de seguir usándolo en un informe de calidad de datos.

4.3 Modelado dimensional

Archivos: 2.

4.3.1 `src/modelo/__init__.py`

Rol: modelado · **Líneas:** 2

Descripción funcional Marcador de paquete Python para `src/modelo/` (módulo del modelo dimensional).

Análisis de la lógica Sin lógica: archivo de marcador.

Fortalezas

- Correcto por convención.

Ejecutabilidad N/A.

¿Produce lo esperado? N/A.

Recomendación de auditoría mantener — estándar de Python.

4.3.2 src/modelo/dimensional.py

Rol: modelado · **Líneas:** 286

Descripción funcional Construye el modelo dimensional (Issue #14) en DuckDB: 7 dimensiones (convocatoria, investigador, categoría, área, territorio, formación, institución) + fact_clasificacion (investigador x convocatoria) + tabla puente N:N bridge_hecho_institucion. Entrada: DataFrame transformado; salida: datos/processed/observatorio.duckdb (gitignored).

Análisis de la lógica Construye cada dimensión con drop_duplicates sobre su clave natural y SK solo para territorio/institucion; construir_fact_clasificacion() une la SK de territorio por columnas geográficas e inserta sk_hecho; construir_bridge_institucion() descompone inst_filia con un dict sk y un loop de filas. cargar_en_duckdb() crea el archivo (mkdir parents) y hace DROP + CREATE TABLE AS SELECT por tabla; resumen_modelo() lista tablas y conteos. Importa normalizar_categoria/ORDEN_CATEGORIAS desde analisis.longitudinal (requiere ROOT/src en sys.path).

Fortalezas

- Esquema claro y documentado (asociación dimensión -> hecho, más bridge para la relación N:N de instituciones).
- SK para territorio e institución y ordenamiento determinista antes de drop_duplicates en varias dimensiones.
- DB_PATH centralizado y carga idempotente (DROP + CREATE).
- La tabla puente resuelve correctamente la granularidad N:N que una dimensión plana no podría representar.

Debilidades

- LA 'PRIMERA APARICIÓN' NO ES CRONOLÓGICA: construir_dim_investigador() ordena solo por ID_PERSONA_PR (no por ANO_CONVO_INT) y luego keep='first'; el snapshot de atributos no corresponde realmente a la primera convocatoria del investigador.
- El fact referencia claves naturales (id_convocatoria, id_persona_pr, id_clas_pr, id_area_con_pr, id_niv_formacion_pr) sin SK: las dimensiones (salvo territorio/institucion) carecen de clave sustituta y no hay FKs/constraints — es más un conjunto de tablas de referencia que un modelo estrella estricto.
- _drop_and_create() es código muerto (definido y nunca llamado), sugiriendo una intención de DDL con constraints que no se implementó.
- construir_bridge_institucion() itera las ~77.237 filas con iterrows: lento y sin vectorización.
- El archivo .duckdb es gitignored y NO existe en el clon: el modelo no es reproducible sin volver a ejecutar (requiere duckdb >= 1.0 y el XLSX).

Mejoras propuestas

- Ordenar por [ANO_CONVO_INT, ID_PERSONA_PR] en construir_dim_investigador() para que 'primera aparición' sea real.
- Añadir SK (surrogate) a todas las dimensiones y referenciarlas en el fact; definir PK/FK con ALTER TABLE o CREATE TABLE con constraints.
- Vectorizar el bridge (explode + map) en lugar de iterrows.
- Documentar el modelo como 'tablas de referencia + hecho' si no se van a imponer FKs, para no sobrevender el término 'estrella'.

Ejecutabilidad Sí para la parte de investigadores en el clon: requiere duckdb, pandas y el XLSX de tarea_join (presente); escribe datos/processed/observatorio.duckdb (gitignored; en el clon solo hay .gitkeep, el archivo NO está). No requiere datos/raw. NOTA: para incorporar producción (sprint5_duckdb.py) sí haría falta el CSV crudo.

¿Produce lo esperado? Sí (con datos): genera las tablas documentadas y evidencias/duckdb_resumen_modelo.txt existe como evidencia de ejecución del equipo; el README reporta el modelo estrella como completado (Issue #14). El .duckdb final no está versionado por diseño (.gitignore: *.duckdb).

Riesgo / Limitación

Bugs / problemas detectados:

- src/modelo/dimensional.py:69-74: sort solo por ID_PERSONA_PR ->'keep first' no es la primera aparición temporal; los atributos del snapshot (género, región) pueden pertenecer a una convocatoria posterior.
- src/modelo/dimensional.py:41-44: _drop_and_create nunca se usa (código muerto).
- src/modelo/dimensional.py:192-197: el merge por columnas geográficas con NaN (código DANE ausente) no empareja en pandas (NaN != NaN) ->fact con sk_territorio NULL para filas sin geografía, sin contarlas ni advertirlo.
- src/modelo/dimensional.py:52-56: construir_dim_convocatoria conserva ANO_CONVO_INT = 2019 para la convocatoria id 20 (año nominal 2018), propagando la anomalía de datos documentada al modelo.

Recomendación de auditoría mejorar — modelo útil y correcto en granularidad, pero necesita SK/constraints para ser un 'modelo estrella' real y corregir el snapshot cronológico del investigador antes de citarlo como modelo dimensional canónico.

4.4 Análisis estadístico

Archivos: 7.

4.4.1 `src/analisis/calidad.py`

Rol: analisis · **Líneas:** 105

Descripción funcional Validación documentada de la calidad de los datos del padrón, con foco en atípicos de edad (`EDAD_ANOS_PR`). Documenta explícitamente la decisión metodológica (eliminación frente a imputación por mediana del grupo) para que cualquier auditor pueda reproducirla y discutirla. Expone `detectar_atipicos_edad`, `resumen_tratamiento`, `filtrar` (método elegido) e `imputar_mediana` (alternativa comparativa).

Análisis de la lógica Umbral `UMBRAL_EDAD_MAX=100`. `detectar_atipicos_edad` devuelve los registros con `edad>100` (mínimas columnas de identificación). `resumen_tratamiento` produce el reporte antes/después con justificación. `filtrar` aplica eliminación (decisión por defecto). `imputar_mediana` reemplaza los atípicos por la mediana del grupo (gran área, clasificación, género) como alternativa reproducible. El docstring documenta el razonamiento: 22 casos sobre 77.237 (0.028 %), sesgo despreciable, eliminación preferida por trazabilidad.

Fortalezas

- Decisión metodológica documentada y justificada en el propio código (transparencia auditable).
- Ofrece comparación explícita entre eliminación e imputación (análisis de sensibilidad).
- Funciones puras y pequeñas, sin efectos laterales.
- El filtro se aplica solo donde el análisis depende de la edad.

Debilidades

- Docstring dice 'diez registros' (10) mientras el conteo real documentado en README y evidencias es 22 — inconsistencia documental verificada.
- Umbral fijo (100) sin parametrización.
- `imputar_mediana` usa `apply` fila a fila (lento con 77k filas).
- El resto de anomalías (NO DISPONIBLE 5.2 %, NO REPORTADO 2.7 %, conv. 833 etiquetada 2019) se documenta en el docstring pero no se instrumenta como funciones.

Mejoras propuestas

- Corregir el docstring a 22 casos y parametrizar el umbral.
- Vectorizar `imputar_mediana` (`groupby transform` en vez de `apply`).
- Instrumentar también las otras dimensiones de calidad (completitud, consistencia, unicidad) como funciones.

Ejecutabilidad Sí, verificada empíricamente: `sprint6_validacion_calidad.py` (que lo usa) ejecutado en clon fresco con `EXIT=0` en 61.6 s. Requiere el XLSX consolidado (presente).

¿Produce lo esperado? Sí: `evidencias/calidad_atipicos_edad.csv`, `calidad_comparacion.csv` y `calidad_resumen.json` existen en el clon y coinciden con el tratamiento documentado (22 atípicos, eliminación).

Riesgo / Limitación

Bugs / problemas detectados:

- Docstring (línea ~10): 'diez registros' vs 22 reales — inconsistencia que puede confundir al lector del informe.

Recomendación de auditoría mejorar — lógica correcta y bien documentada; corregir la inconsistencia documental y ampliar la instrumentación de calidad.

4.4.2 `src/analisis/diversidad.py`

Rol: analisis · **Líneas:** 162

Descripción funcional Análisis de variables de diversidad (Issue #20): cobertura temporal de `ID_VICTIMA_CONFLICTO`, `TXT_GRUPO_ETNICO` y `TXT_POBLACION_DISCA`, distribución por categoría, comparación de subrepresentación frente a DANE 2018/RUV 2021, cruce interseccional género x etnia y categorías por minoría. Entrada: DataFrame transformado (mayúsculas aplicadas por `src/Transformacion.py`); salida: DataFrames exportados a `evidencias/diversidad_*.csv`.

Análisis de la lógica Define `DANE_REFERENCIA` (constantes poblacionales) y `VALORES_NULOS` por variable. `cobertura_por_convocatoria()` calcula % NO REGISTRA/NO DISPONIBLE y cobertura por año. `comparar_dane()` (diseñada para `df_2021`) mapea nombres `MinCiencias` -> `DANE` con regex y calcula la razón de subrepresentación. `interseccional_genero-etnia()` y `categoria_por_minoria()` cruzan con género y categoría académica. El script `sprint4_diversidad.py` filtra `df_2021` antes de llamar a `comparar_dane`.

Fortalezas

- Hallazgo central documentado en el propio módulo: las variables solo se capturaron desde 2021 — honestidad metodológica.
- Razón de subrepresentación (`DANE/MinCiencias`) correcta y con manejo de división por cero (`None`).
- Constantes de referencia (`DANE 2018 / RUV 2021`) centralizadas y comentadas.

Debilidades

- Contrato frágil: `comparar_dane`, `interseccional_genero_etnia` y `categoría_por_minoría` asumen que el caller ya filtró la convocatoria 2021; si se pasa el df completo, los resultados mezclan años sin advertencia.
- ACOPLAMIENTO ROTO CON `Transformacion.py`: éste convierte NaN categórico en 'NO REPORTADO', pero diversidad espera 'NO REGISTRA'/'NO DISPONIBLE'; los faltantes reales etiquetados 'NO REPORTADO' no se cuentan como nulos -> cobertura inflada.
- Bug metodológico en `comparar_dane()`: el comentario (línea 78) dice excluir 'NINGUN GRUPO ETNICO' del denominador, pero el código (línea 79) solo excluye 'NO DISPONIBLE'; 'NINGUN GRUPO ETNICO' queda en el denominador y diluye la subrepresentación de las minorías (verificado: comentario vs código).
- Comparación 'SÍ' con tilde (línea 121): funciona con los datos actuales porque `transformar()` pone en mayúsculas 'Sí' -> 'SÍ', pero es frágil ante variantes sin tilde ('Si'/'SI') — el catálogo documenta 'Si'.
- `interseccional_genero_etnia()` puede lanzar `KeyError` 'FEMENINO' si algún grupo étnico no tiene mujeres (la columna no existe tras `unstack`).

Mejoras propuestas

- Que las funciones filtren internamente por `ANO_CONVO_INT == 2021` (parametrizado) para hacer la API autónoma.
- Alinear los valores nulos entre `Transformacion` y diversidad (constante compartida) o contar también 'NO REPORTADO' como faltante.
- Excluir de verdad 'NINGUN GRUPO ETNICO' del denominador (alinear comentario y código) y documentar el impacto en las razones de subrepresentación.
- Normalizar 'SÍ'/'SI'/'Si' con `strip + upper` antes de comparar (o eliminar acentos).
- Usar `.reindex(columns=['FEMENINO', 'MASCULINO'], fill_value=0)` tras el `unstack` en `interseccional_genero_etnia`.

Ejecutabilidad Sí en el clon: solo DataFrame transformado (XLSX presente); las funciones de 2021 requieren que el caller filtre (`sprint4_diversidad.py` lo hace). Sin datos/raw ni `.duckdb`.

¿Produce lo esperado? Sí: `evidencias/diversidad_cobertura_por_convocatoria.csv`, `diversidad_comparacion_dane.csv`, `diversidad_distribucion_*.csv`, `diversidad_interseccional_genero_etnia.csv` y `diversidad_categorias_por_etnia.csv` existen, junto con `artifacts/sprint4_diversidad/fig01-03.png` — consistente con el README (Issue #20, hallazgos 4-6).

Riesgo / Limitación**Bugs / problemas detectados:**

- `src/analisis/diversidad.py:78-79`: contradicción comentario-código sobre 'NINGUN GRUPO ETNICO' en el denominador de `comparar_dane()` — diluye la subrepresentación de minorías (verificado por lectura directa).
- `src/analisis/diversidad.py:145`: `d['FEMENINO']` puede lanzar `KeyError` si el `unstack` no crea la columna (grupo étnico sin mujeres) — crash en caso límite.
- `src/analisis/diversidad.py:44-46`: los NaN convertidos a 'NO REPORTADO' por `Transformacion.py` no se cuentan como nulos -> `pct_no_registra` subestimado y cobertura sobreestimada para registros con faltante real.
- `src/analisis/diversidad.py:121`: comparación 'SÍ' sensible a acentos; no es un bug activo (los datos traen 'Sí' y `transformar()` lo pone en mayúsculas), pero fallaría si la fuente usara 'Si' sin tilde.

Recomendación de auditoría mejorar — hallazgos valiosos y bien enmarcados; prioridad alta: corregir el denominador de `comparar_dane()`, alinear valores nulos con `Transformacion.py` y robustecer comparaciones de cadenas.

4.4.3 `src/analisis/genero.py`

Rol: analisis · **Líneas:** 85

Descripción funcional Análisis de género por área OCDE (Issue #22): porcentaje femenino, tablas pivot y brecha de género por convocatoria a 3 niveles de área. Entrada: `DataFrame` transformado con `NME_GENERO_PR` estandarizado y `ANO_CONVO_INT`; salida: `DataFrames` `tidy/pivot` listos para exportar a evidencias/`genero_*.csv` y graficar.

Análisis de la lógica `pct_femenino_por_area()` filtra género en {FEMENINO, MASCULINO} (excluye NO REPORTADO del denominador, decisión documentada), agrupa por (año, área), hace `value_counts().unstack()` y calcula `n_femenino`, `n_masculino`, `n_total` y `pct`. `tabla_pivot_pct_femenino()` pivotea área x año con promedio y ordena; `brecha_genero()` deriva `pct_masculino = 1 - pct_femenino` y la brecha; `evolucion_pct_femenino()` reordena. El script `sprint2.genero_ocde.py` llama a estas funciones y exporta los CSVs.

Fortalezas

- Decisión metodológica explícita y correcta: excluir NO REPORTADO del denominador evita sesgos de imputación.
- Funciones puras, parametrizables (`col_area`, `col_year`) y reutilizables en scripts y dashboard.
- Documenta el contrato de entrada (género estandarizado, `ANO_CONVO_INT` disponible).

Debilidades

- `evolucion_pct_femenino()` es redundante (mismo DataFrame ordenado); el propio docstring lo admite.
- No aplica `dropna` sobre `col_area`: un área con NaN se pierde silenciosamente del `groupby` (aceptable pero implícito).
- `pct_femenino = n_fem/n_total` puede dividir por cero si un grupo tiene 0 registros reportados ->NaN propagado a `pivote` y `brecha` sin aviso.
- `brecha` usa `pct_masculino = 1 - pct_femenino`, válido solo dentro del universo 'reportado'; la columna `n_masculino` calculada no se aprovecha.

Mejoras propuestas

- Eliminar `evolucion_pct_femenino` o fusionarla con un parámetro `sort`.
- Añadir manejo explícito de grupos vacíos (`n_total=0` ->`pct=NaN` con advertencia).
- Escribir tests con df pequeño (área 100 % hombres, área con solo NO REPORTADO).

Ejecutabilidad Sí en el clon: solo requiere el DataFrame transformado, que se obtiene de `cargar_consolidado()` con fallback al XLSX presente. Sin dependencias de datos/raw ni `.duckdb`.

¿Produce lo esperado? Sí: `evidencias/genero_pct_femenino_por_area.csv`, `genero_pct_femenino_por_gran_area.csv`, `genero_brecha_por_gran_area.csv` y `genero_tabla_pivot_gran_area.csv` existen en el clon, junto con `artifacts/sprint2_genero_ocde/fig01-04.png` — consistente con el README (Issue #22).

Riesgo / Limitación

Bugs / problemas detectados:

- `src/analisis/genero.py:32-33`: `conteos.get('FEMENINO', 0)` funciona porque `DataFrame.get` acepta default escalar, pero si un grupo queda sin filas reportadas, `n_total=0` y `pct_femenino=NaN` (división por cero) sin advertencia — caso límite real si una convocatoria tuviera 100 % NO REPORTADO.
- `src/analisis/genero.py:67`: `pct_masculino = 1 - pct_femenino` asume universo binario; correcto tras el filtro, pero la columna `n_masculino` ya existe y no se usa (inconsistencia menor).

Recomendación de auditoría mantener — análisis correcto y bien documentado; mejoras menores de robustez (grupos vacíos) y limpieza de redundancia.

4.4.4 `src/analisis/longitudinal.py`

Rol: analisis · **Líneas:** 207

Descripción funcional Núcleo del análisis longitudinal (Issue #11): construye el panel investigador×convocatoria a partir de ID_PERSONA_PR, compara trayectorias entre periodos consecutivos (sube/baja/se mantiene/desaparece), calcula matrices de transición de conteos y probabilidades, y tasas de retención con entrada de nuevos investigadores. Cubre las 6 convocatorias históricas (2013-2021).

Análisis de la lógica ORDEN_CATEGORIAS define la escala ordinal (Junior=1 ... Emérito=4) y normalizar_categoria mapea el texto libre a las 4 etiquetas canónicas. construir_panel deduplica (ID, año) conservando la categoría más alta. comparar_periodo hace left join desde t0 hacia t1 (los que desaparecen quedan representados) y clasifica el resultado. matriz_transicion calcula conteos (crosstab) y probabilidades normalizadas por fila, con opción de incluir 'Desaparece' como estado final. tasa_retencion_por_periodo computa retención (presentes en t0 que reaparecen) y nuevos en t1.

Fortalezas

- Implementación estadística correcta: crosstab normalizado por fila = matriz de Markov empírica; left join conserva la atrición.
- Escala ordinal explícita para clasificar sube/baja/se mantiene.
- Estado 'Desaparece' tratado como opción configurable.
- Docstrings con interpretación metodológica (caveat eméritos vitalicios documentado en README/sprint6).

Debilidades

- No se testea la propiedad markoviana ni la homogeneidad temporal; con intervalos irregulares (2013, 2014, 2015, 2017, 2019, 2021) las tasas no son directamente comparables entre periodos.
- comparar_periodo usa apply con closure fila a fila (lento con 77k filas).
- La 'desaparición' incluye a los eméritos vitalicios (diseño del sistema), que sesgan la interpretación si no se aísla (documentado en sprint6, no en el código).
- No hay análisis de supervivencia (Kaplan-Meier/Cox) para el tiempo hasta la salida.

Mejoras propuestas

- Añadir test de markovianidad y matrices condicionadas a historia.
- Vectorizar la clasificación del tracking (np.select en vez de apply).
- Complementar con modelos de supervivencia y reportar IC bootstrap de las tasas.

Ejecutabilidad Sí, verificada empíricamente: usado por sprint2_panel_longitudinal.py (EXIT=0, 108 s) y sprint2_matrices_transicion.py (EXIT=0, 74 s) en clon fresco. Requiere

XLSX consolidado (presente).

¿Produce lo esperado? Sí: matrices de transición (evidencias/matriz_*.csv), paneles y tasas versionados coinciden con la lógica; los regenerados con el clon (3 convocatorias) cubren solo 2 periodos — limitación de datos, no de código.

Riesgo / Limitación

Bugs / problemas detectados:

- Ninguno crítico; riesgo metodológico: interpretar las transiciones como cadena de Markov sin validar la propiedad (caveat recomendado).

Recomendación de auditoría mantener con mejoras — lógica sólida para el alcance descriptivo; la ampliación inferencial (markovianidad, supervivencia) es la recomendación de mayor valor.

4.4.5 src/analisis/produccion.py

Rol: analisis · **Líneas:** 340

Descripción funcional Análisis cruzado producción <-> investigadores (Sprint 5): cobertura de reconocimiento, productividad por categoría/género/área/territorio, mix de tipologías y reconocidos sin producción. Entrada: prod (3.166.629 filas, Socrata 33dq-ab5a) e inv (consolidado transformado), cruzados por ID_PERSONA_PD <-> ID_PERSONA_PR (+ ID_CONVOCATORIA); salida: DataFrames exportados a evidencias/produccion_*.csv y consumidos por sprint5-produccion.py.

Análisis de la lógica normalizar_produccion() pone columnas en MAYÚSCULAS, convierte IDs a Int64 y extrae ANO_CONVO_INT. cobertura_reconocidos() marca es_reconocido por isin global y agrega por año; cobertura_autores_unicos() devuelve un dict de solapamiento. productividad_por_categoria()/por_genero_area() agregan n_productos por (persona, convocatoria) y hacen left merge con inv. productividad_territorial() suma por departamento con per cápita y ratio de concentración. mix_tipologias() y tipologias_por_categoria() agregan por NME_TIPO_MEDICION_PD. reconocidos_sin_produccion() cruza conjuntos de autores por ID_CONVOCATORIA.

Fortalezas

- Responde de forma completa las 6 preguntas críticas del Sprint 5 con funciones bien nombradas y docstrings que explican la interpretación de cada métrica.
- Manejo correcto de tipos para el cruce (to_numeric -> Int64 en ambos lados).

- Coberturas y brechas con denominadores explícitos (`pct_con_al_menos_un_producto`, `razon_fm`, `ratio_concentracion`).
- Evidencias producidas y versionadas: 7 CSVs + resumen JSON en `evidencias/`.

Debilidades

- Requiere `datos/raw/produccion_grupos.csv` (~1,2 GB, gitignored): en el clon ninguna función de este módulo es ejecutable hasta descargar el archivo.
- `normalizar_produccion()` solo usa `pd.to_datetime` para `ANO_CONVO`: si el CSV exporta años como enteros (2013), `to_datetime` los interpreta como nanosegundos -> año 1970; no replica la estrategia multi-formato de `Transformacion.py` (riesgo latente según el formato real del export).
- `reconocidos_sin_produccion()` hace `groupby(...).apply(set)` sobre 3,2 M filas: lento y con alto uso de memoria.
- Los merges asumen granularidad única (persona, convocatoria) en `inv`; duplicados inflarían `n_productos` (sin verificación previa).
- `brecha_productividad_genero()` lanza `KeyError` si un área no tiene mujeres (columna ausente tras el pivot).

Mejoras propuestas

- Replicar el parseo multiestrategia de año de `Transformacion.py` en `normalizar_produccion()` o unificarlo en un helper compartido.
- Vectorizar `reconocidos_sin_produccion()` con `merge` sobre autores únicos por convocatoria.
- Verificar unicidad de (`ID_PERSONA_PR`, `ID_CONVOCATORIA`) en `inv` antes de `merge`.
- Reindexar columnas en `brecha_productividad_genero` para áreas sin un género.

Ejecutabilidad NO en el clon: `cargar_produccion()` falla porque `datos/raw/produccion_grupos.csv` no existe (gitignored; requiere `python -m src.ingesta.produccion`, ~1,2 GB y red). Las funciones son puras y ejecutables si se les pasan `DataFrames`, pero el flujo documentado exige el CSV crudo. No depende del `.duckdb`.

¿Produce lo esperado? Sí (con datos): `evidencias/produccion_cobertura_por_convocatoria.csv`, `produccion_productividad_por_categoria.csv`, `produccion_brecha_genero.csv`, `produccion_concentracion_territorial.csv`, `produccion_mix_tipologias.csv`, `produccion_tipologias_por_categoria.csv`, `produccion_reconocidos_sin_produccion.csv` + `produccion_resumen_cobertura.json` + 6 figuras en `artifacts/sprint5-produccion/` — todos existen en el clon y coinciden con el README (Sprint 5, hallazgos 8-10).

Riesgo / Limitación**Bugs / problemas detectados:**

- `src/analisis/produccion.py:43`: `pd.to_datetime(ANO_CONVO)` sin manejo de ints/seriales -> año 1970 si el campo es numérico (inconsistente con `Transformacion.py:99-116`).
- `src/analisis/produccion.py:199`: `pivot.rename` no falla si falta 'FEMENINO', pero `pivot['prom_femenino']` sí lanza `KeyError` en áreas sin mujeres.
- `src/analisis/produccion.py:89`: división por `len(autores_unicos)` -> `ZeroDivisionError` si prod está vacío (caso límite).
- `src/analisis/produccion.py:63-65`: `isin` usa IDs normalizados a `Int64`; si `id_persona_pr` llega con ceros a la izquierda desde el XLSX (leído como `str`) y prod como `int`, el cruce pierde matches (depende de la inferencia de tipos).

Recomendación de auditoría mantener/mejorar — módulo metodológicamente sólido y bien documentado; priorizar la robustez de tipos/años y la reproducibilidad del input crudo (o versionar un parquet de producción) para que el cruce sea reproducible sin 1,2 GB.

4.4.6 `src/analisis/redes.py`

Rol: analisis · **Líneas:** 225

Descripción funcional Análisis de redes de co-filiación institucional (Issues #15/#16): normaliza nombres de instituciones, extrae pares de instituciones de `INST_FILIA` (separador '|'), construye grafos `NetworkX`, calcula métricas (grado, `betweenness`, densidad, componentes) y genera HTML interactivo con `Pyvis`. Entrada: `DataFrame` transformado con `INST_FILIA`; salida: `DataFrames` de pares/nodos/aristas (`evidencias/redes_*.csv`) y HTML en hallazgos/.

Análisis de la lógica `normalizar_institucion()` aplica heurística de paréntesis (si el paréntesis contiene una institución madre, lo usa; si no, lo elimina), elimina sufijos SEDE/SECCIONAL y normaliza mayúsculas. `construir_pares()` filtra filas con '|', divide en 2 partes (`n=1`), normaliza, elimina self-loops y ordena pares para que $(A,B) == (B,A)$. `construir_grafo()` agrega pesos por grupo (`a,b`). `metricas_grafo()`, `tabla_nodos()`, `tabla_aristas()`, `metricas_por_convocatoria()` y `subgrafo_grado_minimo()` componen el resto. `generar_html_pyvis()` pinta nodos por grado ponderado y `betweenness`.

Fortalezas

- Normalización institucional con reglas documentadas y heurística de paréntesis — valor real frente a las 2.762 cadenas crudas de `inst_filia`.
- Eliminación de self-loops tras normalizar y orden canónico de pares: evita aristas duplicadas $(A,B) == (B,A)$.

- Módulo completo (construcción, métricas, subgrafos, visualización) con lazy import de pyvis.
- Artefactos versionados en evidencias/redes_*.csv y artifacts/sprint3_redes/.

Debilidades

- **CRÍTICO:** construir_pares() divide con n=1, capturando solo el PRIMER par de investigadores con 3+ instituciones; se pierden las combinaciones restantes (A,C), (B,C) ->el grafo subestima la co-filiación.
- El grafo global (anio=None) pondera por filas investigador x convocatoria: un investigador con el mismo par en 2 convocatorias cuenta doble, inflando el peso de aristas 'investigadores compartidos'.
- import networkx en mitad del archivo (línea 49) y un import networkx as nx_local sin uso dentro de generar_html_pyvis — estilo e higiene.
- Si INST.FILIA termina en '|' (cadena vacía tras split), se crea un nodo con nombre " en el grafo (no se filtra la cadena vacía, a diferencia de modelo/dimensional.py).
- El bloque 'if G.has_edge' en construir_grafo es código muerto (cada grupo (a,b) es único en el groupby).

Mejoras propuestas

- Usar itertools.combinations sobre las instituciones separadas para generar todos los pares de investigadores multi-filiados.
- Agregar peso por investigadores únicos (drop_duplicates por ID_PERSONA_PR antes de contar) en el grafo global.
- Filtrar cadenas vacías y NaN después del split (strip + dropna + != ").
- Mover los imports al inicio del módulo; eliminar nx_local no usado.

Ejecutabilidad Sí en el clon (solo DataFrame transformado, XLSX presente); requiere networkx y, para generar_html_pyvis, pyvis (ambos declarados en pyproject.toml). Sin datos/raw ni .duckdb.

¿Produce lo esperado? Sí: evidencias/redes_pares_cofiliacion.csv, redes_nodos_global.csv, redes_aristas_global.csv y redes_metricas_por_convocatoria.csv existen, junto con artifacts/sprint3_redes/fig01-03.png; los HTML Pyvis los genera sprint3_grafo_interactivo.py. Coincide con el README (Issues #15/#16).

Riesgo / Limitación

Bugs / problemas detectados:

- src/analisis/redes.py:62-64: str.split('|', n=1) descarta afiliaciones 3+; red incompleta

(subestimación de aristas y nodos).

- `src/analisis/redes.py:74-79`: `sorted()` sobre pares que contienen " (de 'A | ') crea un nodo vacío en el grafo; con tipos no comparables podría lanzar `TypeError`.
- `src/analisis/redes.py:93-98`: el peso del grafo global cuenta filas investigador x convocatoria, no investigadores únicos: 'investigadores compartidos' queda inflado entre años.
- `src/analisis/redes.py:16-48`: `normalizar_institucion` devuelve el valor original (posiblemente sin estandarizar) en el caso base.

Recomendación de auditoría refactorizar — análisis con buen valor analítico, pero la pérdida de pares multi-filiación y el conteo duplicado de pesos comprometen las métricas publicadas (`n_aristas`, peso de aristas); corregir antes de citar resultados de red.

4.4.7 `src/analisis/territorial.py`

Rol: analisis · **Líneas:** 107

Descripción funcional Análisis de concentración territorial (Issue #13): índice Herfindahl-Hirschman (HHI) por convocatoria, top territorios y tabla de cuotas departamento x año. Entrada: DataFrame transformado; salida: DataFrames exportados a `evidencias/territorial_*.csv`.

Análisis de la lógica `hhi()` normaliza conteos a cuotas y suma sus cuadrados (devuelve NaN si la suma es 0). `hhi_por_convocatoria()` filtra 'NO REPORTADO' y `dropna`, agrupa por año y reporta `hhi`, `n_total` y `n_territorios`. `top_territorios()` calcula conteos por (año, territorio) con pct sobre el total anual y filtra los n territorios más frecuentes del total acumulado. `tabla_cuotas()` pivotea departamento x año con las cuotas. `sprint2_territorial.py` los consume (HHI por departamento y por región).

Fortalezas

- HHI implementado correctamente (cuotas normalizadas, rango [0,1]) con docstring de interpretación.
- Filtro explícito de 'NO REPORTADO' y `dropna` antes de calcular.
- Métricas auxiliares (`n_total`, `n_territorios`) dan contexto al índice.

Debilidades

- top_territorios mezcla dos alcances: el 'top n' se calcula sobre el total acumulado del período, pero pct se calcula por año -> un territorio top en un año específico puede quedar excluido de la tabla (pérdida de información por año).
- hhi() con una sola unidad devuelve 1.0 (concentración total), correcto matemáticamente pero debe interpretarse junto con n_territorios.
- El filtro de 'NO REPORTADO' depende de que Transformacion.py haya etiquetado así los faltantes: acoplamiento implícito a un valor mágico.
- Sin normalización adicional de nombres de departamentos (p.ej. 'Bogotá D.C.' vs 'Bogota') más allá del upper del pipeline.

Mejoras propuestas

- Separar el alcance del top-n (por año o global) en parámetros explícitos.
- Mover el filtro de 'NO REPORTADO' a una constante compartida (p.ej. en analisis/__init__) para evitar valores mágicos.
- Añadir HHI normalizado (HHI*) para comparar entre años con distinto número de territorios.
- Tests: serie de conteos, suma cero, una sola unidad.

Ejecutabilidad Sí en el clon: solo requiere DataFrame transformado (XLSX presente). Sin datos/raw ni .duckdb.

¿Produce lo esperado? Sí: evidencias/territorial_hhi_por_convocatoria.csv, territorial_hhi_region_por_convocatoria.csv y territorial_cuotas_departamento.csv existen, junto con artifacts/sprint2_territorial/fig01-03.png — consistente con el README (Issue #13).

Riesgo / Limitación

Bugs / problemas detectados:

- src/analisis/territorial.py:81-87: el top-n global combinado con pct anual puede producir tablas donde ningún año muestre el territorio #1 de ese año; no es error de cálculo, pero distorsiona el concepto de 'top territorios por convocatoria'.
- src/analisis/territorial.py:44 y 71: si col_geo contuviera valores no-cadena (el NaN ya se dropea, pero p.ej. códigos numéricos), .str.strip() lanzaría AttributeError — depende del contrato de ingesta.

Recomendación de auditoría mantener — análisis correcto y exportable; mejorar la semántica del top-n y centralizar el filtro de faltantes.

4.5 Orquestación de análisis (scripts por sprint)

Archivos: 17.

4.5.1 scripts/sprint2_duckdb.py

Rol: orquestacion · **Líneas:** 127

Descripción funcional Construye el modelo dimensional estrella (7 dimensiones + fact_clasificacion + bridge_hecho_institucion) en datos/processed/observatorio.duckdb a partir del dataset consolidado transformado, y lo valida con 6 consultas de negocio (investigadores por convocatoria, distribución de categorías, top-10 instituciones, gran área, concentración Bogotá/Medellín/Cali, edad promedio por categoría y año). Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/modelo/dimensional (cargar_en_duckdb, resumen_modelo, DB_PATH). Salidas: la base DuckDB y evidencias/duckdb_resumen_modelo.txt.

Análisis de la lógica main() encadena [1/3] transformar(cargar_consolidado()), [2/3] cargar_en_duckdb(df, path=DB_PATH, verbose=True) + resumen_modelo(con) y [3/3] ejecución de CONSULTAS (dict nombre→SQL) con con.execute(sql).fetchdf(), imprimiendo cada resultado y acumulando lineas_reporte que se escriben al final en evidencias/duckdb_resumen_modelo.txt (encoding utf-8). La construcción del esquema y el DROP/CREATE idempotente viven en src/modelo/dimensional.py, no en el script; este solo orquesta carga, validación y persistencia del reporte. Cierra la conexión al final.

Fortalezas

- Modelo estrella explícito y bien documentado en el docstring (dims, fact, bridge N:N con dim_institucion), coherente con src/modelo/dimensional.py.
- Las consultas de validación son preguntas de negocio reales (concentración territorial, edad por categoría, distribución OCDE), no solo conteos triviales; funcionan como smoke test del modelo.
- Idempotente gracias al DROP TABLE IF EXISTS + CREATE en cargar_en_duckdb: permite re-ejecución sin estado corrupto acumulado.
- El reporte de texto en evidencias es verificable y de hecho existe en el clon con el formato exacto que genera este script.

Debilidades

- Usa duckdb fetchdf(), API deprecada desde duckdb 0.9.0 en favor de df(); con el pin duckdb ^1.0 del pyproject, una instalación reciente puede emitir DeprecationWarning o directamente fallar (riesgo de ejecutabilidad futura).
- resumen_modelo(con) solo imprime en stdout: el resumen del modelo estrella (conteos por tabla) NO queda en el reporte de evidencias, que solo captura las 6 consultas.
- Sin try/finally: si una consulta o la construcción falla, la conexión duckdb queda abierta y no hay mensaje de error contextual.

- No valida el total de registros (esperado 77.237) contra `fact_clasificacion` ni reporta duplicados de `id_persona_pr` por convocatoria.
- `DB_PATH` está fijado en `src/modelo/dimensional.py` y no hay parámetro CLI; el script no permite apuntar a otra base ni modo `–solo-consultas`.

Mejoras propuestas

- Reemplazar `fetchdf()` por `df()` (línea 110) y también en `src/modelo/dimensional.py:283`.
- Incluir el output de `resumen_modelo()` en `lineas_reporte` para que la evidencia contenga el esquema completo.
- Envolver el flujo en `try/finally` con `con.close()` y capturar excepción con mensaje accionable.
- Añadir una consulta de sanidad: `COUNT(*)` de `fact_clasificacion` $\approx$ 77.237 y verificar que ninguna `dim` tenga claves `NULL` usadas por los `JOINS`.
- Parametrizar `DB_PATH` vía `argparse/entorno` para reutilizar el script en entornos CI.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias: `pyproject` declara `duckdb ^1.0`, pero el `.venv` del clon está vacío (solo `pip/setuptools`) y `requirements.txt` no incluye `duckdb`. El input sí existe: `cargar_consolidado()` prioriza `datos/raw/investigadores_consolidado.csv` (NO existe, `gitignored`) y cae al fallback `datos/tarea_join/investigadores_consolidado.xlsx` (SÍ existe). La base `datos/processed/observatorio.duckdb` no existe en el clon (solo `.gitkeep`) y el script la crea. Riesgo adicional: `fetchdf()` deprecado según la versión de `duckdb` instalada.

¿Produce lo esperado? Sí, con matiz: `evidencias/duckdb-resumen_modelo.txt` existe en el clon y su contenido (encabezado `'Modelo estrella — observatorio.duckdb'` + las 6 consultas con sus `to_string`) coincide exactamente con lo que genera el script, lo que demuestra que se ejecutó al menos una vez con datos. El archivo `observatorio.duckdb` NO está en el clon (`gitignored`, `datos/processed` solo contiene `.gitkeep`), por lo que esa salida no es reproducible sin re-ejecución. Coincidencia declarado-vs-generado: OK para el reporte.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint2_duckdb.py:110` — `con.execute(sql).fetchdf()`: `fetchdf()` está deprecado desde `duckdb 0.9.0` (reemplazado por `.df()`); con `duckdb ^1.0` en `pyproject.toml` una versión reciente puede eliminarlo, rompiendo el script en runtime.
- `scripts/sprint2_duckdb.py:104` — `resumen_modelo(con)` imprime a `stdout` pero su salida no se agrega a `lineas_reporte`: la evidencia guardada (`duckdb-resumen_modelo.txt`) omite el resumen del modelo estrella que el propio `docstring` describe como parte de la validación.
- `scripts/sprint2_duckdb.py:97-123` — sin `try/finally`: cualquier excepción entre [2/3] y [3/3] deja la conexión `duckdb` sin cerrar y sin reporte parcial.

Recomendación de auditoría mantener — es la base del modelo dimensional del observatorio y sus consultas de validación son valiosas; solo requiere la migración `fetchdf()→df()`, incluir el resumen en el reporte y robustecer el manejo de conexión.

4.5.2 `scripts/sprint2_genero_ocde.py`

Rol: orquestacion · **Líneas:** 182

Descripción funcional Analiza la representación femenina y la brecha de género por área OCDE (gran área, área) en las 6 convocatorias 2013-2021. Usa `src/analisis/genero` (`pct_femenino_por_area`, `tabla_pivot_pct_femenino`, `brecha_genero`, `evolucion_pct_femenino`) sobre `transformar(cargar_consolidado())`. Salidas: 4 figuras PNG en `artifacts/sprint2_genero_ocde/` (barras, heatmap gran área×convocatoria, líneas de evolución, brecha primera vs última convocatoria) y 4 CSVs en `evidencias/`.

Análisis de la lógica `main()` imprime el % femenino global (filtrando solo FEMENINO/MASCULINO), genera las 4 figuras con funciones dedicadas (`fig_barras_pct_femenino`, `fig_heatmap_pct_femenino`, `fig_lineas_evolucion`, `fig_brecha_genero`), y exporta las tablas: `pct` por gran área, `pivot` con promedio, `brecha` (M%-F %) y `pct` por área. Cada función de figura llama a la función de `analisis/genero` correspondiente y guarda con `plt.savefig(dpi=150)` tras `plt.tight_layout()`; usa `matplotlib.use('Agg')` para entornos headless.

Fortalezas

- Código limpio y directo: separación clara carga→figuras→exportación, funciones de figura autocontenidas y reutilizables.
- Uso correcto de la capa analítica (`src/analisis/genero`): el script no recalcula métricas, solo orquesta y visualiza.
- Backend Agg antes de importar pyplot, correcto para ejecución por consola/CI.
- Salidas verificadas: las 4 figuras y los 4 CSVs existen en el clon, en las rutas declaradas en el docstring.

Debilidades

- Escalas fijas que saturan la información: heatmap con `vmin=0`, `vmax=60` (`fig02`) y líneas con `ylim(0,70)` (`fig03`); si las áreas STEM superan 60 % de presencia masculina, el color y la curva quedan aplastados y no se distingue entre áreas.
- `fig_brecha_genero` alinea `b0` con `reindex` sobre las áreas de la última convocatoria: si una gran área existe en `a0` pero no en `a1` (o viceversa), se produce NaN silencioso y la comparación pierde áreas sin ninguna advertencia.

- El resumen global (pct femenino total) solo se imprime en consola; no se persiste como evidencia.
- Sin validación del mínimo de registros por área/convocatoria: celdas con n muy bajo (posibles identificadores) se grafican igual que áreas con miles de registros.

Mejoras propuestas

- Autoeescalar el heatmap (vmin/vmax del dato o por cuantiles) o documentar por qué 60 % es el tope; igual para ylim de fig03.
- En fig_brecha_genero usar union de áreas (b0 y b1) y rellenar NaN con 0 o marcar 'sin dato', e imprimir advertencia.
- Exportar el resumen global a un CSV de evidencias.
- Anotar n por celda (o tamaño de punto) en el heatmap para visibilizar áreas con pocos investigadores.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (matplotlib, seaborn, pandas; .venv del clon vacío, requirements.txt no las lista aunque pyproject sí). El input existe: cargar_consolidado() usa el fallback datos/tarea_join/investigadores_consolidado.xlsx (presente). No requiere datos/raw ni duckdb. Con poetry install el script corre de punta a punta en headless.

¿Produce lo esperado? Sí, coincidencia total: existen artifacts/sprint2_genero_ocde/fig01_barras_pct_femenino.png, fig02_heatmap_pct_femenino.png, fig03_lineas_evolucion.png, fig04_brecha_genero.png y evidencias/genero_pct_femenino_por_gran_area.csv, genero_tabla_pivot_gran_area.csv, genero_brecha_por_gran_area.csv, genero_pct_femenino_por_area.csv. Las 8 salidas declaradas en el docstring y en el README están presentes y con los nombres exactos, lo que indica ejecución exitosa previa.

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint2_genero_ocde.py:132-135 — b0.reindex(areas) con areas derivada solo de b1: si el conjunto de gran áreas difiere entre convocatorias extremas, las barras de b0 quedan en NaN (no se dibujan) sin advertencia, y las áreas exclusivas de a0 desaparecen del gráfico.
- scripts/sprint2_genero_ocde.py:81 — sns.heatmap con vmin=0, vmax=60 fijos: cualquier gran área con % femenino >60 (esperable en ciencias de la salud o educación) satura el rojo/verde de RdYlGn y pierde discriminación visual.
- scripts/sprint2_genero_ocde.py:108 — ax.set_ylim(0, 70) en fig03: series que superen 70 % (posible en áreas pequeñas) quedan cortadas en el borde superior sin indicación.

Recomendación de auditoría mantener — análisis correcto y reproducible con salidas verificadas; las debilidades son de visualización (escalas fijas) y de robustez

del alineamiento de áreas, corregibles con cambios menores.

4.5.3 scripts/sprint2_matrices_transicion.py

Rol: orquestacion · **Líneas:** 125

Descripción funcional Calcula y visualiza las matrices de probabilidad de transición entre categorías (Junior/Asociado/Senior/Emérito) para los 5 periodos consecutivos de las 6 convocatorias (2013-2021), en dos variantes: observada (solo investigadores retenidos) y extendida (incluye el estado 'Desaparece'). Usa src/analisis/longitudinal (construir_panel, comparar_periodo, matriz_transicion, matrices_todos_periodos). Salidas: 2 figuras de heatmaps en artifacts/sprint2_transiciones/ y 20 CSVs (conteos y probabilidades $\times$ ext/obs $\times$ 5 periodos) en evidencias/.

Análisis de la lógica main() construye el panel (ID_PERSONA_PR, año, categoria, orden), imprime las matrices por periodo en consola con un bucle comparar_periodo+matriz_transicion, y luego recalcula todo con matrices_todos_periodos para las figuras y la exportación. fig_heatmaps() dibuja un heatmap por periodo en una sola fila de subplots (1 $\times$ n) con sns.heatmap(annot=True, fmt='.2f', vmin=0, vmax=1). Los nombres de archivo derivan del periodo reemplazando el guion largo '-' por '_' (p. ej. matriz_probabilidades_ext_2015_2017.csv).

Fortalezas

- Presenta explícitamente las dos variantes metodológicas (observada vs extendida con 'Desaparece'), lo que permite ver el sesgo de selección por deserción en lugar de ocultarlo.
- La lógica estadística está delegada en src/analisis/longitudinal (crosstab, renormalización por fila), manteniendo el script como orquestador puro.
- El manejo del guion largo '-' en nombres de archivo evita problemas de codificación de rutas en Windows.
- Salidas verificadas: las 2 figuras y los 20 CSVs de la nomenclatura actual existen en el clon.

Debilidades

- Cómputo duplicado: el bucle de impresión (líneas 83-91) recalcula las matrices que matrices_todos_periodos vuelve a calcular en la línea 94-95; con 77.237 registros y 5 periodos es ineficiente y propenso a divergencias si cambia la lógica.
- La matriz 'observada' renormaliza por fila excluyendo a los desaparecidos: sus probabilidades no suman a la población real y pueden sobreestimar la movilidad; el script no advierte de esta limitación interpretativa.

- Evidencias con nomenclatura heredada: en evidencias/ coexisten archivos de una versión anterior (`matriz_transicion_extendida_*.csv`, `matriz_probabilidades_extendida_*.csv` y variantes sin prefijo `ext/obs` como `matriz_transicion_2017_2019.csv`) que este script ya no produce, duplicando y confundiendo la evidencia.
- `fig_heatmaps` asume `n >= 1` periodo: con un solo par de años `plt.subplots(1, n)` devolvería un Axes escalar y `zip()` fallaría (robustez marginal).
- Sin validación de que el panel contenga exactamente 6 convocatorias / 5 periodos ni de tamaños de celda mínimos.

Mejoras propuestas

- Calcular matrices una sola vez (fuera del bucle de impresión) y reutilizarlas para print, figuras y exportación.
- Añadir advertencia explícita cuando la matriz observada descarte $>X\%$ de la fila por deserción, y priorizar la lectura de la matriz extendida en el informe.
- Eliminar o archivar los CSVs legacy (`extendida_`/sin prefijo) para dejar una sola convención de nombres.
- Proteger `fig_heatmaps` para `n==1` (`ax = axes` si escalar) y añadir un `assert` del número de periodos esperados.
- Escribir un test que verifique que las filas de la matriz de probabilidades sumen 1.0 (`obs`) y 1.0 incluyendo `Desaparece` (`ext`).

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (`pandas`, `matplotlib`, `seaborn`; `.venv` del clon vacío, `requirements.txt` incompleto). El input existe: `datos/tarea_join/investigadores consolidado.xlsx` vía `cargar_consolidado()` (fallback del CSV `raw` inexistente). No requiere `duckdb` ni `datos/raw`. Con `poetry install`, corre de punta a punta.

¿Produce lo esperado? Sí: existen `artifacts/sprint2.transiciones/fig01_heatmaps_ext.png` y `fig02_heatmaps_obs.png`, y en evidencias/ los 20 CSVs con la nomenclatura actual (`matriz_transicion_ext_`/`matriz_probabilidades_ext_`/`matriz_transicion_obs_`/`matriz_probabilidades_obs_` × 2013_2014, 2014_2015, 2015_2017, 2017_2019, 2019_2021), coincidiendo con lo declarado (`evidencias/matriz_transicion_*.csv`, `matriz_probabilidades_*.csv`). Adicionalmente hay 8 archivos de nomenclatura antigua (`extendida_`/sin prefijo) que evidencian ejecuciones previas con una versión distinta del script.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint2_matrices_transicion.py:83-95` — las matrices se calculan dos veces (bucle de impresión y `matrices_todos_periodos`): doble costo de cómputo sobre 77.237 registros y riesgo de que las cifras impresas difieran de las exportadas si la función cambiara entre ambas llamadas.

- `scripts/sprint2_matrices_transicion.py:49` — `plt.subplots(1, n)` con `n=1` devuelve un solo Axes (no iterable); el `zip(axes, periodos)` fallaría con `TypeError` si el panel tuviera un único periodo.
- Evidencias/ legacy: los archivos `matriz_transicion_extendida_2017_2019.csv`, `matriz_probabilidades_extendida_2019_2021.csv` y `matriz_transicion_2017_2019.csv` (sin prefijo) ya no son generados por este script; quedan como evidencia huérfana que puede confundir la trazabilidad.

Recomendación de auditoría mantener — es la entrega central del análisis longitudinal y produce evidencia verificable en el clon; requiere eliminar el cómputo duplicado, advertir el sesgo de la matriz observada y limpiar la nomenclatura legacy.

4.5.4 `scripts/sprint2_panel_longitudinal.py`

Rol: orquestacion · **Líneas:** 229

Descripción funcional Orquesta el análisis longitudinal del Sprint 2 (Issue #11): carga el consolidado (`cargar_consolidado` + `transformar`), construye el panel de investigadores por convocatoria con `src/analisis/longitudinal.py`, genera 7 figuras (evolución total, por género, por gran área, categorías, nuevos ingresos, tracking por periodo, tasas de retención) y exporta 4+ CSVs de evidencias.

Análisis de la lógica `main()` en 4 pasos: [1/4] carga y transforma; [2/4] genera figuras descriptivas (funciones `fig_*` con `matplotlib/seaborn`, backend `Agg`); [3/4] construye el panel (`construir_panel`), calcula tracking (`tracking_todos_periodos`) y retención (`tasa_retencion_por_periodo`); [4/4] exporta resumen, ingresos, tasas y tracking por categoría a evidencias/. Patrón claro: una función por figura, salidas documentadas en el docstring.

Fortalezas

- Docstring con uso y lista explícita de salidas (`artifacts/` y evidencias/).
- Delega la lógica en `src/analisis/longitudinal.py` (separación de responsabilidades correcta).
- Usa `matplotlib.use('Agg')` (headless, reproducible).
- Maneja los 5 periodos consecutivos de las 6 convocatorias de forma genérica (zip de años).

Debilidades

- Estilo global de `seaborn/matplotlib` aplicado al módulo (efecto secundario al importar).

- Sin tests ni manejo de errores (si cargar_consolidado fallara, crash sin mensaje claro).
- Figuras con tamaños fijos (figsize) poco adaptables a distintos formatos de reporte.
- No guarda los DataFrames intermedios (panel) para reutilización.

Mejoras propuestas

- Extraer la configuración de estilo a una función o archivo de tema.
- Añadir un pequeño test de humo (panel con datos sintéticos).
- Permitir configurar figsize por parámetro y guardar el panel en evidencias/.

Ejecutabilidad Sí, verificada empíricamente en la auditoría 2026: ejecutado en un clon fresco (venv Python, pandas 3.0.5) con EXIT=0 en 108.4 s. Requiere el XLSX de datos/tarea_join (presente). NOTA: al correr sobre el XLSX versionado (3 convocatorias) genera solo 2 periodos de tracking, frente a los 5 periodos de las evidencias versionadas — consecuencia del consolidado incompleto, no del script.

¿Produce lo esperado? Sí: genera las 7 figuras en artifacts/sprint2_longitudinal/ (versionadas en el clon) y los CSVs en evidencias/ (panel_longitudinal_resumen_todos_periodos.csv, ingresos_por_convocatoria, tasas_retencion, panel_longitudinal_cat_*.csv). Los artefactos existentes coinciden con lo declarado, pero con 6 convocatorias (los regenerados con el clon tienen 3).

Riesgo / Limitación

Bugs / problemas detectados:

- Ninguno crítico; dependencia implícita de que el consolidado tenga las 6 convocatorias para reproducir los 5 periodos del README (problema de datos, no de código).

Recomendación de auditoría mantener — bien estructurado y verificable; solo requiere el consolidado completo de 6 convocatorias para reproducir los resultados del README.

4.5.5 scripts/sprint2_territorial.py

Rol: orquestacion · **Líneas:** 142

Descripción funcional Analiza la concentración territorial de los investigadores con el índice de Herfindahl-Hirschman (HHI) por departamento y por región para las 6 convocatorias (2013-2021), más las cuotas (%) por departamento y el top-10 de departamentos. Usa

src/analisis/territorial (hhi_por_convocatoria, top_territorios, tabla_cuotas) sobre transformar(cargar_consolidado()). Salidas: 3 figuras PNG en artifacts/sprint2_territorial/ y 3 CSVs en evidencias/.

Análisis de la lógica main() genera fig_top_departamentos (barras apiladas por convocatoria del top 10 con pct por año), calcula e imprime el HHI departamental y regional, genera fig_hhi_evolucion (2 subplots lado a lado con anotaciones de valor) y fig_heatmap_cuotas (heatmap % por departamento×convocatoria, top 15), que además retorna cuotas_pivot para exportar. Exporta territorial_hhi_por_convocatoria.csv, territorial_hhi_region_por_convocatoria.csv y territorial_cuotas_departamento.csv.

Fortalezas

- Usa correctamente el HHI normalizado a [0,1] sobre cuotas reales (cálculo delegado en src/analisis/territorial.hhi), con exclusión de valores NO REPORTADO.
- Triple perspectiva complementaria: top-10, evolución HHI (depto y región) y cuotas completas → buena triangulación del hallazgo de concentración.
- Las figuras anotan los valores exactos de HHI, facilitando la lectura sin tabla auxiliar.
- Salidas verificadas: las 3 figuras y los 3 CSVs existen en el clon en las rutas declaradas.

Debilidades

- fig_hhi_evolucion mezcla tipos en las coordenadas de anotación: el eje X se plotea con strings (astype(str)) pero annotate recibe str(int(row['ANO_CONVO_INT'])), dependiendo de la conversión categórica implícita de matplotlib; frágil ante cambios de versión y de difícil depuración.
- int(row['ANO_CONVO_INT']) lanzaría ValueError si hubiera NaN en el año (los filtros de hhi_por_convocatoria no garantizan limpieza total del año).
- El HHI por región depende de la columna NME_REGION_RES_PR derivada en Transformacion; el script no verifica que exista, fallando con KeyError poco descriptivo si la transformación cambia.
- Sin validación del tamaño muestral por año (el HHI de un año con pocos registros no es comparable) ni intervalo/estabilidad del índice.

Mejoras propuestas

- Anotar usando las mismas posiciones categóricas del plot (p. ej. range(len(data)) o plt.xticks con los strings) en lugar de pasar el string como coordenada.
- Proteger con .dropna() sobre ANO_CONVO_INT antes del gráfico y validar la presencia de NME_REGION_RES_PR con un mensaje claro.

- Exportar también `n_total` y `n_territorios` junto al HHI (ya los retorna `hhi_por_convocatoria`) para contextualizar la evolución.
- Añadir test de humo: HHI Bogotá+Antioquia $\approx 51\%$ de concentración reportado en el README.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (pandas, matplotlib, seaborn; `.venv` del clon vacío, `requirements.txt` incompleto). El input existe: `datos/tarea_join/investigadores consolidado.xlsx` vía `cargar_consolidado()`. No requiere duckdb. Con poetry install, corre de punta a punta; el único supuesto frágil es la columna `NME_REGION_RES_PR` producida por `Transformacion`.

¿Produce lo esperado? Sí, coincidencia total: existen `artifacts/sprint2_territorial/fig01_top_departamentos.png`, `fig02_hhi_evolucion.png`, `fig03_heatmap_cuotas.png` y `evidencias/territorial_hhi_por_convocatoria.csv`, `territorial_hhi_region_por_convocatoria.csv`, `territorial_cuotas_departamento.csv`, todos con los nombres exactos del docstring y del README, lo que confirma ejecución previa exitosa.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint2_territorial.py:82` — `ax.annotate` con `x = str(int(row['ANO_CONVO_INT']))`: se mezcla una cadena como coordenada de datos en un eje categórico (ploteado con `astype(str)`); funciona solo por la conversión unitaria implícita de matplotlib y es un punto de fallo frágil.
- `scripts/sprint2_territorial.py:82` — `int(row['ANO_CONVO_INT'])` lanza `ValueError` si el año es `NaN`; `hhi_por_convocatoria` filtra geo y año (dropna en sub), pero el `DataFrame` impreso/graficado aquí no garantiza ausencia de `NaN` en otras filas.
- `scripts/sprint2_territorial.py:123` — accede a `NME_REGION_RES_PR` sin verificar su existencia: si `Transformacion` renombra/elimina esa columna, el script cae con `KeyError` sin mensaje orientativo.

Recomendación de auditoría mantener — el análisis HHI es metodológicamente sólido y las salidas están verificadas; corregir las anotaciones del gráfico de evolución y blindar la dependencia de la columna de región.

4.5.6 `scripts/sprint3_grafo_interactivo.py`

Rol: orquestacion · **Líneas:** 72

Descripción funcional Genera dos visualizaciones interactivas HTML (Pyvis) del grafo de co-filiación institucional: el grafo completo y un subgrafo filtrado con `grado >= 2`. Usa `src/analisis/redes` (`construir_pares`, `construir_grafo`, `subgrafo_grado_minimo`, `generar_html`).

pyvis, metricas_grafo) sobre transformar(cargar_consolidado()). Salidas: hallazgos/sprint3-grafo_completo.html (docstring: 498 nodos) y hallazgos/sprint3-grafo_filtrado.html (146 nodos).

Análisis de la lógica main() construye los pares de doble afiliación (construir_pares sin filtro de año), el grafo global (construir_grafo con anio=None), el subgrafo de grado mínimo, imprime las métricas de ambos y genera los dos HTML con generar_html_pyvis, que en src/analisis/redes.py calcula betweenness (weight='weight'), escala tamaño por grado ponderado y color por betweenness. No exporta CSVs ni figuras estáticas; sus únicos artefactos son los dos HTML en hallazgos/.

Fortalezas

- Entrega un artefacto de alto valor de comunicación (grafo interactivo navegable) y lo versiona en hallazgos/ como entregable, no como figura estática.
- Delega toda la lógica de red en src/analisis/redes (normalización de instituciones, pesos, betweenness), manteniendo el script mínimo y legible.
- El filtrado por grado ≥ 2 elimina nodos hoja y produce un subgrafo interpretable; las métricas de ambos grafos se imprimen para contraste.
- Salidas verificadas: ambos HTML existen en el clon y están versionados en git.

Debilidades

- Los títulos de los HTML hardcodean '(2019)' (líneas 53 y 62) pero el grafo se construye con TODAS las convocatorias (construir_pares(df) sin filtro de año y construir_grafo con anio=None): la etiqueta contradice el dato real (co-filiación global 2013-2021) y es engañosa para quien abre el HTML.
- Los conteos '498 nodos' y '146 nodos' del docstring son literales de una corrida puntual; al re-ejecutar con datos actualizados el docstring queda desactualizado (drift documental).
- No genera evidencias tabulares (pares, métricas) propias: quien quiera verificar los 498/146 nodos debe re-ejecutar o abrir el HTML.
- Sin parámetro de año: no permite generar el grafo por convocatoria (p. ej. solo 2019) aunque la capa analítica lo soporta (construir_grafo acepta anio).

Mejoras propuestas

- Eliminar el año hardcodeado de los títulos o calcularlo dinámicamente (p. ej. '2013-2021' derivado del rango real del panel) para no contradecir los datos.
- Derivar los conteos de nodos del docstring dinámicamente o quitar los números literales.
- Exportar además evidencias/redes_grafo_metricas.csv (m completo y m filtrado) para trazabilidad de los 498/146 nodos.

- Añadir un flag `-anio` para reproducir la etiqueta 2019 de forma honesta (filtrando el grafo por convocatoria).

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias: pyproject declara `networkx ^3.2` y `pyvis ^0.3`, pero el `.venv` del clon está vacío (solo `pip/setuptools`) y `requirements.txt` no los lista. El input existe: `datos/tarea_join/investigadores_consolidado.xlsx` vía `cargar_consolidado()`. No requiere `duckdb`. Con `poetry install`, corre de punta a punta y sobrescribe los HTML existentes.

¿Produce lo esperado? Sí: `hallazgos/sprint3_grafo_completo.html` y `hallazgos/sprint3_grafo_filtrado.html` existen en el clon (rutas exactas del docstring y del README, que documenta 'hallazgos/ — HTMLs interactivos (grafos Pyvis)'), lo que confirma ejecución previa. Los conteos 498/146 nodos no son verificables por análisis estático sin abrir los HTML, pero la existencia y el versionado de ambos archivos es coherente con una corrida exitosa. El script no declara ni produce figuras PNG ni CSVs, y así es.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint3_grafo_interactivo.py:53` y `62` — títulos hardcodeados 'Co-filiación institucional — Grafo completo (2019)' y '(2019)': el grafo se construye con todas las convocatorias (`construir_pares(df)` línea 39 y `construir_grafo(pares)` línea 40 con `anio=None` en `src/analisis/redes.py:83-90`), por lo que la etiqueta 2019 es incorrecta y contradice el alcance real del grafo.
- `scripts/sprint3_grafo_interactivo.py:10-11` — el docstring fija '498 nodos' y '146 nodos' como constantes; si el dataset cambia, el docstring y los HTML pueden divergir sin ningún aviso.

Recomendación de auditoría mantener — el entregable interactivo es valioso y está versionado; corregir el año hardcodeado en los títulos (el bug más visible) y desacoplar los conteos del docstring para evitar drift documental.

4.5.7 `scripts/sprint3_redes.py`

Rol: `orquestacion` · **Líneas:** 134

Descripción funcional Análisis de redes de co-filiación institucional: grafo donde nodo = institución y arista = investigadores con doble afiliación, a nivel global (2013-2021) y por convocatoria. Usa `src/analisis/redes` (`construir_pares`, `construir_grafo`, `metricas_grafo`, `tabla_nodos`, `tabla_aristas`, `metricas_por_convocatoria`) sobre `transformar(cargar_consolidado())`. Salidas: 3 figuras PNG en `artifacts/sprint3_redes/` (distribución de grado, evolución de

aristas/nodos, top-20 instituciones por grado ponderado) y 4 CSVs en evidencias/ (pares, nodos, aristas, métricas por convocatoria).

Análisis de la lógica `main()` extrae los pares de doble afiliación (solo `INST_FILIA` con `'|'`), construye el grafo global con peso = investigadores compartidos, imprime métricas (nodos, aristas, densidad, componentes, hub), genera 3 figuras con funciones dedicadas y exporta 4 CSVs. `metricas_por_convocatoria` reutiliza `construir_grafo(pares, anio)` para la evolución anual. La normalización de instituciones (colapsar sedes/seccionales) ocurre en `src/analisis/redes.normalizar_institucion`.

Fortalezas

- Cobertura completa del análisis de red: grafo global, evolución por convocatoria, top instituciones y exportación tabular de nodos/aristas para reproducibilidad.
- El uso de grado ponderado (`weight`) en nodos y aristas refleja correctamente el conteo de investigadores compartidos.
- La normalización de instituciones (sedes, paréntesis, mayúsculas) está delegada y documentada en la capa analítica, evitando duplicados espurios.
- Salidas verificadas: las 3 figuras y los 4 CSVs existen en el clon en las rutas declaradas.

Debilidades

- Línea 102: `'instituciones mencionadas' = nunique(inst_a) + nunique(inst_b)` cuenta dos veces toda institución que aparece en ambas columnas; la cifra impresa está sobreestimada (debería ser la unión).
- `fig_distribucion_grado` dibuja una barra por nodo sin etiquetas de eje X legibles: con cientos de nodos la figura es poco informativa; un histograma log-log del grado sería más analítico.
- No reporta métricas de comunidad (modularidad, componentes gigantes) pese a imprimir `n_componentes`; el análisis de 'poder institucional' queda en grado/betweenness solamente.
- Depende de la columna `INST_FILIA` con separador `'|'`: si `Transformacion` cambia el formato (p. ej. `';`), `construir_pares` devuelve un `DataFrame` vacío sin advertencia.

Mejoras propuestas

- Corregir el conteo de instituciones con `pd.unique(concat(inst_a, inst_b))` o una unión de conjuntos.
- Sustituir las barras por nodo por un histograma de distribución de grado (escala log-log) y anotar el grado medio.
- Añadir detección de comunidad (`nx.community.louvain_communities`) y tamaño del componente gigante al resumen.

- Validar en `construir_pares` que `INST_FILIA` contenga '|' y advertir si el número de pares es 0 o anómalo.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (pandas, matplotlib, seaborn, networkx; .venv del clon vacío, requirements.txt no los lista pero pyproject sí). El input existe: `datos/tarea_join/investigadores_consolidado.xlsx` vía `cargar_consolidado()`. No requiere duckdb. Con poetry install, corre de punta a punta.

¿Produce lo esperado? Sí, coincidencia total: existen `artifacts/sprint3_redes/fig01_distribucion_grado.png`, `fig02_evolucion_aristas.png`, `fig03_top_instituciones.png` y `evidencias/redes_pares_cofiliacion.csv`, `redes_nodos_global.csv`, `redes_aristas_global.csv`, `redes_metricas_por_convocatoria.csv`, con los nombres exactos del docstring y del README; confirma ejecución previa exitosa.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint3_redes.py:102` — `pares['inst_a'].nunique() + pares['inst_b'].nunique()` sobreestima 'instituciones mencionadas' al sumar cardinalidades de columnas que comparten elementos; la cifra impresa en consola no es el número real de instituciones.
- `scripts/sprint3_redes.py:48-57` — `fig.distribucion_grado` genera una barra por nodo (`len(grados)` barras) sin eje X útil: con cientos de instituciones la figura se vuelve ilegible y aporta poco frente a un histograma de la distribución de grado.
- `scripts/sprint3_redes.py:100` — `construir_pares(df)` asume implícitamente el separador '|' en `INST_FILIA` (`src/analisis/redes.py:61-64`); si la transformación cambia el formato, el grafo queda vacío sin ningún warning.

Recomendación de auditoría mantener — es el análisis de redes estándar del observatorio con salidas verificadas y evidencia completa; corregir el doble conteo impreso y mejorar la figura de distribución de grado.

4.5.8 `scripts/sprint4_diversidad.py`

Rol: orquestacion · **Líneas:** 172

Descripción funcional Analiza las variables de diversidad (víctima de conflicto, grupo étnico, discapacidad) comparando a los investigadores reconocidos en la convocatoria 2021 con proporciones poblacionales de referencia DANE 2018 / RUV 2021. Usa `src/analisis/diversidad` (`cobertura_por_convocatoria`, `distribucion_categoria`, `comparar_dane`, `interseccional_genero_etnia`, `categoria_por_minoria`) sobre `transformar(cargar_consolidado())`. Salidas: 3 figuras PNG en `artifacts/sprint4_diversidad/` y 7 CSVs en `evidencias/` (uno más que los 6 del docstring).

Análisis de la lógica `main()` calcula la cobertura temporal de las 3 variables (solo se capturan desde 2021, hallazgo crítico documentado en el docstring), filtra `df_2021 = df[ANO_CONVO_INT == 2021]`, imprime las distribuciones excluyendo NO DISPONIBLE/NO REGISTRA, ejecuta `comparar_dane` (subrepresentación de minorías vs DANE/RUV con razón `pct_dane/pct_minc`) y genera 3 figuras + exporta 7 CSVs. `fig_categorias_por_minoria` apila la distribución de categorías académicas por grupo étnico minoritario.

Fortalezas

- El hallazgo metodológico central está bien documentado: las variables de diversidad solo existen desde la convocatoria 894 (2021), por lo que todo el análisis se acota honestamente a 2021.
- La comparación con DANE 2018 / RUV 2021 con razón de subrepresentación es metodológicamente apropiada y da un resultado de política pública accionable.
- Exporta evidencia completa: cobertura, distribuciones por variable, comparación DANE, cruce interseccional género×etnia y categorías por etnia.
- Salidas verificadas: las 3 figuras y los 7 CSVs existen en el clon.

Debilidades

- Drift docstring-código: el docstring declara 6 CSVs pero el código escribe 7 (agrega `diversidad_distribucion_victimas_2021.csv` en la línea 158, no listado).
- Bug metodológico heredado de `src/analisis/diversidad.comparar_dane` (líneas 78-80): el comentario afirma 'excluir NINGUN GRUPO ETNICO del denominador para ver minorías', pero el código solo excluye NO DISPONIBLE; quienes respondieron 'NINGUN GRUPO ETNICO' permanecen en el denominador, diluyendo los porcentajes de minorías y la razón de subrepresentación.
- Categorías académicas hardcodeadas con tildes ('INVESTIGADOR SÉNIOR', 'INVESTIGADOR EMÉRITO', línea 105): si la normalización de `NME.CLASIFICACION_PR` usa otro casing/acentuación, la figura puede salir vacía (el filtro `cats_disponibles` lo mitiga pero no avisa).
- El filtro 'NING' (línea 103) elimina cualquier valor que contenga esas tres letras (p. ej. un grupo 'NINGUNA' mal codificado) sin validación del vocabulario real.
- No hay intervalos de confianza ni tests de significancia para las diferencias vs DANE; la 'subrepresentación' se reporta sin incertidumbre.

Mejoras propuestas

- Actualizar el docstring para incluir `diversidad_distribucion_victimas_2021.csv` (o eliminar esa exportación si no es entregable).
- Corregir `comparar_dane` para excluir 'NINGUN GRUPO ETNICO' del denominador (como dice su comentario) y añadir un test que verifique denominador y suma de `pct`.

- Derivar las categorías de las columnas reales del DataFrame (p. ej. de un catálogo) en lugar de hardcodear strings con acentos.
- Añadir IC (binomial) a los porcentajes vs DANE y una nota sobre el tamaño muestral de 2021.
- Reportar n de 'NINGUN GRUPO ETNICO' y NO DISPONIBLE para que el lector vea la cobertura real del denominador.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (pandas, matplotlib, seaborn; .venv del clon vacío, requirements.txt incompleto). El input existe: datos/tarea_join/investigadores_consolidado.xlsx vía cargar_consolidado(). Requiere que Transformacion genere ANO_CONVO_INT == 2021 para la convocatoria 894 (no verificado en el script). No requiere duckdb ni datos/raw. Con poetry install, corre de punta a punta.

¿Produce lo esperado? Sí: existen artifacts/sprint4_diversidad/fig01_cobertura_temporal.png, fig02_comparacion_dane.png, fig03_categorias_por_minoria.png y los 7 CSVs de evidencias (diversidad_cobertura_por_convocatoria.csv, diversidad_distribucion_etnia_2021.csv, diversidad_distribucion_discapacidad_2021.csv, diversidad_distribucion_victimas_2021.csv, diversidad_comparacion_dane.csv, diversidad_interseccional_genero_etnia.csv, diversidad_categorias_por_etnia.csv). Coincide con el código; difiere del docstring en 1 CSV (victimas_2021 no declarado), lo que confirma que se ejecutó una versión del script y hubo drift documental.

Riesgo / Limitación

Bugs / problemas detectados:

- src/analisis/diversidad.py:78-80 — comentario vs código: 'excluir NINGUN GRUPO ETNICO del denominador' pero el código filtra solo 'NO DISPONIBLE' (etnia = df_2021[TXT_GRUPO_ETNICO != 'NO DISPONIBLE']; n_etnia_resp = len(etnia) incluye NINGUN GRUPO ETNICO): los pct_minciencias y las razones de subrepresentación de minorías quedan diluidos por el denominador inflado. Bug metodológico que afecta la figura fig02 y evidencias/diversidad_comparacion_dane.csv.
- scripts/sprint4_diversidad.py:105 — lista hardcodeada con tildes ['INVESTIGADOR SÉNIOR', 'INVESTIGADOR EMÉRITO']: depende del casing/acentuación exacta que produzca Transformacion en NME_CLASIFICACION_PR; si difiere, la figura 03 muestra categorías faltantes en silencio.
- scripts/sprint4_diversidad.py:103 — cat_etnia[...].str.contains('NING', case=False, na=False) es un filtro de subcadena: eliminaría también valores como 'NINGUNA...' si el vocabulario de la variable cambiara.
- scripts/sprint4_diversidad.py:158 — exporta diversidad_distribucion_victimas_2021.csv sin declararlo en el docstring (líneas 12-22), generando evidencia no documentada.

Recomendación de auditoría mejorar — el análisis es el más sensible metodológicamente (comparación con DANE/RUV) y tiene salidas verificadas, pero el denominador incorrecto en comparar_dane compromete la cifra central de

subrepresentación; corregir ese bug antes de publicar y alinear docstring-código.

4.5.9 scripts/sprint5_duckdb.py

Rol: orquestacion · **Líneas:** 193

Descripción funcional Extiende el modelo dimensional en observatorio.duckdb (creado por scripts/sprint2_duckdb.py) con fact_produccion, dim_producto, dim_grupo y la vista vw_investigador_x_produccion. Usa src/ingesta.cargar_produccion y src/analisis/produccion.normalizar_produccion. Entradas: datos/raw/produccion_grupos.csv y la DB previa. Salida: DB ampliada + reporte evidencias/duckdb_resumen_sprint5.txt con 5 consultas de validación.

Análisis de la lógica Carga y normaliza producción (IDs a Int64, extrae ANO_CONVO_INT), registra el DataFrame como tabla temporal 'prod_raw' y crea fact_produccion (una fila por $\text{id_persona_pd} \times \text{id_convocatoria} \times \text{id_producto_pd}$), dim_producto y dim_grupo con SELECT DISTINCT. Crea la vista vw_investigador_x_produccion con LEFT JOIN fact_clasificacion fc ON $\text{f.id_persona_pd} = \text{fc.id_persona_pr}$ AND $\text{f.id_convocatoria} = \text{fc.id_convocatoria}$, definiendo $\text{es_reconocido} = (\text{fc.id_persona_pr} \text{ IS NOT NULL})$. Luego ejecuta 5 consultas: productos por convocatoria; cobertura por año con pct_reconocido; productividad por categoría = $\text{COUNT}(*)/\text{COUNT}(\text{DISTINCT id_persona_pd})$ unida a dim_categoria (cat.categoria_normalizada / orden_normalizado); top 10 grupos por n_productos; y tipologías dominantes con ventana $\text{SUM}(\text{COUNT}(*)) \text{ OVER } ()$ para el porcentaje. El reporte se escribe concatenando los to_string() de cada DataFrame.

Fortalezas

- Modelado dimensional limpio: separa hechos (fact_produccion) de dimensiones (dim_producto, dim_grupo) y expone una vista lista para análisis cruzados, reutilizable por otros scripts o la app.
- Idempotente: DROP TABLE/VIEW IF EXISTS antes de cada CREATE, permitiendo re-ejecución sin corrupción acumulada.
- Validación integrada al pipeline: imprime conteos de filas por tabla y ejecuta consultas de negocio que verifican el resultado en el mismo run.
- Los artefactos declarados existen en el clon y el reporte reproduce exactamente las métricas esperadas (ver produce_esperado).

Debilidades

- La vista asume que el id_convocatoria del dataset de producción es el mismo dominio que el de fact_clasificacion; si no coinciden (p. ej. tipos str vs int o convocatorias de grupos

vs de clasificación), el LEFT JOIN degrada silenciosamente a `es_reconocido = False` sin ningún aviso.

- No reporta la cobertura del join (cuántos productos quedan sin clasificación asociada), clave para interpretar el `pct_reconocido`.
- La definición de 'reconocido' aquí es 'reconocido en esa misma convocatoria', distinta de la usada en `sprint5_produccion.py` (cualquier convocatoria), lo que produce cifras de cobertura no comparables entre scripts del mismo sprint.
- No hay validación de tipos entre `id_persona_pd` (Int64) y `id_persona_pr` de `fact_clasificacion` (depende de `sprint2`); un desajuste de tipo en DuckDB rompe el join en runtime, no en validación.
- La modificación de la DB no es transaccional: si falla a mitad del script, la DB queda con un subconjunto de objetos nuevos.

Mejoras propuestas

- Agregar validación post-join: COUNT productos sin match en `fact_clasificacion` y advertir si supera un umbral.
- Añadir CAST/verificación explícita de tipos de `id_persona` e `id_convocatoria` antes de construir la vista.
- Escribir un test de humo que verifique `n_investigadores ≈ 77.237` y que `prom_productos` por categoría esté en el rango esperado (Junior ~30, Asociado ~65, Senior ~121).
- Parametrizar `DB_PATH` y permitir `-solo-validacion` para no reconstruir tablas.
- Documentar el supuesto de que `fact_produccion.id_convocatoria` es el id de la convocatoria de clasificación del grupo.

Ejecutabilidad NO ejecutable tal cual con el clon: falta `datos/processed/observatorio.duckdb` (solo hay `.gitkeep`; requiere ejecutar `scripts/sprint2.duckdb.py` antes, que a su vez necesita el raw de investigadores) y `datos/raw/produccion_grupos.csv` (`gitignored`, ~1.2 GB). Requiere la dependencia `duckdb` instalada. El resto (`pathlib`, `sys`) es estándar. Con el dataset y la DB presentes, el script corre de punta a punta.

¿Produce lo esperado? Sí: `evidencias/duckdb_resumen_sprint5.txt` existe y contiene las 5 consultas. La consulta `productividad_por_categoria` reproduce Junior 29.99, Asociado 65.36, Senior 121.49, Emérito 33.75 (redondeando: 30/65/121), que es exactamente la métrica que los investigadores buscaban. La cobertura por año va de 51.88 % (2013) a 63.34 % (2021). No hay discrepancias entre lo declarado y lo generado.

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint5_duckdb.py:155-157` — el LEFT JOIN a `fact_clasificacion` no valida que `f.id_`

convocatoria = fc.id_convocatoria tenga coincidencias; si los dominios de convocatoria difieren (producción vs clasificación), toda la columna es_reconocido queda en False y la cobertura reportada (51-63 %) sería espuria sin ningún error visible.

- scripts/sprint5_duckdb.py:59 — COUNT(*)::DOUBLE es sintaxis válida en DuckDB, pero el SUM(CASE WHEN es_reconocido THEN 1 ELSE 0 END) de la consulta cobertura_reconocimiento devuelve tipo DOUBLE (visible como '123683.0' en el reporte) frente a COUNT(*) entero; inconsistencia de tipos menor en el output.
- scripts/sprint5_duckdb.py:162 — valida COUNT(*) de fact_produccion/dim_producto/dim_grupo pero nunca valida la vista vw_investigador_x_produccion (el objeto más usado después).
- scripts/sprint5_duckdb.py:96-136 — DROP TABLE IF EXISTS sin transacción; una excepción entre DROPs deja la DB parcialmente actualizada (riesgo de estado inconsistente).

Recomendación de auditoría mantener — es el núcleo del modelo DuckDB del sprint 5, bien estructurado y con resultados verificados; requiere solo las mejoras de validación de join y tipos para blindar la cobertura reportada.

4.5.10 scripts/sprint5_produccion.py

Rol: orquestacion · **Líneas:** 251

Descripción funcional Análisis integral de producción de grupos: cobertura de reconocimiento (autores únicos y por convocatoria), productividad por categoría, brecha de género × gran área OCDE, concentración territorial, mix de tipologías y reconocidos sin producción. Usa src/ingesta (cargar_consolidado, cargar_produccion), src/Transformacion.transformar y 10 funciones de src/analisis/produccion. Entradas: investigadores consolidados + producción Socrata; salidas: 6 figuras PNG y 8 evidencias (7 CSVs + 1 JSON).

Análisis de la lógica Flujo 1→8: (1) carga y transforma; (2) cobertura_autores_unicos devuelve dict de solape de IDs; (3) cobertura_reconocidos agrupa por ANO_CONVO_INT marcando es_reconocido = ID_PERSONA_PD ∈ set(ID_PERSONA_PR del padrón EN CUALQUIER convocatoria); (4) productividad_por_categoria agrupa producción por (persona, convocatoria), hace left merge con el padrón y agrega por (año, NME_CLASIFICACION_PR) media/mediana/pct con ≥1 producto; (5) brecha_productividad_genero pivotea promedios F/M por área OCDE; (6) productividad_territorial calcula % productos vs % investigadores y ratio; (7) mix_tipologias y tipologias_por_categoria (porcentajes por año/categoría) y reconocidos_sin_produccion (intersección de sets por convocatoria); (8) genera 6 figuras matplotlib/seaborn y exporta CSVs y JSON.

Fortalezas

- Orquestación clara y progresiva [1/8]...[8/8] con impresión de resultados intermedios (auditable en consola).
- Separación limpia entre lógica de análisis (src/analisis/produccion.py) y presentación (figuras), facilitando testing y reuso.
- Salidas exhaustivas y documentadas en el docstring; las 6 figuras y las 8 evidencias existen en el clon.
- Incluye indicador de calidad de captura (reconocidos sin producción) y métricas robustas (mediana además de media).

Debilidades

- Definición de 'reconocido' inconsistente con sprint5_duckdb.py: cobertura_reconocidos marca como reconocido a quien lo fue en cualquier convocatoria (src/analisis/produccion.py:63-65), mientras la vista DuckDB exige la misma convocatoria; las coberturas resultantes (63.34 % DuckDB vs. CSV) no son comparables.
- Carga ~3.16M filas de producción en pandas y hace merges/grupos completos en memoria: alto consumo de RAM y sin streaming ni particionado.
- normalizar_produccion convierte IDs a numérico Int64 (src/analisis/produccion.py:38-40): cualquier id con ceros a la izquierda se pierde, riesgo silencioso de cruces falsos si el padrón conserva el formato string.
- El pct de la figura 1 se dibuja a total+5000 unidades fijas, escala que no se adapta a convocatorias pequeñas.
- No hay validación de que el join producción↔padrón haya encontrado coincidencias (una caída de cobertura se reporta como hallazgo, no como error).

Mejoras propuestas

- Unificar la definición de 'reconocido' entre scripts (recomendado: misma convocatoria) y documentarla en un único lugar.
- Validar tipos y solape de IDs antes de los merges (advertir si el match < umbral).
- Reducir memoria: leer solo columnas necesarias de producción o procesar por convocatoria.
- Añadir tests sobre produccion_productividad_por_categoria.csv (p. ej. productos_promedio Senior > Junior en todas las convocatorias).
- Hacer la posición de la etiqueta de % proporcional al total (ax.text con offset relativo).

Ejecutabilidad NO ejecutable tal cual con el clon: cargar_produccion() lanza FileNotFoundError porque datos/raw/produccion_grupos.csv está gitignored y ausente. cargar_consolidado() sí funciona (cae a datos/tarea_join/investigadores_consolidado.xlsx). Dependencias: pandas, numpy, matplotlib, seaborn (todas estándar del proyecto). Con el CSV de producción en datos/raw, el script corre completo.

¿Produce lo esperado? Sí: las 6 figuras de `artifacts/sprint5-produccion/` y las 8 evidencias (`produccion_cobertura_por_convocatoria.csv`, `produccion_productividad_por_categoria.csv`, `produccion_brecha_genero.csv`, `produccion_concentracion_territorial.csv`, `produccion_mix_tipologias.csv`, `produccion_tipologias_por_categoria.csv`, `produccion_reconocidos_sin_produccion.csv`, `produccion_resumen_cobertura.json`) existen. El CSV de productividad muestra Junior 9.9-17.9 y Senior 39.0-50.1 productos promedio según convocatoria, coherente con el hallazgo 'Senior produce más'. Nota: la cifra 30/65/121 mencionada en el encargo corresponde a la métrica acumulada de `sprint5_duckdb.py`, no a este CSV (que es por convocatoria).

Riesgo / Limitación

Bugs / problemas detectados:

- `src/analisis/produccion.py:63-65` (invocado desde `sprint5-produccion.py:203`) — `cobertura_reconocidos` marca `es_reconocido` sin condicionar la convocatoria: un producto de 2013 firmado por alguien reconocido solo en 2021 cuenta como 'firmado por reconocido', sobreestimando la cobertura frente a la definición por convocatoria del DuckDB (documentado en el docstring de la función como decisión, pero no advertido en el script orquestador).
- `scripts/sprint5-produccion.py:75-78` — la etiqueta de porcentaje se posiciona en `total+5000` y el bucle itera `zip(n_reconocidos, n_productos, pct_reconocidos)`: si `n_productos < 5000` la etiqueta queda fuera del rango visible del eje (cosmético).
- `scripts/sprint5-produccion.py:243-244` — `import json` dentro de `main()` (estilo menor; debería ir al tope con los demás imports).
- `src/analisis/produccion.py:38-40` — `pd.to_numeric(errors='coerce')` convierte IDs no numéricos a NA silenciosamente; si `ID_PERSONA_PD` tiene formatos mixtos, filas completas se pierden del análisis sin reporte.

Recomendación de auditoría **mantener** — es el script central del sprint 5, con salidas verificadas y documentadas; las mejoras prioritarias son unificar la definición de 'reconocido' entre scripts y validar la calidad del join antes de interpretar coberturas.

4.5.11 `scripts/sprint6_geografia_institucional.py`

Rol: orquestacion · **Líneas:** 302

Descripción funcional Análisis geográfico institucional: cruza el departamento de residencia del investigador con el departamento de la sede de su institución principal (vía el mapping de la tabla maestra IES), respondiendo la pregunta del director sobre dónde se forman/afilian los investigadores de departamentos como Vichada (¿en Meta o Bogotá?). Usa `src/ingesta.cargar_consolidado`, `src/Transformacion.transformar` y `evidencias/mapping_inst_filial_to_ies.csv`. Salidas: 3 figuras y 3 CSVs.

Análisis de la lógica `asignar_institucion_principal()`: toma la primera institución de `INST_FILIA` (split por '|'), la normaliza (`upper/strip`) y hace lookup en el mapping por `inst_filia_str` → (`nombre_canonico`, `sigla`, `departamento_sede`, `naturaleza`); normaliza departamentos con `NORM_DPTO` + eliminación de acentos (`unicodedata`); genera `DPTO_INSTITUCION` y `DPTO_RESIDENCIA` canónicos. `tabla_flujos()`: `nunique` de `ID_PERSONA_PR` por par (`residencia`, `institución`), excluyendo `NO_MAPEADA/SIN_FILIA`. `resumen_por_residencia()`: por departamento, total/local/fuera, % local y destino externo principal. Figuras: heatmap top 15×12 de flujos, % local por departamento (top 25) y destino de departamentos pequeños (`VICHADA`, `GUAINIA`, etc.).

Fortalezas

- Responde exactamente la pregunta del director con un indicador accionable (% local vs. destino externo principal) y foco explícito en departamentos pequeños.
- Normalización consistente de departamentos (`NORM_DPTO` + quita acentos) que unifica el formato del padrón ('BOGOTÁ, D. C.') con el de la tabla maestra ('Bogotá D.C.').
- Exporta el flujo completo (pares `residencia`→`institución` con conteos) y no solo resúmenes, permitiendo reproducir el análisis.
- Artefactos verificados: 3 figuras y 3 CSVs existen; el resumen muestra resultados coherentes (BOGOTA 91.8 % local, ANTIOQUIA 89.2 %, destinos externos hacia BOGOTA).

Debilidades

- El lookup depende de coincidencia exacta entre `INST_FILIA` (tras `upper/strip`, SIN quitar acentos) y las claves de mapping (mayúsculas sin acentos): un padrón con tildes ('UNIVERSIDAD TECNOLÓGICA') caería a `NO_MAPEADA` silenciosamente; el script solo reporta el % global de mapeados, no por string.
- Tomar la primera institución cuando `INST_FILIA` trae varias (separadas por |) es una simplificación no documentada como limitación.
- `EXTRANJERO/EXTERIOR/NO IDENTIFICADO` no se excluyen de los flujos y pueden ocupar lugares en los top (heatmap/top destinos).
- Los tops (15×12, top 25, umbral) están hardcodeados sin parámetros CLI.

Mejoras propuestas

- Quitar acentos (`unicodedata`) a `INST_FILIA` antes del lookup y reportar el % de match por grupo de error (`NO_MAPEADA` con top-10 strings).
- Excluir `EXTRANJERO/EXTERIOR/NO IDENTIFICADO` de los flujos o tratarlos como categoría aparte en las figuras.
- Documentar la regla de 'primera institución' y evaluar asignar por frecuencia/antigüedad en vez de orden textual.

- Parametrizar top_n y umbrales por CLI.

Ejecutabilidad SÍ, ejecutable tal cual con el clon: cargar_consolidado() resuelve vía datos/tarea_join/investigadores_consolidado.xlsx, el mapping existe (evidencias/mapping_inst_filia.to_ies.csv) y solo requiere pandas, numpy, matplotlib y seaborn.

¿Produce lo esperado? SÍ: artifacts/sprint6-geografia/{fig-heatmap-residencia-institucion,fig_pct.local_por_dpto,fig_top_destinos-pequenos}.png y evidencias/{geografia_flujo_completo,geografia_resumen_por_dpto-residencia,geografia_top_flujos}.csv existen. El resumen confirma el hallazgo buscado (Bogotá 91.8% local, departamentos pequeños con destinos externos), y geografia_flujo_completo.csv alimenta el Sankey territorial.

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint6-geografia_institucional.py:81-88 — INST_FILIA solo se pasa a upper/strip, sin quitar acentos, mientras las claves del mapping (generadas desde MAPEO y el dump) son mayúsculas sin acentos; cualquier tilde en el dato fuente rompe el match y degrada a NO_MAPEADA sin aviso específico.
- scripts/sprint6-geografia_institucional.py:88 — el reemplazo cubre ", 'NAN' y 'NO REPORTADO', pero si el padrón usa otro marcador de ausencia (p. ej. 'NO DISPONIBLE' o 'SIN INFORMACION') quedaría como institución ficticia y se contaría en los flujos.
- scripts/sprint6-geografia_institucional.py:118-119 — se excluyen NO_MAPEADA/SIN_FILIA pero no 'EXTRANJERO'/'EXTERIOR'/'NO IDENTIFICADO': los flujos internacionales/desconocidos pueden dominar los top y desdibujar el mensaje territorial.
- scripts/sprint6-geografia_institucional.py:141-145 — fuera.iloc[0] depende de que el grupo mantenga el orden descendente global por n_investigadores; correcto por construcción (flujo ya ordenado), pero frágil si tabla_flujos se reordenara.

Recomendación de auditoría mantener — es el análisis estrella del sprint 6 (pregunta directa del director, evidencia verificada y reutilizable); requiere endurecer la normalización de strings y el manejo de categorías no territoriales para que la cobertura de mapeo no degrade silenciosamente.

4.5.12 scripts/sprint6_ocde_composicion.py

Rol: orquestacion · **Líneas:** 183

Descripción funcional Responde dos preguntas del director sobre la producción nacional: qué rama del conocimiento produce más y cómo se compone por tipo de medición (nuevo conocimiento, formación, apropiación), segmentando por gran área OCDE. Cruza producción (Socrata 33dq-ab5a) con el padrón por (ID_PERSONA_PD = ID_PERSONA_PR, ID_

CONVOCATORIA) mediante inner join. Usa src/ingesta, src/Transformacion.transformar y src/analisis/produccion.normalizar_produccion. Salidas: 3 figuras y 3 CSVs.

Análisis de la lógica cruzar(): merge inner de producción con (ID_PERSONA_PR→ID_PERSONA_PD, ID_CONVOCATORIA, NME.GRAN_AREA_PR) y filtra 'NO REGISTRA'/'NO REPORTADO'. Volumen = groupby área con size(); productividad = n_productos (filas producto-por-convocatoria) / n_investigadores (nunique de ID_PERSONA_PR del padrón en esa área, todas las convocatorias); composición = groupby (área, tipo medición) con pct dentro del área. Tres figuras: barras horizontales de volumen, barras apiladas de composición con PALETA.TIPO, y barras de productividad.

Fortalezas

- Responde directamente la pregunta del director con una lectura clara (volumen por área, composición por tipo de medición).
- Paleta de tipos de medición explícita y consistente con la presentación.
- Filtra áreas no informadas y excluye del denominador de investigadores a áreas sin producción (evita ratios 0).
- Artefactos verificados: 3 figuras y 3 CSVs (ocde_volumen, ocde_productividad, ocde_composicion) existen en el clon.

Debilidades

- El denominador de productividad mezcla unidades: n_productos cuenta filas producto × convocatoria (una persona puede sumar productos de 5 convocatorias), mientras n_investigadores es nunique de personas en TODO el padrón → infla la productividad por área hasta ~5× frente a un promedio por persona-convocatoria.
- El inner join reduce la 'producción nacional' a productos firmados por reconocidos: el título '¿Qué rama produce más en Colombia?' es engañoso si se interpreta como toda la producción del país.
- No reporta cuántos productos quedan fuera del cruce (autores no reconocidos o convocatorias sin match), pese a imprimir solo len(df) del resultado.
- La productividad se calcula sin controlar categoría ni año: un área con muchos Senior en 2021 (convocatoria más productiva) se beneficia del efecto composición.

Mejoras propuestas

- Calcular productividad como productos por (persona, convocatoria, área) y luego promediar por persona-área, o dividir por persona-convocatoria únicos.
- Reportar la pérdida del inner join (n productos sin match / por qué).

- Renombrar la métrica a 'producción de investigadores reconocidos por área' y añadir caveat en el título.
- Controlar por año y categoría (p. ej. productividad estandarizada) antes de afirmar que un área produce más.

Ejecutabilidad NO ejecutable tal cual con el clon: cargar_produccion() falla por ausencia de datos/raw/produccion_grupos.csv (gitignored). Requiere pandas, matplotlib y seaborn. Con el dataset de producción presente, el resto del pipeline (padrón vía datos/tarea.join) está disponible.

¿Produce lo esperado? Sí: artifacts/sprint6_ocde/{fig_volumen_por_area,fig_composicion_tipo_por_area,fig_productividad_por_area}.png y evidencias/{ocde_volumen_por_area,ocde_productividad_por_area,ocde_composicion_por_area}.csv existen y son coherentes con la estructura del script.

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint6_ocde_composicion.py:151-154 — n_productos/n_investigadores mezcla granularidades: numerador filas (persona×convocatoria×producto) y denominador personas únicas en todo el periodo; si una persona aparece en varias convocatorias, su producción completa se divide por 1, inflando sistemáticamente la productividad de todas las áreas.
- scripts/sprint6_ocde_composicion.py:62 — how='inner' descarta en silencio los productos sin autor reconocido o con ID_CONVOCATORIA no coincidente; el print de [2/5] solo muestra len(df) del cruce y no la pérdida frente a len(prod).
- scripts/sprint6_ocde_composicion.py:64 — el filtro solo cubre 'NO REGISTRA'/'NO REPORTADO'; variantes como 'SIN INFORMACIÓN' o NaN en NME_GRAN_AREA_PR pasarían y crearían una categoría basura en volumen/composición.

Recomendación de auditoría mejorar — el análisis responde la pregunta del director y los entregables existen, pero la métrica de productividad por área es metodológicamente inconsistente (unidades mezcladas) y debe corregirse antes de usarse como hallazgo.

4.5.13 scripts/sprint6_sankey_categoria.py

Rol: orquestacion · **Líneas:** 184

Descripción funcional Genera diagramas de Sankey de transición de categoría (Junior/Asociado/Senior/Emérito + 'Desaparece') para cada par de convocatorias consecutivas 2013→2021 (5 pares). Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/analisis/longitudinal (construir_panel, comparar_periodo, matriz_transicion, ORDEN_

CATEGORIAS). Salidas: 5 HTML interactivos + 5 PNG estáticos y evidencias/sankey_transiciones.csv con todos los flujos.

Análisis de la lógica Construye el panel longitudinal por convocatoria, obtiene los años ordenados (2013, 2014, 2015, 2017, 2019, 2021) y sus pares consecutivos. Por par: `comparar_periodo(panel, a, b) → matriz_transicion(comp, incluir_desaparece=True)` devuelve (conteos, probs); reordena filas por `ORDEN_CATEGORIAS`; `nodos = categoría+año` (izquierda `t`, derecha `t+1`) con colores de `COLOR_CAT`; `flujos = celdas > 0` con color `rgba` del origen; figura `go.Sankey` con `arrangement='snap'` y `hovertemplate`. Exporta HTML (plotly CDN) y PNG vía `write_image`, y consolida todos los flujos en `sankey_transiciones.csv` con clasificación `Sube/Baja/Mantiene/Desaparece` (`_etiqueta_transicion`) y resumen por periodo.

Fortalezas

- Respetar un orden canónico de categorías (`ORDEN_CATEGORIAS`) para filas y columnas, evitando ordenamientos arbitrarios en el diagrama.
- Los flujos se exportan en formato largo (`periodo, origen, destino, n`), lo que permite reproducir el análisis sin el Sankey.
- Estado 'Desaparece' explícito: mide retención/abandono real entre convocatorias.
- Artefactos verificados: los 5 pares de HTML/PNG y `sankey_transiciones.csv` (76 líneas) existen en el clon.

Debilidades

- `fig.write_image` exige `kaleido`, dependencia no declarada en el docstring ni en requirements (si falta, el script muere en la primera iteración).
- Los HTML usan `include_plotlyjs='cdn'`: sin conexión a internet no renderizan.
- Cualquier categoría del panel que no esté en `ORDEN_CATEGORIAS` se excluye silenciosamente de `fila_orden` (línea 54) y desaparece del Sankey sin warning, con la consiguiente pérdida de investigadores del total.
- El título suma `total_t` incluyendo los que 'Desaparecen' (correcto como total en `t`, pero puede confundir si se lee como flujo completo).
- Parámetros (`top, umbral, altura`) hardcodeados sin interfaz CLI.

Mejoras propuestas

- Declarar `kaleido` en requirements/README o manejar su ausencia con un `try/except` que solo escriba HTML.
- Validar que todas las categorías del panel estén en `ORDEN_CATEGORIAS` y advertir/fallar si no.

- Escribir los PNG con un backend alternativo (p. ej. captura de HTML con playwright) o documentar el requisito kaleido.
- Exportar también las probabilidades de transición (matriz) como evidencia complementaria.

Ejecutabilidad Sí en cuanto a datos desde el clon: `cargar_consolidado()` resuelve vía `datos/tarea_join/investigadores_consolidado.xlsx`. Requiere `plotly` Y `kaleido` instalados; sin `kaleido`, `write_image` (línea 146) lanza error y aborta. Con ambas librerías el script corre completo.

¿Produce lo esperado? Sí: `artifacts/sprint6_sankey/sankey_2013_2014.{html,png}` ... `sankey_2019_2021.{html,png}` y `evidencias/sankey_transiciones.csv` existen. El CSV muestra flujos coherentes (2013-2014: Junior→Junior 1967, Junior→Desaparece 2823; Senior→Senior 591, etc.) y 5 periodos (2013-2014, 2014-2015, 2015-2017, 2017-2019, 2019-2021).

Riesgo / Limitación

Bugs / problemas detectados:

- `scripts/sprint6_sankey_categoria.py:52-55` — `fila_orden` filtra las categorías presentes en `ORDEN_CATEGORIAS`, pero si `construir_panel` produce una categoría no mapeada (p. ej. variante con acento 'INVESTIGADOR SÉNIOR' vs 'Senior'), esa fila se elimina del Sankey y del conteo sin ningún mensaje; el total del título quedaría subestimado.
- `scripts/sprint6_sankey_categoria.py:105` — `total_t = conteos.sum().sum()` incluye la columna 'Desaparece'; el texto del título 'investigadores reconocidos en t' es correcto, pero si `fila_orden` ya excluyó categorías, `total_t` no coincide con el padrón de `t`.
- `scripts/sprint6_sankey_categoria.py:146-147` — `write_image` sin comprobación de `kaleido`: dependencia implícita que rompe la ejecución en entornos mínimos.

Recomendación de auditoría mantener — produce el entregable de transiciones solicitado con evidencia verificada; la prioridad es declarar `kaleido` y validar el mapeo de categorías para no perder investigadores silenciosamente.

4.5.14 `scripts/sprint6_sankey_territorial.py`

Rol: orquestacion · **Líneas:** 135

Descripción funcional Materializa el Sankey territorial residencia del investigador → departamento de su institución, usando como entrada directa `evidencias/geografia_flujo_completo.csv` (generado por `sprint6_geografia_institucional.py`), sin recargar el padrón. Respuesta visual a la pregunta del director sobre los flujos de departamentos pequeños hacia Meta/Bogotá. Salidas: `sankey_territorial.html` (interactivo) y `sankey_territorial.png` (estático).

Análisis de la lógica `construir_sankey()`: selecciona top 12 departamentos de residencia por volumen total, top 10 de institución y flujos con `n_investigadores ≥ 15`; construye nodos 'X (res.)' e 'Y (inst.)' con colores de PALETA por residencia (nodos de institución en gris); flujos con `alpha 0.40` si el flujo es local (`residencia == institución`) y `0.65` si es transversal; figura `go.Sankey` con `arrangement='snap'` y `hovertemplate` con formato de miles. Exporta HTML (`plotly CDN`) y PNG vía `write_image` a 2x.

Fortalezas

- Diseño de pipeline limpio: consume el CSV ya generado por el script de geografía (una sola fuente de verdad, sin duplicar la lógica de cruce).
- Parámetros de saturación explícitos (`top_res`, `top_inst`, `umbral_min`) que evitan un diagrama ilegible con miles de flujos.
- Distinción visual local vs. transversal (`alpha`) y paleta estable.
- Artefactos verificados: `sankey_territorial.html` y `sankey_territorial.png` existen en el clon.

Debilidades

- `write_image` requiere `kaleido` (dependencia no declarada): el PNG falla en entornos sin esa librería aunque el HTML se escriba bien.
- HTML con `include_plotlyjs='cdn'`: requiere internet para renderizar.
- Los tops y el umbral están fijos en `main()` sin interfaz CLI para explorar variantes.
- No exporta el subconjunto de flujos usados en el diagrama (CSV), limitando la reproducibilidad del gráfico exacto.
- Depende de que `geografia_flujo_completo.csv` use los strings normalizados; si ese script cambia de normalización, este se rompe sin mensaje claro.

Mejoras propuestas

- Declarar `kaleido` o envolver `write_image` en `try/except` para degradar a solo HTML con aviso.
- Exportar el CSV de los flujos filtrados usados en la figura.
- Parametrizar `top_res/top_inst/umbral_min` por `argparse` para iterar rápido.
- Añadir validación de columnas esperadas (`DPTO.RESIDENCIA`, `DPTO.INSTITUCION`, `n_investigadores`) al leer el CSV.

Ejecutabilidad Sí, ejecutable tal cual con el clon: la entrada `evidencias/geografia_flujo_completo.csv` existe; requiere `plotly` y `kaleido` instalados (sin `kaleido` falla solo la escritura del PNG). No depende de `datos/raw` ni del padrón.

¿Produce lo esperado? Sí: artifacts/sprint6_sankey/sankey_territorial.html y sankey_territorial.png existen, completando el set de 6 sankeys del sprint 6 (5 de categoría + 1 territorial). La lógica produce exactamente el flujo solicitado (residencia → institución con umbral de ruido).

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint6_sankey_territorial.py:125-129 — write_image sin comprobación de kaleido: dependencia implícita; en un entorno sin kaleido el script aborta tras haber generado el HTML (comportamiento no degradable).
- scripts/sprint6_sankey_territorial.py:69-70 — top_residencias.index(row['DPTO_RESIDENCIA']) asume que toda fila de sub está en la lista top; correcto por el filtro isin previo (línea 56-58), pero si top_residencias tuviera duplicados (no los tiene, viene de nlargest), el index() devolvería la primera aparición.
- scripts/sprint6_sankey_territorial.py:56-58 — el filtro por umbral_min ≥ 15 es absoluto, no proporcional al total del departamento: un flujo pequeño de VICHADA (dpto de pocos investigadores) puede quedar excluido justo donde el director quiere verlo; convendría umbral relativo por origen.

Recomendación de auditoría mantener — script simple, enfocado y con artefactos verificados que completan el entregable del director; mejoras menores (kaleido declarado, CSV de flujos filtrados, umbral relativo para departamentos pequeños) aumentan su robustez y utilidad.

4.5.15 scripts/sprint6_tabla_maestra_ies.py

Rol: orquestacion · **Líneas:** 635

Descripción funcional Construye el borrador de la tabla maestra canónica de IES a partir de evidencias/instituciones_unicas_raw.csv (dump de strings únicos de INST_FILIA con frecuencias), aplicando un diccionario MAPEO hardcodeado de ~250 claves string → (nombre_canonico, sigla, departamento_sede, naturaleza). Solo usa pandas, sin tocar el padrón ni producción. Salidas: tabla_maestra_ies.csv, mapping_inst_filia_to_ies.csv e ies_pendientes_revision.csv.

Análisis de la lógica aplicar_mapeo(row): si inst_filia_str $\in$ MAPEO devuelve la tupla canónica con estado 'asignado'; si no, NA con estado 'pendiente_revision'. main(): lee el dump, aplica el mapeo fila a fila (apply axis=1), calcula cobertura sobre apariciones (asignadas/total), agrupa las asignadas por (nombre_canonico, sigla, naturaleza) con agregados (departamentos únicos, apariciones, variantes) ordenadas por apariciones desc, y exporta los 3 archivos, imprimiendo cobertura y top 20.

Fortalezas

- Borrador transparente y auditable: el mapeo completo es legible, con comentarios por pasadas y objetivo de cobertura (>90%).
- Reporta cobertura real sobre apariciones del padrón y separa explícitamente las entradas pendientes de revisión.
- Cero dependencias externas más allá de pandas y ejecutable con el clon (el input `instituciones_unicas_raw.csv` existe, 2763 filas).
- Decisiones de negocio explícitas: una IES por sede/razón social con `departamento_sede` por ubicación física.

Debilidades

- Mapeo 100 % manual y hardcodeado (~250 de 2763 strings): frágil ante variantes nuevas (acentos, guiones, 'SAS', 'LTDA') y costoso de mantener; es una solución de 'borrador', no un componente de datos.
- La columna 'sedes_registradas' en realidad cuenta departamentos distintos (nunique de `departamento_sede`), no sedes físicas: nombre engañoso.
- Coincidencia exacta por mayúsculas-sin-acentos: si el dump cambia de normalización (p. ej. conservara tildes), todo el mapeo se rompe en silencio y la cobertura cae sin error.
- Variantes duplicadas sin mapear (p. ej. 'UNIVERSIDAD NACIONAL DE COLOMBIA' sin sede) asumen Bogotá por defecto sin validación.
- `apply(axis=1)` con función Python es lento, aunque tolerable para ~2.8k filas.

Mejoras propuestas

- Externalizar el MAPEO a un CSV/JSON versionado (fuente de datos, no código) con validación de unicidad de claves.
- Añadir normalización difusa o reglas (contiene 'UNIVERSIDAD NACIONAL DE COLOMBIA' → UNAL) para cubrir variantes no listadas, reportando la confianza.
- Renombrar 'sedes_registradas' a 'departamentos_sede' o contar sedes reales.
- Añadir un test que falle si la cobertura baja de un umbral (p. ej. 85 %) ante cambios del dump.

Ejecutabilidad SÍ, ejecutable tal cual con el clon: solo necesita pandas y evidencias/instituciones_unicas_raw.csv (existente, columnas `inst_filia_str`, `n_apariciones`, `pct`, `pct_acum` — coinciden con lo que lee el script). No depende de datos/raw ni de observatorio.duckdb.

¿Produce lo esperado? SÍ: evidencias/tabla_maestra_ies.csv, evidencias/mapping_inst_filia_to_ies.csv e evidencias/ies_pendientes_revision.csv existen. El mapping tiene las columnas

esperadas (inst_filia_str, nombre_canonico, sigla, departamento_sede, naturaleza, estado) y es el insumo que consumen sprint6_geografia_institucional.py y la app.

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint6_tabla_maestra_ies.py:607 — 'sedes_registradas' = nunique de departamento_sede: dos sedes en el mismo departamento (p. ej. dos campus de la UNAL en Bogotá) cuentan como 1, mientras dos sedes en departamentos distintos cuentan como 2; la métrica no es número de sedes.
- scripts/sprint6_tabla_maestra_ies.py:38-562 — claves hardcodeadas sin validación de duplicados ni de normalización: si instituciones_unicas_raw.csv se regenera con otro case/acentos, la cobertura cae sin ninguna señal de error.
- scripts/sprint6_tabla_maestra_ies.py:592 — pd.concat([raw, asignaciones], axis=1) asume que aplicar_mapeo devuelve una Series por fila alineada por índice; correcto aquí, pero frágil si se reordenara raw antes.
- scripts/sprint6_tabla_maestra_ies.py:620-623 — ies_pendientes_revision.csv incluye todas las entradas sin mapeo, pero el script no imprime las 10 pendientes más frecuentes para facilitar la revisión manual (solo cobertura agregada).

Recomendación de auditoría refactorizar — como borrador cumple y sus productos existen, pero un catálogo canónico de IES no debería vivir en código: externalizar el mapeo a datos versionados, renombrar métricas y añadir cobertura mínima garantizada antes de que otros scripts (geografía, app) dependan de él en producción.

4.5.16 scripts/sprint6_validacion_calidad.py

Rol: orquestacion · **Líneas:** 121

Descripción funcional Valida y documenta el tratamiento de valores atípicos de la variable EDAD_ANOS_PR del padrón: detecta los casos >100 (máximo observado 956, error de digitación), compara tres estrategias (original, eliminación, imputación por mediana de grupo) y genera evidencia auditable. Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/analisis/calidad (detectar_atipicos_edad, resumen_tratamiento, filtrar, imputar_mediana, UMBRAL_EDAD_MAX). Salidas: 2 CSVs, 1 JSON y 1 figura.

Análisis de la lógica Flujo 1→4: (1) transformar(cargar_consolidado()); (2) detectar_atipicos_edad devuelve los registros con EDAD_ANOS_PR >100 ordenados desc; (3) resumen_tratamiento calcula n_total/n_atipicos/n_validos y justificación, y se construye un DataFrame comparativo edad_stats con media/mediana/desv/max para original, filtrar() (eliminación)

e `imputar_mediana()`; (4) histogramas antes/después con umbral trazado como `axvline`, y exportación de atípicos a CSV, comparación a CSV y resumen a JSON.

Fortalezas

- Metodología explícita y auditable: umbral documentado (100), justificación cuantitativa ($10/77.237 = 0.013\%$) y decisión adoptada (eliminación) con alternativa de imputación comparada.
- La evidencia confirma los números esperados: $n=77.237$ registros, $\text{máx original } 956.25 \rightarrow 100$ tras filtro (evidencias/calidad_comparacion.csv).
- Script pequeño, legible y sin dependencias exóticas (pandas, matplotlib, seaborn).
- Reutiliza funciones de negocio de `src/analisis/calidad.py` con docstrings de decisión metodológica.

Debilidades

- `n_registros` usa `len(df)` (cuenta filas, incluidas las de EDAD NaN) mientras `media/mediana/std` ignoran NaN: unidades de conteo mixtas en la misma tabla comparativa.
- La mediana se deja sin redondear mientras `media` y `desv_estandar` se redondean a 2 decimales (formato inconsistente).
- El script solo valida edad; otros problemas documentados en `calidad.py` (departamento NO DISPONIBLE 5.2 %, género NO REPORTADO 2.7 %, convocatoria 833 con año 2019) se citan pero no se verifican ni exportan.
- La figura compara 'original' vs 'filtrado' pero no muestra el efecto de la imputación, pese a que se calcula en el DataFrame comparativo.

Mejoras propuestas

- Calcular `n_registros` como `count no-NA` de `EDAD_ANOS_PR` para consistencia con las demás métricas.
- Redondear mediana igual que `media` y documentar el criterio.
- Ampliar la validación a duplicados, nulos por columna y distribución de tipos, exportando un perfil de calidad completo.
- Fijar `seed`/versión del dataset en el JSON de resumen para trazabilidad.

Ejecutabilidad SÍ, ejecutable tal cual con el clon: `cargar_consolidado()` resuelve desde `datos/tarea_join/investigadores_consolidado.xlsx` (no requiere raw Socrata); depende solo de pandas, matplotlib y seaborn. El único requisito no verificado es que `transformar()` produzca `EDAD_ANOS_PR`, lo cual confirman los artefactos existentes.

¿Produce lo esperado? Sí: evidencias/calidad_atipicos_edad.csv, evidencias/calidad_comparacion.csv, evidencias/calidad_resumen.json y artifacts/sprint6_calidad/fig_edad_antes_despues.png existen. El CSV de comparación confirma 77.237 registros, media 44.84 → 44.73, desv 13.23 → 10.04 y máx 956.25 → 100.0, coherente con la decisión de eliminación.

Riesgo / Limitación

Bugs / problemas detectados:

- scripts/sprint6_validacion_calidad.py:62-84 — edad_stats mezcla unidades: len(df) cuenta filas (incluye NaN en EDAD) mientras mean/median/std de pandas ignoran NaN; si hubiera filas con edad faltante, n_registros y las demás columnas no serían comparables.
- scripts/sprint6_validacion_calidad.py:70-73 — df['EDAD_ANOS_PR'].median() sin round frente a round(..., 2) en media (formato inconsistente; en el CSV aparece 43.29 por coincidencia del valor).
- scripts/sprint6_validacion_calidad.py:50-51 — detectar_atipicos_edad solo imprime, no informa el conteo de NaN de EDAD_ANOS_PR, que podría alterar la interpretación del umbral.

Recomendación de auditoría mantener — cumple su propósito (decisión metodológica auditable) con evidencia verificada; requiere solo ajustes menores de consistencia de conteos y redondeo.

4.5.17 src/analysis/__init__.py

Rol: orquestacion · **Líneas:** 80

Descripción funcional Fachada del paquete analisis: re-exporta las funciones públicas de genero, territorial, redes, diversidad, longitudinal y produccion, y declara __all__. Define la API estable que consumen scripts/sprint*/*.py y streamlit_app.py.

Análisis de la lógica Importa explícitamente cada función/constante de los 6 submódulos y las lista en __all__. No contiene lógica de negocio; su función es el contrato de importación (from analisis import pct_femenino_por_area, hhi, construir_grafo, comparar_dane, ...). Cualquier script que importe analisis dispara la importación de networkx (vía redes) y de longitudinal en el arranque.

Fortalezas

- API pública explícita y bien documentada para todos los análisis del proyecto.
- __all__ completo y ordenado facilita autocompletado e inspección.
- Permite a scripts y dashboard importar sin conocer la estructura interna del paquete.

Debilidades

- Importación eager de los 6 submódulos: importar 'analysis' carga networkx y longitudinal (peso y acoplamiento); si una dependencia opcional falta, todo el paquete falla al importar.
- Duplicación de símbolos entre el bloque de imports y `__all__` (mantenimiento manual: añadir una función exige tocar dos lugares).
- No hay validación de esquema en la frontera: las funciones asumen columnas MAYÚSCULAS y ANO_CONVO_INT sin verificar (contrato implícito con Transformacion.py).

Mejoras propuestas

- Convertir redes (y pyvis) en import perezoso o submódulo opcional para no penalizar al dashboard.
- Generar `__all__` a partir de una sola lista de nombres para evitar duplicación.
- Añadir una función `validate_esquema(df)` que verifique columnas requeridas antes de los análisis.

Ejecutabilidad Sí, importable en el clon si las dependencias están instaladas (pandas, numpy, networkx; pyvis solo dentro de `generar_html_pyvis`). No lee archivos: la carga la hace ingesta. El fallo de cargar `produccion` en el clon no afecta el import, pero sí a las funciones de `.produccion` al ejecutarlas.

¿Produce lo esperado? No produce outputs; habilita el resto del pipeline. Coincide con la estructura documentada en el README (`src/analysis` como 'lógica reutilizable').

Riesgo / Limitación

Bugs / problemas detectados:

- `src/analysis/__init__.py:4-36`: importa funciones que no usa directamente; si `analysis/longitudinal.py` fallara al importar (dependencias), derriba también `genero/territorial`, que son independientes (acoplamiento innecesario).

Recomendación de auditoría mantener — fachada correcta; refactorizar a imports perezosos o dividir el paquete para reducir el acoplamiento del dashboard.

4.6 Visualización y dashboard

Archivos: 2.

4.6.1 `app/streamlit_app.py`

Rol: visualizacion · **Líneas:** 17

Descripción funcional Placeholder de 17 líneas que solo configura la página (título 'Observatorio Ustadística', icono de gráfico, layout wide) y muestra un título con el aviso 'Dashboard en construcción. Consulta el README para el plan de desarrollo.' No carga datos, no importa src/ y no contiene ningún análisis. Es la entrada que ejecuta el Dockerfile (CMD `streamlit run app/streamlit_app.py`).

Análisis de la lógica Estructura mínima: `import streamlit; st.set_page_config (page_title 'Observatorio Ustadística', page_icon \{\}U0001f4ca, layout wide); st.title; st.info`. Sin `st.cache_data`, sin filtros, sin tabs, sin conexión con src/. El docstring documenta el uso con 'poetry run streamlit run app/streamlit_app.py'.

Fortalezas

- Sintaxis Streamlit válida: no crashea al arrancar
- Sirve como esqueleto para desarrollo futuro (README del proyecto menciona 'plan de desarrollo')

Debilidades

- Es un placeholder que contradice al dashboard real (`streamlit_app.py`, 983 líneas, 7 tabs)
- El README documenta el deploy del dashboard real (main file `streamlit_app.py` en Streamlit Cloud, línea 192) pero el Dockerfile despliega este placeholder: dos 'dashboards' distintos según la vía de deploy
- La imagen Docker no copia el dashboard real (COPY solo src/, app/, datos/) ni hallazgos/, así que en el contenedor el usuario final ve solo el cartel de 'en construcción'
- Duplica la identidad del proyecto (título distinto al del dashboard real: 'Observatorio Ustadística' vs 'Observatorio MinCiencias - Informe crítico')

Mejoras propuestas

- Eliminar el placeholder y apuntar el Dockerfile al dashboard real (copiar `streamlit_app.py` de la raíz, `hallazgos/` y `datos/`), o
- Mover el dashboard real a `app/streamlit_app.py` (haciendo `app/` la entrada canónica) y actualizar README y Streamlit Cloud en consecuencia, o
- Si se mantiene como esqueleto, excluirlo del Dockerfile y documentar que no es el dashboard de producción

Ejecutabilidad Sí, se ejecuta tal cual (solo requiere streamlit instalado), pero solo muestra el aviso de 'Dashboard en construcción'. No lee ningún dato ni módulo del proyecto.

¿Produce lo esperado? NO produce el dashboard de 7 capítulos documentado: renderiza un cartel estático. En el despliegue Docker, el contenedor (python:3.12-slim + poetry + datos/) muestra este placeholder en vez del dashboard real, contradiciendo el README y el propósito del observatorio.

Riesgo / Limitación

Bugs / problemas detectados:

- Bug de deploy (no de sintaxis): el Dockerfile ejecuta 'streamlit run app/streamlit_app.py' (línea 22) -> el contenedor despliega el placeholder y no el dashboard real; además streamlit_app.py de la raíz ni siquiera se copia a la imagen (COPY src/, app/, datos/ solamente)
- Contradicción de documentación: README línea 175 ('poetry run streamlit run streamlit_app.py') y línea 192 (Streamlit Cloud main file streamlit_app.py) apuntan al dashboard real, mientras que el docstring de este archivo y el Dockerfile apuntan al placeholder
- La imagen Docker tampoco copia hallazgos/sprint3_grafo_filtrado.html: aunque se cambiara el CMD al dashboard real, la tab Redes fallaría en el contenedor

Recomendación de auditoría eliminar: el placeholder no aporta valor de producción y rompe la consistencia del deploy (Docker muestra algo distinto a lo documentado y a Streamlit Cloud). La acción correcta es hacer que el Dockerfile ejecute el dashboard real (streamlit_app.py de la raíz o movido a app/) y copiar hallazgos/; o, si se prefiere app/ como canónica, mover el dashboard real ahí y borrar este archivo.

4.6.2 streamlit_app.py

Rol: visualización · **Líneas:** 983

Descripción funcional Dashboard Streamlit principal del Observatorio MinCiencias con 7 pestañas tipo capítulo (Calidad, Trayectoria, Territorio, Diversidad, Redes, Productividad, Datos crudos). Lee el consolidado vía src/ingesta.cargar_consolidado (XLSX de respaldo en datos/tarea.join), lo transforma con src/Transformacion.transformar y lo cruza con ~30 funciones de src/analisis/*. Cada sección sigue el patrón pregunta -> evidencia -> hallazgo -> caveat, con filtros globales en la barra lateral.

Análisis de la lógica Estructura: constantes (COORDENADAS_DEPTO, POBLACION_DANE_2018, CAT_CORTA) -> cargar_datos() con @st.cache_data (aplica transformar y crea edad_valida) -> sidebar_filtros() (año, género, gran área, depto top-15, categoría, checkbox excluir calidad) -> cabecera() con 6 métricas -> 7 tabs que llaman a una función sección_* cada una. Caches: cargar_datos, panel_completo(len(df), df) y cargar_produccion_normalizada. Conexión con src/: sys.path.insert(ROOT/src) e imports de ingest, Transformacion y analisis.{longitudinal, territorial, genero, diversidad, redes, produccion}. La sección 5 embebe

hallazgos/sprint3.grafo_filtrado.html (existe) y la 6 depende de datos/raw/produccion_grupos.csv (NO existe en el repo).

Fortalezas

- Arquitectura limpia por secciones con una funcion por capitulo; facil de auditar
- Narrativa critica consistente (pregunta/hallazgo/caveat) con caveats metodologicos muy buenos (p. ej. subrepresentacion vs poblacion con educacion superior, Eméritos vitalicios)
- Manejo defensivo: try/except que degrada con st.info en vez de crashear (HHI, genero, DANE, longitudinal)
- Reutiliza las funciones reales de src/analisis; no duplica logica de analisis en la UI
- Filtros globales coherentes entre secciones y metricas de cabecera bien elegidas
- Imports y rutas verificados: todos los modulos src/analisis importados existen con las firmas usadas

Debilidades

- La seccion 6 (Productividad, capitulo 6 del informe) queda vacia en esta maquina: falta datos/raw/produccion_grupos.csv (~1,2 GB); solo muestra un warning
- El grafico HHI de la seccion 3 esta roto por un bug de shape y degrada silenciosamente a 'HHI no disponible'
- Numeros hardcodeados en texto y logica: '30.086 investigadores unicos', fallback 16796 para la anomalia de convocatoria, '6 convocatorias (2013-2021)' en la caption
- Caché de panel_completo con clave debil (len(df)): filtros con el mismo tamaño colisionan y pueden devolver paneles obsoletos
- Docstring dice '6 secciones tipo capitulos' pero hay 7 tabs (desincronizacion interna)
- Seccion 6 no cubierta por try/except: si el CSV de produccion existiera vacio, cov.iloc[0]/iloc[-1] lanzarian IndexError sin captura
- Metricas de cabecera usan df_full mientras las secciones usan df filtrado: el texto 'global' mezcla poblaciones

Mejoras propuestas

- Corregir el bug del HHI: eliminar el .reset_index() o renombrar usando las columnas reales (ANO_CONVO_INT, hhi)
- Documentar el prerequisite del dataset de produccion en la UI (ya hay warning) o agregar fallback a evidencias/produccion_*.csv
- Calcular en runtime los numeros hardcodeados (unicos, convocatorias) en vez de fijarlos
- Cachear panel_completo con un hash real del filtro (tupla de selecciones) en vez de len(df)
- Envolver la seccion 6 completa en try/except para evitar IndexError con produccion vacia

- Sincronizar docstring (6 -> 7 secciones) y caption

Ejecutabilidad Parcial. ‘streamlit run streamlit_app.py’ desde la raíz funciona con streamlit, pandas, numpy, plotly y openpyxl (requirements.txt los declara). cargar_consolidado cae al XLSX datos/tarea_join/investigadores_consolidado.xlsx (existe; datos/raw/investigadores_consolidado.csv no existe). Con ese insumo, las secciones 1, 2, 3 (sin HHI), 4, 5 y 7 deberían renderizar. La sección 6 (productividad) no se ejecuta: requiere descargar datos/raw/produccion_grupos.csv con ‘python -m src.ingesta.produccion’; la app lo maneja con st.warning y return. En entorno poetry (Docker) openpyxl figura en poetry.lock (grupo main) aunque pyproject.toml no lo declara: riesgo de lock desincronizado.

¿Produce lo esperado? Sí, el dashboard implementa los 7 capítulos del informe y coincide con el README (‘streamlit_app.py # Dashboard - 7 tabs’). Sin embargo, el capítulo 6 (productividad) no produce nada sin el dataset de producción, y el HHI (parte del capítulo 3) nunca dibuja por el bug de shape. El placeholder app/streamlit_app.py no contradice a este archivo en local ni en Streamlit Cloud (el README apunta a este), pero sí en el despliegue Docker.

Riesgo / Limitación

Bugs / problemas detectados:

- Sección 3: hhi_por_convocatoria(df, col_geo=col).reset_index() devuelve 5 columnas (index + ANO_CONVO_INT + hhi + n_total + n_territorios) y luego hhi_df.columns = [‘Año’, ‘HHI’] lanza ValueError ‘Length mismatch’; el except lo captura y el gráfico HHI nunca se muestra (líneas 434-435)
- Sección 6: depende de datos/raw/produccion_grupos.csv que no está en el repo; toda la sección (4 métricas + 6 gráficos) se degrada a un warning
- Sección 1: fallback hardcodeado ‘else 16796’ y umbral mágico ID_CONVOCATORIA == 20 sin validación -> número potencialmente incorrecto en silencio
- panel_completo(len(df), df) usa len(df) como hash de cache -> colisiones entre filtros del mismo tamaño (línea 347)
- Si el CSV de producción existiera pero vacío, cov.iloc[0]/cov.iloc[-1] (líneas 761-762) lanzarían IndexError no capturado y crashearía la app
- Caption y docstring desincronizados: ‘6 secciones’ en el header (línea 7) vs 7 tabs reales

Recomendación de auditoría mejorar: es la pieza central del proyecto, bien estructurada y con 5 de 7 secciones funcionales con el XLSX actual. Corregir el bug del HHI, robustecer la cache, documentar/proveer el dataset de producción y eliminar los hardcodes antes de considerar un refactor.

4.7 Documentación ejecutable

Archivos: 2.

4.7.1 docs/manual.ipynb

Rol: documentacion · **Líneas:** 805

Descripción funcional Manual reproducible de 49 celdas (23 markdown + 26 código) generado por scripts/generar_manual.py (no ejecutado: todas las celdas tienen execution_count null). 14 secciones que reproducen los hallazgos del observatorio: calidad (atípicos de edad), trayectoria y matrices de transición, Sankeys de categoría, concentración territorial (HHI), tabla maestra de IES, geografía institucional (residencia vs institución), brecha de género, diversidad vs DANE/RUV, productividad por categoría, brecha de género en productividad, composición OCDE y recomendaciones de política pública.

Análisis de la lógica Celda 1: ROOT = pathlib.Path.cwd() con fallback a ROOT.parent si (ROOT/src) no existe (funciona desde la raíz del repo o desde docs/); inserta src/ en sys.path. Celda 2: inv = transformar(cargar_consolidado()) y prod = normalizar_produccion(cargar_produccion()). Secciones 2-3: funciones de src/analisis/calidad.py y longitudinal.py (construir_panel, comparar_periodo, matriz_transicion, matrices_todos_periodos). Secciones 4, 7, 8 y 13: subprocess.run(['python3', 'scripts/sprint6_*.py'], cwd=ROOT). Secciones 6-8 y 13: pd.read_csv de evidencias/*.csv. IFrame apunta a artifacts/sprint6_sankey/*.html. Todo el código del manual reutiliza los módulos reales de src/ (no duplica lógica).

Fortalezas

- Intención didáctica y reproducible clara: cada hallazgo con su código mínimo y su caveat
- Reutiliza el código real (src/ingesta, src/Transformacion, src/analisis/*): el manual no inventa lógica paralela
- Los insumos que referencia existen en el repo: evidencias/tabla_maestra_ies.csv, mapping_inst_filia_to_ies.csv, geografia_*.csv, ocde_*.csv, scripts/sprint6_*.py y artifacts/sprint6_sankey/*.html
- Caveats de alta calidad (Eméritos vitalicios, comparación contra población con educación superior, captura de doble afiliación solo en 2021)
- Resolución de ROOT con fallback (mejor que la del EDA)

Debilidades

- Dos celdas con bugs de API que crashean: m['conteos'] (matrices_todos_periodos devuelve tuplas) y columna= en hhi_por_convocatoria (la firma es col_geo=)
- subprocess hardcodea 'python3', no portable a Windows (el repo vive en Windows)
- Depende de datos/raw/produccion_grupos.csv (~1,2 GB) que NO está en el repo: la celda 2 de carga crashea y las secciones 11-13 no tienen prod

- Inconsistencias numericas entre el texto del manual y el codigo real: '22 registros con edad >100 (0,03 %)' vs src/analisis/calidad.py '10 registros (0,013 %)'; '211 IES canonicas / 91 %' vs celda '258 strings mapeados'
- Nunca ejecutado de punta a punta (execution_count null en las 26 celdas de codigo): el manual no tiene evidencia de reproducibilidad
- El generador scripts/generar_manual.py contiene los mismos bugs (los hereda el notebook)

Mejoras propuestas

- Corregir la celda de matrices: m es una tupla (conteos, probs) -> iterar con for periodo, (conteos, probs) in matrices.items()
- Corregir la celda HHI: hhi_por_convocatoria(inv_clean, col_geo='NME_DEPARTAMENTO_RES.PR')
- Usar 'python' o sys.executable en subprocess para portabilidad a Windows
- Anadir celda condicional que avise si falta el dataset de produccion y permita continuar con las secciones que no lo necesitan
- Armonizar cifras con src/analisis/calidad.py y scripts/sprint6_tabla_maestra_ies.py, y regenerar con generar_manual.py
- Ejecutar el notebook completo y commitear los outputs como evidencia de reproducibilidad

Ejecutabilidad Parcial y no reproducible de punta a punta tal cual. Con 'poetry run jupyter notebook docs/manual.ipynb' (README) la carga de inv funciona (XLSX existe; openpyxl en lock/requirements), pero: (1) cargar_produccion() lanza FileNotFoundError (datos/raw/produccion_grupos.csv ausente) -> la celda 2 crashea y las secciones 11-13 carecen de prod; (2) la celda de matrices lanza TypeError (m['conteos'] sobre una tupla); (3) la celda HHI lanza TypeError (kwarg column= inexistente); (4) los 4 subprocess con 'python3' fallan en Windows. Las secciones 6-10 (tabla maestra, geografia, sankeys, genero, diversidad) leerian archivos que sí existen en el repo y funcionarían si se superan las anteriores.

¿Produce lo esperado? El manual documenta los 12 hallazgos y sus rutas de evidencias/artifacts existen, pero NO es ejecutable tal cual: 3 celdas fallan por API incorrecta y 1 por dependencia de datos ausente. No contradice al dashboard (usa las mismas funciones de src/), pero la cifra '22 atipicos de edad' contradice a src/analisis/calidad.py ('10'), indicando que el texto del manual no se actualizo con el codigo final. El placeholder app/streamlit_app.py es irrelevante para este notebook.

Riesgo / Limitación

Bugs / problemas detectados:

- Celda seccion 3: 'print(m['conteos'])' - matrices_todos_periodos devuelve {periodo: (conteos, probs)} (tupla, segun docstring y codigo de src/analisis/longitudinal.py linea 164-179); m

es una tupla ->TypeError: tuple indices must be integers

- Celda seccion 5: 'hhi_por_convocatoria(inv_clean, columna='NME_DEPARTAMENTO_RES_PR')' - la firma es (df, col_geo, col_year) ->TypeError: unexpected keyword argument 'columna'
- Celda 2: 'cargar_produccion()' ->FileNotFoundError porque datos/raw/produccion-grupos.csv no existe en el repo (hay que descargarlo, ~1,2 GB)
- 4 celdas usan subprocess.run(['python3', ...]): en Windows no existe python3.exe ->FileNotFoundError (portabilidad rota)
- Texto vs codigo: '22 registros con edad superior a 100 (0,03 %)' vs src/analisis/calidad.py 'diez registros (0,013 %)'; '211 IES canonicas que mapean el 91 %' vs celda que reporta '258 strings mapeados' (scripts/sprint6_tabla_maestra_ies.py)

Recomendación de auditoría refactorizar: el manual es valioso como documentación de los hallazgos pero no es reproducible tal cual. Corregir los 2 bugs de API (`m['conteos']`, `col_geo`), hacer portable el subprocess, documentar el prerequisite del dataset de produccion (o leer muestras de evidencias/) y armonizar las cifras con `src/`. Los mismos cambios deben aplicarse en `scripts/generar_manual.py` para que el notebook se pueda regenerar.

4.7.2 scripts/generar_manual.py

Rol: documentacion · **Líneas:** 610

Descripción funcional Genera `docs/manual.ipynb`, un notebook ejecutable de 49 celdas (23 markdown + 26 código) que reproduce los doce hallazgos del observatorio de investigadores MinCiencias. Construye el notebook programáticamente con `nbformat`: va apilando celdas markdown y de código mediante los helpers `md()`/`code()` y las escribe en `docs/` al final. Cada sección (1-14) combina texto explicativo con cifras del observatorio y celdas que reutilizan el pipeline moderno (`src/ingesta`, `src/Transformacion.py`, `src/analisis/*`) en lugar de duplicar lógica.

Análisis de la lógica El script no ejecuta análisis: solo ensambla el notebook. Declara `ROOT` como el padre del directorio del script y crea `docs/` con `mkdir(exist_ok=True)`. Luego encadena 14 secciones: portada, setup (celda que inserta `src/` en `sys.path`), carga de datos (from `ingesta` import `cargar_consolidado`, `cargar_produccion`; `transformar`; `normalizar_produccion`), y secciones temáticas que importan funciones de `src/analisis` (`calidad`, `longitudinal`, `territorial`, `genero`, `diversidad`, `produccion`). Incrusta figuras mediante `IFrame` con rutas relativas a `artifacts/sprint6_sankey/*.html` y CSV ya generados en `evidencias/` (`tabla_maestra_ies.csv`, `geografia_*.csv`, `ocde_*.csv`). Cuatro celdas lanzan `subprocess.run(["python3", "scripts/sprint6_*.py"], cwd=ROOT, check=True)` para regenerar Sankeys y CSVs. Al final,

`main()` crea `nbformat.v4.new_notebook()` con metadata `kernelspec python3` y escribe el JSON en `docs/manual.ipynb` con codificación `utf-8`.

Fortalezas

- Estructura clara y reproducible: 49 celdas en 14 secciones numeradas, con markdown explicativo de cada hallazgo, incluyendo citas textuales del director y caveats metodológicos (Emérito vitalicio, diversidad solo capturada desde 2021, comparación DANE vs población con educación superior).
- Reutiliza el pipeline moderno (`ingesta, Transformacion, analisis.*`) en las celdas en lugar de duplicar lógica de análisis, lo que mantiene coherencia con `scripts/sprint*.py`.
- Documenta explícitamente la decisión metodológica de tratamiento de atípicos (eliminación sobre imputación) con justificación cuantitativa, y ofrece la alternativa `imputar_mediana` aunque no se aplique por defecto.
- El notebook generado es versionable y el script es idempotente (siempre escribe el mismo `manual.ipynb` desde cero).
- El notebook existente en el clon coincide con lo que el script genera: 49 celdas (23 markdown + 26 código), mismo título UTF-8 correcto y mismas celdas clave (`carga, subprocess, matrices`) — verificado byte a byte en la sección relevante.

Debilidades

- Cifras hardcodeadas en markdown (22 atípicos de edad, 77.237 filas, 51 % Bogotá+Antioquia, 242 investigadores Antioquia→Bogotá, retención del 92 %) que nunca se recalculan; ya hay discrepancia real: el docstring de `src/analisis/calidad.py` dice 'diez registros' y '0.013 %', mientras el manual dice 22 y 0,03 %.
- El script asume el dataset completo de 6 convocatorias (77.237 filas), pero el clon solo dispone del XLSX de respaldo `datos/tarea_join/investigadores_consolidado.xlsx` con 3 convocatorias (2017, 2019, 2021) y 50.891 filas, por lo que las cifras regeneradas no coincidirían con el texto (p. ej. 'Cinco diagramas' de Sankey solo podrían ser 2 pares).
- No declara dependencias del script ni del notebook: `nbformat` (necesario para ejecutar el script), `seaborn` e `IPython` (importados por las celdas) no están en `requirements.txt`.
- Depende de `subprocess` con 'python3' hardcodeado en 4 celdas, lo que lo hace no portable a Windows.
- Rutas relativas en `IFrame` (`"../artifacts/..."`) dependen del `cwd` del kernel de Jupyter; frágiles si el notebook se abre desde otro directorio.
- No valida que los CSV leídos existan ni que los `subprocess` tengan éxito con mensajes claros (`check=True` lanza excepción genérica).

Mejoras propuestas

- Corregir la celda de matrices: `matrices_todos_periodos()` retorna dict de tuplas (conteos, probs); usar `m[0]` o cambiar la función para retornar dict con claves 'conteos'/'probabilidades'.
- Reemplazar "python3" por `sys.executable` (o "python") en los `subprocess.run` para portabilidad a Windows.
- Agregar `nbformat`, `seaborn` e `IPython` a `requirements.txt`.
- Recalcular cifras clave en celdas (`n_atipicos`, `top departamentos`, `flujos`) y compararlas con `asserts` contra los valores del texto, o generar el markdown de hallazgos dinámicamente.
- Documentar que el XLSX de respaldo cubre solo 3 convocatorias y que las secciones 11-13 requieren datos/`raw/produccion_grupos.csv`; añadir `try/except` en la carga de producción con mensaje de acción clara.
- Reemplazar `subprocess` por llamadas directas a funciones de `src/analisis` dentro del notebook (más robusto y auditable), o usar rutas basadas en `ROOT` en lugar de rutas relativas en `IFrame`.

Ejecutabilidad El script en sí (`python scripts/generar_manual.py`) es ejecutable tal cual en el clon siempre que `nbformat` esté instalado (no está en `requirements.txt`, aunque probablemente sí en `.venv` por Jupyter); escribe `docs/manual.ipynb` y no requiere datos. El notebook generado NO es ejecutable de punta a punta en el clon: (1) la celda de carga llama `cargar_produccion()`, que lanza `FileNotFoundError` porque `datos/raw/produccion_grupos.csv` está ausente (`gitignored`), rompiendo las secciones 11-13; (2) las 4 celdas con `subprocess.run(["python3", ...])` fallan en Windows (`python3` no es ejecutable estándar; sería `python` o `py`); (3) la celda de matrices accede `m[conteos]` sobre tuplas y lanza `TypeError`; (4) los `IFrame` usan rutas relativas a `artifacts/` que dependen del `cwd`. Los CSV de evidencias/ y los HTML de `artifacts/sprint6_sankey/` existen (generados previamente con el dataset completo), por lo que las celdas de solo-lectura (`IFrame`, `read_csv`) funcionarían si las anteriores no fallaran.

¿Produce lo esperado? Sí genera `docs/manual.ipynb` con 49 celdas (23 markdown + 26 código) y metadata `kernelspec python3 / language_info 3.11`. El notebook existente en el clon coincide con lo que el script generaría: misma estructura (49 celdas), mismo título (em-dash UTF-8 correcto, sin BOM) y las mismas celdas clave (carga de datasets, `subprocess` con `python3`, `IFrame` y la celda bugueada con `m[conteos]`); el notebook no tiene outputs ni `execution_count`, es decir, nunca fue ejecutado. El notebook resultante NO es totalmente ejecutable: además del `TypeError` de `m[conteos]` y del `FileNotFoundError` de `cargar_produccion()`, usa `import seaborn` e `IPython.display` sin declarar dependencias y rutas relativas frágiles; las secciones 1-2 y 5-6 (solo inv, sin producción) serían las únicas viables de forma inmediata con el XLSX de respaldo.

Riesgo / Limitación

Bugs / problemas detectados:

- Línea 197 (celda 'Trayectoria longitudinal'): `matrices_todos_periodos()`

- (src/analisis/longitudinal.py:164-179) retorna {periodo: (conteos_df, probs_df)}, pero la celda hace `m[conteos]` → `TypeError: tuple indices must be integers or slices, not str` en runtime; el notebook existente contiene el mismo bug.
- Líneas 213, 318, 357 y 509: `subprocess.run(["python3", "scripts/sprint6-*.py"], ...)` — 'python3' no existe en Windows (el clon vive en Windows); falla con `FileNotFoundError`. Debería ser `sys.executable` o 'python'.
 - Celda de carga (líneas 107-117): `prod = normalizar_produccion(cargar_produccion())` lanza `FileNotFoundError` en el clon porque `datos/raw/produccion_grupos.csv` no existe; rompe secciones 11, 12 y 13.
 - Línea 136-140 vs `src/analisis/calidad.py:1-26`: el manual afirma '22 registros con edad superior a 100 años' y '0,03%', pero el módulo de calidad documenta 'diez registros' y '0.013%' — cifras contradictorias sin recalcular.
 - Líneas 44-47 de la portada: dice 'los doce hallazgos' pero enumera 14 secciones numeradas (1-14) — incoherencia menor de numeración.
 - `requirements.txt` no incluye `nbformat` (necesario para ejecutar el script), ni `seaborn` ni `IPython` (importados por celdas del notebook).
 - Celda `setup` (líneas 89-94): `ROOT = pathlib.Path.cwd()` depende del directorio desde donde se lance el kernel; el fallback a `ROOT.parent` solo cubre un nivel, frágil en Jupyter Lab/VS Code con `cwd` distinto.

Recomendación de auditoría mejorar — El script cumple bien su rol documental (estructura clara, reuso de `src/`, notebook reproducible y coincidente con el del clon), pero no es ejecutable de punta a punta en el estado actual del repositorio: hay un bug de API real (`m[conteos]`), dependencia de un CSV de producción ausente, `subprocess` no portables a Windows y cifras hardcodeadas que ya divergen del código de calidad. Se recomienda una mejora enfocada: corregir la celda de matrices, usar `sys.executable`, declarar dependencias y recargar/validar las cifras — sin necesidad de refactorizar la arquitectura del script.

4.8 Análisis exploratorio de datos

Archivos: 1.

4.8.1 notebooks/01_eda.ipynb

Rol: eda · **Líneas:** 240

Descripción funcional EDA exploratorio de 16 celdas (6 markdown + 10 código) sobre el dataset consolidado de investigadores reconocidos (77.237 registros, 30 variables, 6 convocatorias 2013-2021). Carga con `src/ingesta.cargar_consolidado`, transforma con `src/Transformacion.transformar` y produce: tipos y faltantes, estadísticas de 3 variables numéricas, frecuencias de 5 categóricas y 4 visualizaciones (género, gran área OCDE, edad por

convocatoria, top 15 departamentos). Es el punto de entrada analítico del proyecto, anterior a los sprint 2-6.

Análisis de la lógica Celda 1: `ROOT = pathlib.Path().resolve().parent` (comentario: 'raíz del repo (notebooks/./)', asume `cwd = notebooks/`); `sys.path.insert(0, ROOT/src)`; imports de `pandas/numpy/matplotlib/seaborn` y 'from `ingesta` import `cargar_consolidado`', 'from `Transformacion` import `transformar`'. Celda 2: `df_raw = cargar_consolidado()`; `df = transformar(df_raw)`; `print shape` y `head`. Celdas siguientes analizan `df` directamente: `dtypes`, `faltantes (isnull().mean())`, `heatmap` de NaN, describe de `EDAD_ANOS_PR/NRO_ORDEN_FORM_PR/ORDEN_CLAS_PR`, `histogramas`, `value_counts` de 5 categoricas, `pie` de genero, `barh` de areas, `boxplot` de edad por convocatoria, `barplot` de top departamentos. Sin cache ni subprocesos; input unico: el consolidado.

Fortalezas

- Sencillo y autocontenido: funciona sin el dataset de produccion (solo necesita el consolidado)
- Usa el pipeline real de `ingesta+transformacion`, misma fuente que el dashboard (consistencia de datos)
- Estructura EDA estandar y legible: carga, calidad, numericas, categoricas, visualizaciones
- Todas las columnas que referencia (`EDAD_ANOS_PR`, `NRO_ORDEN_FORM_PR`, `ORDEN_CLAS_PR`, `NME_REGION_RES_PR`, etc.) existen en el header del CSV consolidado (30 columnas, verificado)

Debilidades

- Resolución de `ROOT` fragil: depende de que el `cwd` del kernel sea `notebooks/`; si JupyterLab arranca el kernel desde la raíz del repo, `ROOT` apunta al padre del repo y 'from `ingesta` import ...' lanza `ModuleNotFoundError` (a diferencia de `manual.ipynb`, no tiene fallback a `ROOT.parent`)
- El análisis de faltantes se hace sobre `df` YA transformado: `transformar` imputa mediana en numericas y 'NO REPORTADO' en categoricas (`tratar_faltantes`), así que el `heatmap` de NaN y los % de faltantes quedan vacíos o mínimos (solo sobreviven `COD_DANE_*` y `ANO_CONVO_FECHA`); la sección 'Calidad de datos' del EDA es poco informativa
- Nomenclatura inconsistente: el título dice 'Grupo 7' (la carpeta del repo es Grupo 8; `Transformacion.py` también dice Grupo 7)
- Usa `palette=` en `sns.boxplot/sns.barplot` (deprecación en `seaborn` ≥ 0.14 ; cosmético, no rompe)
- No documenta como se lanza (`cwd` requerido) ni fija versiones de `seaborn`

Mejoras propuestas

- Adoptar el patron de ROOT del manual.ipynb: `Path.cwd()` con fallback a `ROOT.parent` (o derivar de la ubicacion del notebook) para funcionar desde cualquier cwd
- Correr el analisis de calidad sobre `df_raw` (antes de transformar) o documentar que los faltantes ya fueron imputados/etiquetados
- Unificar la nomenclatura de grupo (Grupo 7 vs Grupo 8) en titulo y docstrings
- Quitar `palette=` o fijar `seaborn<0.14` en pyproject
- Anadir celda que confirme `shape == (77237, 30)` como asercion de insumo

Ejecutabilidad Sí, con una condicion de entorno: se ejecuta tal cual si el kernel arranca con `cwd = notebooks/` (p. ej. ‘jupyter notebook notebooks/01_eda.ipynb’ desde la raiz, que en el cliente clasico pone el cwd en notebooks/). Entrada: `datos/tarea_join/investigadores_consolidado.xlsx` via `cargar_consolidado` (existe; el CSV de `datos/raw` no existe, se cae al XLSX). Requiere `pandas`, `numpy`, `matplotlib`, `seaborn` y `openpyxl`. No requiere el dataset de produccion. Si el cwd del kernel es la raiz del repo, el import de `ingesta` falla (`ModuleNotFoundError`).

¿Produce lo esperado? Cumple su rol de EDA: carga y describe el consolidado y genera las 4 visualizaciones sin errores bajo la condicion de cwd correcta. Nota de verificacion: el CSV paralelo `datos/tarea_join/investigadores_consolidado.csv` tiene ~50.891 filas (no 77.237): conviene confirmar que el XLSX (fuente real de `cargar_consolidado`) tenga las 77.237 filas y que no haya divergencia entre formatos. La seccion de faltantes es casi vacia por la imputacion previa de transformar. No contradice al dashboard (misma fuente de datos).

Riesgo / Limitación

Bugs / problemas detectados:

- `ROOT = pathlib.Path().resolve().parent` (celda 1): si el cwd del kernel no es `notebooks/`, `ROOT` no apunta a la raiz del repo y ‘from `ingesta` import `cargar_consolidado`’ lanza `ModuleNotFoundError`; no hay fallback (a diferencia de `docs/manual.ipynb` que sí lo tiene)
- Analisis de calidad sobre `df` transformado: `transformar` (tratar `faltantes`) imputa mediana en numericas y ‘NO REPORTADO’ en categoricas, por lo que `df.isnull()` queda casi sin NaN; el mapa de calor y los % de faltantes de la seccion 2 no muestran la calidad real del dato crudo
- Titulo ‘Grupo 7’ vs carpeta ‘Grupo 8’ (y `Transformacion.py` tambien dice Grupo 7): inconsistencia de nomenclatura del proyecto
- `sns.boxplot/palette` y `sns.barplot/palette`: deprecacion de `palette` en `seaborn>=0.14` (aviso, no falla)

Recomendación de auditoría mejorar (mantener): el EDA es valido y util como punto de entrada; endurecer la resolucion de rutas con el patron del `manual.ipynb`, mover el analisis de calidad a `df_raw` (o aclarar que los faltantes ya fueron tratados), unificar la nomenclatura de grupo y fijar/eliminar `palette=`.

4.9 Pruebas automatizadas

Archivos: 2.

4.9.1 tests/__init__.py

Rol: testing · **Líneas:** 0

Descripción funcional Marcador de paquete Python para tests/ (vacío, 0 líneas).

Análisis de la lógica Sin lógica.

Fortalezas

- Correcto por convención (permite descubrir tests con pytest).

Ejecutabilidad N/A.

¿Produce lo esperado? N/A.

Recomendación de auditoría mantener — estándar de Python.

4.9.2 tests/test_ingesta.py

Rol: testing · **Líneas:** 8

Descripción funcional Suite de tests del módulo de ingesta. Contiene un único test (test_catalogo_exists) que verifica que datos/catalogo.yaml existe.

Análisis de la lógica test_catalogo_exists usa Path('datos/catalogo.yaml').exists() y assert. Configuración de testpaths en pyproject.toml apunta a tests/.

Fortalezas

- Estructura de tests creada y ejecutable con pytest.
- Configuración en pyproject.toml (testpaths) correcta.

Debilidades

- Cobertura insuficiente: 1 test trivial; no prueba descargar/guardar/transformar ni ninguna función de análisis.
- pandera declarado en dev-dependencies pero sin uso (no hay contratos de esquema).
- El test depende del cwd (Path relativo): falla si pytest se ejecuta desde otra carpeta.

Mejoras propuestas

- Añadir tests de humo para cargar_consolidado (shape, columnas, unicidad de ID_PERSONA_PR).
- Añadir contratos de esquema con pandera (completitud, tipos, codificación).
- Usar rutas relativas al proyecto (pathlib con `__file__`).

Ejecutabilidad Sí (pytest); verificado que el único test pasa si el catálogo existe. No se ejecutó pytest en la auditoría por prioridad, pero no hay dependencias externas más allá de pathlib.

¿Produce lo esperado? Parcial: valida solo la existencia del catálogo, no el comportamiento de la ingesta — no cumple el objetivo de 'probar' el pipeline.

Riesgo / Limitación

Bugs / problemas detectados:

- Path relativo frágil (depende del directorio de ejecución).

Recomendación de auditoría mejorar — ampliar la cobertura a transformación y análisis; es la acción de mayor impacto en la deuda técnica de pruebas.

4.10 Código legacy (etapa académica inicial)

Archivos: 13.

4.10.1 notebooks/tarea_analisis_bases_de_datos/2019_codigo.py

Rol: legacy · **Líneas:** 174

Descripción funcional EDA descriptivo de la convocatoria 2019 (Investigadores_Reconocidos_por_convocatoria.20250909.csv): calcula el % de faltantes por variable, conteos y gráficos de barras para género, región/departamento de nacimiento y residencia, histogramas

de variables numéricas y tablas cruzadas (género x región, gran área x género). Es un script exportado de Colab y corresponde a la tarea de análisis de bases de datos.

Análisis de la lógica pandas + matplotlib: lectura con `pd.read_csv`, limpieza de nombres de columnas, `df.isna().sum()` con % de faltantes ordenado, `value_counts` y `pd.crosstab(normalize='index')` para proporciones por fila, detección de columnas clave por subcadena ('GENER', 'REGION', 'DEPARTAMENTO'), histogramas y heatmap con `imshow`. No usa librerías de inferencia estadística; es puramente descriptivo.

Fortalezas

- Reporta faltantes por variable con porcentaje ordenado, práctica correcta de EDA
- Usa `crosstab` con proporciones por fila (composición de género dentro de cada gran área), lectura adecuada para comparar grupos
- Detecta columnas de forma defensiva por subcadena en el nombre, tolerante a cambios de esquema
- Excluye columnas tipo ID/código de los histogramas (evita describir claves)

Debilidades

- Ruta hardcodeada de Colab `"/content/Investigadores_Reconocidos_por_convocatoria_20250909.csv"`, inexistente en el clon y en cualquier equipo local
- Usa `display()` de IPython/Colab; como script Python estándar lanza `NameError`
- Bloque de cálculo de faltantes duplicado (`falt` y `faltantes`): código muerto
- El heatmap de proporciones no está acompañado de ninguna prueba de independencia (chi-cuadrado)
- Etiquetas de porcentaje con desplazamiento fijo `'v + 50'` sobre conteos absolutos, frágil si cambia la escala
- Las proporciones se calculan sobre `n` total incluyendo `NaN` en el denominador; sin documentar
- No guarda figuras ni resultados; sin estructura de función o parámetros

Mejoras propuestas

- Parametrizar la ruta (`Pathlib`/argumentos) en vez de hardcodear `/content`
- Reemplazar `display()` por `print()` para que corra como script
- Eliminar el bloque duplicado de faltantes
- Añadir `chi2.contingency` a las tablas cruzadas y reportar residuos
- Guardar figuras y tablas en una carpeta de salida

Ejecutabilidad NO. Falta el input `'/content/Investigadores_Reconocidos_por_convocatoria_20250909.csv'` (ruta de Colab, no existe en el clon) y además usa `display()` de IPython, que falla fuera de un entorno de notebook.

¿Produce lo esperado? Metodología descriptiva correcta para el objetivo (EDA por convocatoria): no hay errores estadísticos graves porque no realiza inferencia. Único matiz: las proporciones usan el total con NaN y no se prueba la asociación en los cruces, por lo que los heatmaps son solo exploratorios.

Riesgo / Limitación

Bugs / problemas detectados:

- `display()` no existe en scripts Python estándar (`NameError`)
- Cálculo de faltantes duplicado (`falt` y `faltantes`) sin uso del primero
- `plt.text(pct + 0.5, ...)` asume coordenadas de barras horizontales con holgura fija

Recomendación de auditoría Mantener como referencia histórica del EDA inicial de la convocatoria 2019. No migrar: el pipeline moderno (`src/ingesta/minciencias.py`, `src/analisis/calidad.py`) ya sistematiza faltantes y perfiles descriptivos; este script no aporta lógica reutilizable.

4.10.2 notebooks/tarea_analisis_bases_de_datos/datos_2021.ipynb

Rol: legacy · **Líneas:** 2338

Descripción funcional Notebook Colab con el EDA de la convocatoria 2021 (Convocatoria 894): filtra `Investigadores_Consolidado` por año (21.094 registros x 30 variables), describe, analiza faltantes y duplicados, y gráfica género (pie), edad (histograma + detección de outliers >100 años), región de residencia, nivel de formación, gran área, área, clasificación (violinplot edad x clasificación) y proporciones de víctimas del conflicto, discapacidad y grupo étnico. Cierra con interpretaciones sustantivas en markdown.

Análisis de la lógica `pandas` + `matplotlib` + `seaborn`: filtra con `df['ANO_CONVO'].astype(str).str.contains('2021')`, `describe()`, `isnull().sum()`, `duplicated()`, `value_counts()`, `sns.histplot/countplot/violinplot`, y crea `df_2021_filtered` (edad ≤ 100) para los gráficos posteriores. Todo descriptivo; sin pruebas de hipótesis.

Fortalezas

- Identifica y documenta el outlier de edad (máximo 956.25 años) y filtra edad ≤ 100 antes de las gráficas (decisión de limpieza explícita)

- Revisa duplicados y faltantes por columna con interpretación
- El violinplot edad x clasificación apoya una conclusión sustantiva correcta (Eméritos mayores, Junior más jóvenes)
- Buena práctica de narrativa: cada gráfico va acompañado de interpretación en markdown

Debilidades

- Ruta hardcodeada de Colab `’/content/Investigadores_Consolidado.xlsx’`, inexistente en el clon
- La celda del countplot de nivel de formación x género usa `df_2021_gender_filtered`, variable que NO está definida en ninguna celda del notebook (NameError al re-ejecutar; probablemente se borró la celda que la creaba)
- `describe()` incluye IDs y códigos (media de `ID_PERSONA_PR` y `COD_DANE` sin sentido estadístico)
- Pie chart para género con categorías de `n=4-5` (`’Intersexual’`, `’No disponible’`): visualización poco informativa
- Salidas embebidas en base64 inflan el archivo a 2338 líneas, dificultando el diff
- Filtrado por `str.contains(’2021’)` sobre la fecha depende del formato de la columna (frágil si el Excel cambia el tipo)
- Sin seed, sin guardado de resultados y sin checklist de calidad al final

Mejoras propuestas

- Definir `df_2021_gender_filtered` o eliminar la celda rota
- Excluir IDs/códigos del `describe()`
- Reemplazar el pie por barras horizontales
- Limpiar salidas (Output $\rightarrow$ Clear) antes de commitear
- Parametrizar la lectura del Excel y extraer el año con `dt.year` en vez de `str.contains`

Ejecutabilidad NO. Falta `’/content/Investigadores_Consolidado.xlsx’` (no existe en el clon) y la celda de nivel de formación x género lanza NameError porque `df_2021_gender_filtered` no está definida en el notebook.

¿Produce lo esperado? Metodología descriptiva correcta para el objetivo (EDA por convocatoria). No hay errores de inferencia porque no se realizan pruebas; el filtro edad ≤ 100 es razonable pero el umbral no se justifica y no se hace análisis de sensibilidad (cuántos registros y cómo cambian los perfiles).

Riesgo / Limitación**Bugs / problemas detectados:**

- `df_2021_gender_filtered` usada sin definición (`NameError` en re-ejecución)
- Filtro por `str.contains('2021')` sobre una columna de fecha: frágil ante cambios de formato
- `describe()` sobre columnas ID/código produce estadísticos sin interpretación

Recomendación de auditoría Mantener como referencia del EDA de la convocatoria 2021 (documenta hallazgos de calidad como el outlier de edad). No migrar tal cual: los análisis equivalentes modernos están en `src/analisis/` (`produccion.py`, `genero.py`, `longitudinal.py`) con datos ya depurados.

4.10.3 notebooks/tarea_analisis_bases_de_datos/inv_17_codigo_graficas.R

Rol: legacy · **Líneas:** 614

Descripción funcional EDA descriptivo de la convocatoria 2017 (Investigadores_Reconocidos_por_convocatoria_20250831.xlsx): tablas de frecuencia y gráficos (barras, pie, histograma con densidad, boxplot) para área, gran área, género, país/región/departamento/municipio de nacimiento y de residencia, nivel de formación, clasificación, edad, víctimas del conflicto, grupo étnico, discapacidad e INST_FILIA. Detecta el valor imposible de edad (952 años) y crea una versión filtrada (edad <100). Corresponde a la tarea de análisis de bases de datos.

Análisis de la lógica `readxl` + `ggplot2` + `dplyr` + `forcats`: `count()` con filtros por umbral de frecuencia para dividir categorías altas/bajas, `geom_bar/geom_col` con `coord_flip` y `reorder/fct_infreq`, pie charts con `coord_polar`, histograma con `aes(y = ..density..)` + `geom_density` + `vline` de la media. Sin pruebas formales; análisis univariado exploratorio.

Fortalezas

- Reconoce explícitamente las variables de identificación (IDs, códigos DANE) y evita graficarlas, decisión de análisis correcta
- Detecta el valor imposible de edad (952) y construye `edad_sin_error` (edad <100), buen control de calidad de datos
- Ordena categorías con `reorder/fct_infreq` y separa frecuencias altas y bajas para legibilidad
- Autoevalúa gráficos poco informativos ('grafica sin sentido', 'no sirve la grafica'): honestidad analítica y criterio

Debilidades

- Ruta hardcodeda 'Downloads/Investigadores_Reconocidos_por_convocatoria_20250831.xlsx', inexistente en el clon
- Usa `plot_missing()` de `DataExplorer` sin cargar `library(DataExplorer)`: error en ejecución
- Referencia el objeto `Investigadores_Consolidado` (línea 21) sin definirlo en el archivo: error de ejecución
- Pie charts con decenas de categorías (país de nacimiento, región): mala práctica de visualización
- Umbrales de filtrado arbitrarios y comentarios desalineados con el código (p. ej., comenta 'menor a 140' pero filtra `n<40`)
- Número mágico hardcodedo: `sum(NME_MUNICIPIO_RES_PR1$n)/13001`
- Sintaxis deprecada `geom_text(aes(label = ..count..))` y `aes(y = ..density..)` en `ggplot2` `>= 3.4` (usa `after_stat`)
- No guarda gráficos ni tablas; sin documentación de parámetros ni versión de los datos

Mejoras propuestas

- Cargar `DataExplorer` o reemplazar `plot_missing` por un cálculo propio de faltantes
- Definir `Investigadores_Consolidado` o eliminar la línea que lo referencia
- Sustituir pie charts por barras horizontales cuando hay muchas categorías
- Documentar y justificar los umbrales (`n>250`, `n>500`, `n<40`, etc.)
- Usar `after_stat(count)/after_stat(density)` y parametrizar la ruta y el denominador (13001)

Ejecutabilidad NO. Falta 'Downloads/Investigadores_Reconocidos_por_convocatoria_20250831.xlsx' (no existe en el clon) y además falla en `plot_missing()` por falta de `library(DataExplorer)` y en la referencia a `Investigadores_Consolidado` (objeto no definido).

¿Produce lo esperado? Descriptivo correcto para el objetivo (EDA por convocatoria): no hay errores de inferencia porque no realiza pruebas. El filtro edad `<100` es apropiado. Los pie charts con muchas categorías son un problema de visualización, no estadístico.

Riesgo / Limitación

Bugs / problemas detectados:

- `plot_missing()` sin `library(DataExplorer)`: función no encontrada
- `Investigadores_Consolidado` referenciado sin ser definido (línea 21)
- Sintaxis `..count../..density..` deprecada en `ggplot2` moderno (deprecation warning/error según versión)
- Denominador hardcodedo 13001 en el cálculo de proporción de municipios de residencia

Recomendación de auditoría Mantener como referencia histórica del EDA de la convocatoria 2017 (documenta hallazgos de calidad de datos). No migrar: `src/analisis/calidad.py` y `src/ingesta/minciencias.py` sistematizan la limpieza; este script no aporta lógica reutilizable sin reescribirlo.

4.10.4 notebooks/tarea_dimensiones_y_hechos/codigo_dimensiones_C.R

Rol: legacy · **Líneas:** 40

Descripción funcional Crea tablas de dimensiones del modelo estrella a partir de `Investigadores_Consolidado`: dimensión investigador (solo `ID_PERSONA_PR`), dimensión municipio de nacimiento (`COD_DANE_NAC_PR` + país/región/departamento/municipio) y dimensión convocatoria (`ID_CONVOCATORIA` + nombre + fecha de convocatoria), exportadas a tres `xlsx`. Corresponde a la tarea de dimensiones y hechos.

Análisis de la lógica `readxl` + `openxlsx` + `dplyr`: `unique()` sobre la columna de `ID`, `distinct(COD_DANE_NAC_PR, .keep_all = TRUE)` para el municipio, `distinct(ID_CONVOCATORIA, .keep_all = TRUE)` para la convocatoria, y `write.xlsx()` por dimensión. Sin joins; solo extracción de claves y descriptores.

Fortalezas

- Separa claves naturales (IDs) de atributos descriptivos, enfoque correcto de modelado dimensional
- Usa `distinct()` sobre la clave para garantizar unicidad en las dimensiones
- Genera artefactos accionables (`xlsx`) consumibles por la tabla de hechos

Debilidades

- Ruta hardcodeada `'Downloads/Investigadores_Consolidado.xlsx'`, inexistente en el clon
- Dimensión investigador sin atributos (solo `ID_PERSONA_PR`): tabla sin valor analítico y sin vínculo a institución/universidad
- `distinct(COD_DANE_NAC_PR, .keep_all = TRUE)` toma la PRIMERA fila por código: si el mismo código aparece con nombres distintos o con NA (investigadores en el 'Exterior'), la dimensión es no determinista
- No maneja NA: los códigos DANE faltantes (Exterior) entran como fila NA en la dimensión
- Nombres de salida con errores de tipo (`Dimension_invetigadores.xlsx`, `Dimension_municipios_de_naciminetos.xlsx`) que NO coinciden con los nombres que consume la tabla de hechos (`dimensiones_investigadores.xlsx` del notebook): dos implementaciones paralelas de dimensiones incompatibles entre sí

- Sin claves sustitutas para municipio y convocatoria; sin validación de cardinalidad
- No hay dimensión de formación, clasificación, área ni residencia aquí (la residencia se hace en el notebook, no en este script)

Mejoras propuestas

- Unificar con las dimensiones del notebook dimensiones.ipynb (una sola implementación)
- Añadir claves sustitutas y atributos a todas las dimensiones (incluida la de investigador)
- Decidir y documentar el tratamiento de 'Exterior'/NA antes de construir la dimensión
- Corregir nombres de archivo y validar la cardinalidad de la dimensión contra la tabla de hechos

Ejecutabilidad NO. Falta 'Downloads/Investigadores.Consolidado.xlsx' en el clon.

¿Produce lo esperado? El diseño (claves naturales + descriptores) es conceptualmente correcto para un modelo estrella, pero incompleto: la dimensión investigador no tiene atributos, la de municipio depende del orden de filas (.keep_all = TRUE) y no se resuelve el caso 'Exterior'. Riesgo de claves inconsistentes entre dimensiones de archivos distintos.

Riesgo / Limitación

Bugs / problemas detectados:

- Typos en los nombres de archivo de salida (investigadores, municipios de nacimientos) que rompen la trazabilidad con la tabla de hechos
- distinct() con .keep_all = TRUE: resultado dependiente del orden de filas en la base
- Filas con COD_DANE_NAC_PR = NA ('Exterior') entran como clave NA en la dimensión

Recomendación de auditoría Mantener como referencia histórica del modelo dimensional académico. No migrar: el modelo moderno está en src/modelo/dimensional.py; además conviene consolidar con el notebook de dimensiones para eliminar la duplicación incompatible.

4.10.5 notebooks/tarea_dimensiones_y_hechos/dimensiones.ipynb

Rol: legacy · **Líneas:** 289

Descripción funcional Notebook Colab que construye tres dimensiones del modelo estrella a partir de Investigadores.Consolidado: Dim_Universidad (INST_FILIA con clave sustituta 'Universidad'), Dim_Formacion (ID_NIV_FORMACION_PR, NME_NIV_FORM_PR, NRO_ORDEN_FORM_PR) y Dim_Residencia (COD_DANE_RES_PR + país/región/departamento/municipio), y las exporta a dimensiones_investigadores.xlsx (3

hojas) con descarga vía `files.download`. Es la implementación de dimensiones que luego consume `tabla_hechos_codigo.R`.

Análisis de la lógica `pandas + openpyxl + google.colab.files: dropna().drop_duplicates().reset_index(drop=True)` por dimensión, `insert(0, 'Universidad', range(1, len+1))` para la clave sustituta, `pd.ExcelWriter` con una hoja por dimensión y `files.download()`.

Fortalezas

- Agrega clave sustituta a `Dim_Universidad` (buena práctica de modelado dimensional)
- Separa dimensiones por tema y las exporta en un único workbook con hojas nombradas
- Conserva la jerarquía de formación en `NRO_ORDEN_FORM_PR` (ordinal), lo que permite ordenar después
- Salida estructurada y lista para el join de hechos

Debilidades

- Ruta `'Investigadores_Consolidado (1).xlsx'` (archivo subido a Colab), inexistente en el clon
- `INST_FILIA` contiene LISTAS separadas por PIPE: `drop_duplicates()` trata `'UNIV A|UNIV B'` como una `'universidad'` falsa (dimension inflada) y `dropna()` elimina a los investigadores sin filiación
- `Dim_Residencia` con `dropna()` elimina TODAS las filas con `COD_DANE_RES_PR NA`, es decir, todos los investigadores en el `'Exterior'` quedan fuera de la dimensión: la tabla de hechos luego hace `left_join` y pierde esas claves
- Sin clave sustituta para residencia y formación (solo claves naturales); la clave `'Universidad'` no se usa en el join de hechos (une por `INST_FILIA`)
- Modelo incompleto: no hay dimensión de nacimiento, de área de conocimiento ni de clasificación
- El código está comentado en su versión de upload (`files.upload`) y no es reproducible fuera de Colab

Mejoras propuestas

- Splitear `INST_FILIA` por `'|'` (formato largo o primera institución documentado) antes de deduplicar
- Incluir `'Exterior'` como categoría explícita (p. ej., código 0/99999) en lugar de `dropna()`
- Añadir claves sustitutas consistentes y dimensiones de nacimiento/área/clasificación
- Documentar la granularidad de la fila de hechos (persona x convocatoria) y validar cardinalidad de la dimensión
- Externalizar la lectura del Excel y la ruta de salida

Ejecutabilidad NO. Requiere subir el Excel en Colab (google.colab.files) y el archivo 'Investigadores_Consolidado (1).xlsx' no está en el clon; files.download() tampoco existe fuera de Colab.

¿Produce lo esperado? Diseño de dimensiones razonable para un modelo estrella académico, pero con pérdida real de datos: el dropna() de Dim_Residencia excluye al 'Exterior' y la granularidad de INST_FILIA con pipes queda mal modelada. Estos defectos se propagan a la tabla de hechos (joins con NA).

Riesgo / Limitación

Bugs / problemas detectados:

- dropna() en Dim_Residencia elimina las filas 'Exterior' (COD_DANE NA), dejando hechos sin clave de residencia
- INST_FILIA con pipes ('UNIV A|UNIV B') tratada como una sola institución en la dimensión
- La clave sustituta 'Universidad' no se corresponde con el join de la tabla de hechos (que une por INST_FILIA)

Recomendación de auditoría Mantener como referencia del diseño dimensional académico. No migrar: src/modelo/dimensional.py es el reemplazo moderno; si se conserva la idea, corregir el manejo de pipes y de 'Exterior'.

4.10.6 notebooks/tarea_dimensiones_y_hechos/tabla_hechos_codigo.R

Rol: legacy · **Líneas:** 20

Descripción funcional Construye la tabla de hechos del modelo estrella: hace left_join de Investigadores_Consolidado con Dim_Universidad (por INST_FILIA) y con Dim_genero (por NME_GENERO_PR), selecciona columnas por posición numérica y exporta Tabla_hechos.xlsx. Corresponde a la tarea de dimensiones y hechos.

Análisis de la lógica readxl + openxlsx + dplyr: dos left_join con las dimensiones, subconjunto por índices c(1,4,5,14,15,18,21,26,27:29,31,32) y write.xlsx(). La granularidad resultante es persona x convocatoria.

Fortalezas

- Integra las claves de dimensiones a la fila de hechos mediante joins, enfoque correcto de modelo estrella
- El join por INST_FILIA y NME_GENERO_PR es conceptualmente válido si las dimensiones son únicas por esas columnas

Debilidades

- Investigadores_Consolidado NO se define en este archivo: depende de un objeto global de una sesión R anterior (probablemente creado en codigo dimensiones_C.R). No reproducible en una sesión limpia
- Selección de columnas por posición numérica (índices mágicos) sin documentar qué columnas son: frágil ante cualquier cambio de esquema
- El join por INST_FILIA arrastra el problema de las dimensiones: investigadores sin filiación o con INST_FILIA en PIPE quedan con clave de universidad NA o mal asignada
- No verifica cardinalidad: si la dimensión no es única por INST_FILIA, el left_join duplicaría filas silenciosamente
- Rutas hardcodeadas 'Downloads/dimensiones_investigadores.xlsx' y 'Downloads/Dim_genero (1).csv', inexistentes en el clon
- Nombre de salida sin carpeta explícita ('Tabla.hechos.xlsx' en el working directory)

Mejoras propuestas

- Leer el Excel al inicio del script (autocontenido)
- Seleccionar columnas por nombre (dplyr::select) y documentar el diccionario de la tabla de hechos
- Validar que nrow() no cambie tras cada join y reportar claves NA
- Escribir con ruta explícita y verificable

Ejecutabilidad NO. Falta el objeto Investigadores_Consolidado (no definido en el archivo) y los inputs 'Downloads/dimensiones_investigadores.xlsx' y 'Downloads/Dim_genero (1).csv' no existen en el clon.

¿Produce lo esperado? La lógica de hechos es correcta para un modelo estrella simple (fila persona x convocatoria con claves de dimensión), pero la selección por posición y la dependencia de objetos globales la hacen propensa a errores. Nota: la tabla de hechos no contiene medidas/indicadores, solo claves; aceptable como entregable académico pero de uso analítico limitado.

Riesgo / Limitación

Bugs / problemas detectados:

- Investigadores_Consolidado no definido en el script (error de objeto no encontrado)
- Índices mágicos c(1,4,5,14,15,18,21,26,27:29,31,32) sin correspondencia verificable con el esquema
- Sin control de cardinalidad tras los joins (riesgo de duplicación de filas)

Recomendación de auditoría Mantener como referencia del entregable de hechos. **No migrar:** `src/modelo/dimensional.py` genera el modelo estrella moderno; si se quisiera conservar, reescribir con joins por nombre, lectura autocontenida y validaciones.

4.10.7 notebooks/tarea_gran_tabla/analisis2.ipynb

Rol: legacy · **Líneas:** 294

Descripción funcional Análisis bivariado de la GRAN_TABLA(SIN ID): matriz de correlación entre numéricas, countplots con porcentajes (género x convocatoria, clasificación x gran área, formación x clasificación), tablas de contingencia con pruebas chi-cuadrado y coeficiente de contingencia de Pearson (clasificación x región, formación x región, convocatoria x gran área), y Kruskal-Wallis + post-hoc de Dunn con Bonferroni para edad según gran área. NOTA: a pesar de la extensión .ipynb, el contenido es un script Python exportado de Colab (no es un notebook JSON), lo que sugiere una descarga mal nombrada.

Análisis de la lógica pandas + seaborn + scipy: `pd.crosstab`, `chi2.contingency`, `np.sqrt(chi2/(chi2+n))` para el coeficiente de contingencia, `scipy.stats.kruskal` y `scikit_posthocs.posthoc_dunn(p_adjust='bonferroni')`. Incluye anotación manual de porcentajes sobre barras apiladas y `!pip install` dentro del script.

Fortalezas

- Usa chi-cuadrado con coeficiente de contingencia de Pearson: medida de asociación adecuada para variables nominales
- Elige Kruskal-Wallis (no paramétrico) para edad por gran área: robusto ante la no normalidad de la edad
- Aplica post-hoc de Dunn con corrección Bonferroni tras Kruskal-Wallis: secuencia inferencial correcta
- Ordena la tabla de contingencia por jerarquía de clasificación (ORDEN_CLAS_PR) para lectura sustantiva

Debilidades

- La extensión .ipynb es engañosa: el archivo es un script .py exportado de Colab; no abre como notebook
- Ruta hardcodeada `'/content/GRAN_TABLA(SIN ID).xlsx'`, inexistente en el clon
- `!pip install scikit-posthocs` dentro del código: magia de notebook que falla como script y rompe la reproducibilidad de dependencias

- Chi-cuadrado sin verificación de frecuencias esperadas: las tablas región x clasificación y región x formación tienen celdas pequeñas ('Exterior', 'No disponible') que invalidan la aproximación asintótica
- Correlación de Pearson sobre variables ordinales (NRO_ORDEN_FORM_PR, ORDEN_CLAS_PR) tratadas como continuas; lo adecuado es Spearman
- El bloque de anotación de porcentajes usa aritmética frágil sobre ticks de matplotlib (código muerto 'categoria' y riesgo de división por cero)
- No guarda resultados ni figuras; sin semilla (no aplica) ni control de múltiples comparaciones más allá de Bonferroni

Mejoras propuestas

- Renombrar el archivo a .py y eliminar la magia !pip install (declarar dependencia en requirements)
- Parametrizar la ruta de lectura
- Reportar la mínima frecuencia esperada o agrupar categorías raras antes del chi-cuadrado (o usar test exacto)
- Usar Spearman para ordinales y simplificar las anotaciones de barras
- Guardar tablas de resultados en CSV

Ejecutabilidad NO. Falta '/content/GRAN_TABLA(SIN ID).xlsx' y las magias de Colab (!pip install) fallan al ejecutarlo como script Python estándar.

¿Produce lo esperado? Metodología inferencial mayormente correcta: chi-cuadrado para asociación de nominales y Kruskal-Wallis + Dunn para edad por grupo. Riesgo: celdas con frecuencias esperadas bajas debilitan el chi-cuadrado, y con $n \sim 50.000$ casi cualquier asociación resulta significativa, por lo que el coeficiente de contingencia (bien calculado) es la métrica realmente informativa.

Riesgo / Limitación

Bugs / problemas detectados:

- Extensión .ipynb con contenido de script .py (no es JSON de notebook)
- !pip install dentro del archivo (falla fuera de notebook)
- Lógica de anotación de barras dependiente del espaciado de xticks (puede dividir por cero o etiquetar mal con hue)
- Pearson aplicado a variables ordinales

Recomendación de auditoría Mantener como referencia del análisis bivariado de la gran tabla. No migrar tal cual: los análisis equivalentes modernos están en src/analisis/ (genero.py, territorial.py, produccion.py); si se quiere conservar

el chi-cuadrado, reimplementarlo en el pipeline con validación de supuestos y guardado de resultados.

4.10.8 notebooks/tarea_gran_tabla/analisis_de_clusters.R

Rol: legacy · **Líneas:** 94

Descripción funcional Clustering k-prototypes sobre la GRAN_TABLA(SIN ID) para datos mixtos: convierte las categóricas en factor/ordinal, elimina casos incompletos con `cc()`, aplica `kproto(k = 3)` con `set.seed(123)` y describe los clusters con tablas de proporción por variable (gran área, género, región de nacimiento/residencia, formación, clasificación, discapacidad) y medias de edad. Corresponde a la tarea de análisis multivariado de la gran tabla.

Análisis de la lógica `klaR + clustMixType + factoextra + dplyr`: `mutate(across(c(1:10,16:23), factor))` y `ordered()` para formación/clasificación), `cc()` para casos completos (retiene el 91.6 % de la base), `kproto(GT_NA, k = 3)`, `filter` por cluster y `table()/mean()` por grupo para perfilado.

Fortalezas

- Elección correcta del método: k-prototypes (`clustMixType`) es el algoritmo apropiado para datos mixtos categórico-numérico
- Fija semilla (`set.seed(123)`): el resultado del `kproto` es reproducible
- Reporta explícitamente la proporción de datos retenidos tras eliminar casos incompletos (91.6 %)
- Perfilado de clusters con proporciones por variable y media de edad: enfoque descriptivo adecuado para interpretar grupos

Debilidades

- `k = 3` fijado sin validación: no hay curva de `withinss/silhouette`, ni comparación de varios `k`, ni análisis del `lambda` (importancia de variables) de `kproto`
- `cc()` (función de casos completos del paquete `mice`) elimina el 8.4 % de las filas sin evaluar sesgo: los registros faltantes pueden ser sistemáticos (p. ej., 'Exterior', sin filiación, sin código DANE)
- `mice` NO está cargado en este script (solo `readxl`, `klaR`, `dplyr`, `factoextra`, `clustMixType`): `cc()` solo existe si se ejecutó antes `analisis_multivariado_base_gran_table.R` en la misma sesión: no reproducible
- Incluye el outlier de edad (956 años) en el clusterizado sin filtrar (los análisis previos sí lo descartaban)

- Categorías 'No disponible'/'Exterior' dominantes distorsionan las distancias mixtas del kproto (el algoritmo estandariza categorías de forma uniforme)
- El perfilado es solo inspección visual de proporciones: sin chi-cuadrado/ANOVA por variable vs cluster ni tamaño de clusters reportado
- Ruta hardcodeada 'Downloads/GRAN_TABLA(SIN ID).xlsx', inexistente en el clon

Mejoras propuestas

- Validar k con varios valores (p. ej., $k = 2..8$) usando métricas de clustMixType (lambda, withinss) o silhouette para datos mixtos
- Cargar mice (o definir casos completos con complete.cases()) y documentar el sesgo de la eliminación; evaluar imputación como alternativa
- Filtrar edad >100 antes de clusterizar (coherencia con la limpieza previa)
- Reportar el lambda de cada variable (contribución al clustering) y validar perfiles con pruebas formales
- Documentar la granularidad y el número de filas por cluster

Ejecutabilidad NO. Falta 'Downloads/GRAN_TABLA(SIN ID).xlsx' en el clon y, en una sesión limpia, cc() no existe porque mice no está cargado (depende de ejecutar antes el script multivariado en la misma sesión R).

¿Produce lo esperado? El método (k-prototypes) es el adecuado para datos mixtos, pero la selección de k sin validación y la eliminación de NA por casos completos sin evaluar sesgo son errores metodológicos: los clusters pueden ser artefactos del número de grupos impuesto y de la pérdida de 8.4 % de la muestra. Los perfiles por proporción son descriptivos y no validan la estructura encontrada.

Riesgo / Limitación

Bugs / problemas detectados:

- cc() (mice) usado sin cargar mice en el script: dependencia oculta de sesión previa
- $k = 3$ fijo sin justificación ni validación (no hay análisis de sensibilidad a k)
- Outlier de edad (956 años) incluido en el clusterizado
- Eliminación de casos incompletos (8.4 %) sin análisis de sesgo

Recomendación de auditoría Mantener como referencia académica de la tarea (muestra el uso correcto de k-prototypes para datos mixtos). No migrar: el pipeline moderno no incluye clustering; si se reintroduce, hacerlo con validación de k, manejo de NA documentado y datos depurados.

4.10.9 notebooks/tarea_gran_tabla/analisis_multivariado_base_gran_table.R

Rol: legacy · **Líneas:** 405

Descripción funcional Análisis multivariado de la gran tabla: análisis de correspondencia (CA) para pares de categóricas (gran área x género, INST_FILIA x género, nivel de formación x clasificación), análisis de correspondencia múltiple (MCA) para conjuntos de categóricas (5 variables; 3 variables de población) y FAMD (factor analysis of mixed data) con ANO_CONVO, gran área, género, país de nacimiento y EDAD_ANOS_PR. Reporta eigenvalores, contribuciones y cos2, y construye biplots con ggrepel. Incluye análisis de faltantes (plot_missing, md.pattern) y retira una fila totalmente vacía.

Análisis de la lógica FactoMineR (CA, MCA, FAMD) + factoextra + ggplot2 + ggrepel + mice + DataExplorer: table() ->CA(bc/ bd/ bf, graph = F), MCA(GT[, c(2,5,6,7,16)], graph = F) y MCA(GT[, c(20:22)], ncp = 16), FAMD(GT[-50891, c(2,5,6,7,15)], ncp = 20, graph = T); extrae b\$eig, \$row/\$col/\$var/\$quant.var/\$quali.var coord/cos2/contrib y grafica varianza y biplots.

Fortalezas

- Elección correcta de método según tipo de datos: CA para tablas de contingencia 2 x k, MCA para múltiples categóricas, FAMD para datos mixtos
- Reporta contribuciones y cos2 (calidad de representación de filas/columnas), buenas prácticas de lectura de CA/MCA/FAMD
- Detecta y elimina la fila totalmente vacía (md.pattern / lis.finite) antes del FAMD
- Gráficos de varianza explicada y acumulada con umbral del 70 % para decidir número de dimensiones

Debilidades

- Ruta hardcodeda 'Downloads/GRAN_TABLA(SIN ID).xlsx', inexistente en el clon
- CA sobre INST_FILIA x género con miles de filas (instituciones y combinaciones PIPE sin normalizar): tabla dispersa donde las categorías con frecuencia 1 dominan el chi-cuadrado; resultado poco interpretable
- Remoción de la fila 50891 por índice hardcodedo (GT[-50891, ...]): frágil si cambia la base; debería ser un filtro por contenido
- Comentario del MCA (dice NME_PAIS_RES_PR) no coincide con el código (columna 16): documentación desalineada
- Los CA no van acompañados de chisq.test: no se reporta significancia ni frecuencias esperadas

- MCA/FAMD con categorías 'No disponible'/'Exterior' y NAs sin tratamiento explícito documentado (depende del default `na.method` de `FactoMineR`)
- FAMD con `ncp = 20` para solo 5 variables: sobredimensionado
- No guarda resultados (coordenadas/contribuciones) en archivos

Mejoras propuestas

- Excluir `INST_FILIA` del CA o agregar a nivel de institución (eliminar pipes) antes de analizar
- Acompañar cada CA con `chisq.test` y verificación de frecuencias esperadas
- Eliminar la fila vacía con un filtro (p. ej., `rowSums(is.na()) < ncol`) en vez del índice 50891
- Alinear comentarios con columnas y documentar el tratamiento de NA
- Guardar `eig/coord/cos2/contrib` en `xlsx` para trazabilidad

Ejecutabilidad NO. Falta 'Downloads/GRAN_TABLA(SIN ID).xlsx' en el clon.

¿Produce lo esperado? Metodología en general correcta: CA/MCA/FAMD están bien aplicados según el tipo de variables. Riesgos: el CA sobre `INST_FILIA` con tabla dispersa infla la inercia de categorías raras (poca interpretabilidad); FAMD con `ncp=20` es inofensivo pero sobredimensionado; la lectura de contribuciones sin pruebas de significancia puede llevar a sobreinterpretación.

Riesgo / Limitación

Bugs / problemas detectados:

- Índice hardcodeado 50891 para eliminar la fila vacía (rompe con cualquier cambio de la base)
- Comentario del MCA (`NME_PAIS_RES_PR`) desalineado con la columna 16 del código
- CA sobre `INST_FILIA` sin normalizar los valores PIPE (instituciones múltiples tratadas como categorías únicas)
- Sintaxis `ggplot` con `aes(shape = '..')` dentro de `geom_point` que no varía la forma real (leyenda engañosa)

Recomendación de auditoría Mantener como referencia académica (buen ejemplo de aplicación de CA/MCA/FAMD). No migrar tal cual: el pipeline moderno no realiza análisis factorial; si se requiere en el futuro, reimplementarlo con datos limpios, validación de supuestos y guardado de resultados.

4.10.10 notebooks/tarea_gran_tabla/codigo_analisis_univariado.py

Rol: legacy · **Líneas:** 123

Descripción funcional Análisis univariado de la gran tabla: revisa tipos de datos, describe() de las numéricas (NRO_ORDEN_FORM_PR, ORDEN_CLAS_PR, EDAD_ANOS_PR) con % de faltantes, histogramas y boxplots, value_counts de las categóricas, gráficos de barras de las principales variables y un bloque robusto de parsing de ANO_CONVO (texto dd/mm/aaaa o serial de Excel) para contar investigadores por año. Corresponde a la tarea de análisis univariado de la gran tabla.

Análisis de la lógica pandas + seaborn + matplotlib: describe().T con missing_% agregada, sns.histplot/boxplot, value_counts(dropna=False), y parsing de fecha con doble estrategia: pd.to_datetime(dayfirst=True) y, como respaldo, pd.to_datetime(serial, unit='D', origin='1899-12-30'); barplot por año de convocatoria.

Fortalezas

- Parsing de fechas robusto con doble estrategia (texto y serial de Excel): buena práctica defensiva para datos heterogéneos
- Reporta % de faltantes junto con los estadísticos descriptivos
- Separa explícitamente variables numéricas, temporales y categóricas antes de analizar
- Análisis claro y estructurado, fácil de seguir

Debilidades

- Ruta hardcodeada '/content/GRAN_TABLA(SIN ID).xlsx' (Colab), inexistente en el clon
- Lee el Excel dos veces (todo el archivo y luego solo la columna ANO_CONVO): ineficiente y evidencia de parche sobre el dato original
- Inconsistencia en el tratamiento de NA: las tablas usan value_counts(dropna=False) pero los gráficos usan el default dropna=True
- Las columnas numéricas llegan como texto/object (heredado de la creación de la gran tabla con dtype='object' en codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py), obligando a conversiones ad hoc: síntoma de un problema de tipos aguas arriba
- Sin pruebas estadísticas ni guardado de resultados

Mejoras propuestas

- Leer ANO_CONVO una sola vez y reutilizar el DataFrame
- Uniformizar el tratamiento de NA entre tablas y gráficos

- Corregir el dtype en el origen (creación de la gran tabla) en vez de parchear aquí
- Parametrizar la ruta y guardar el resumen descriptivo en CSV

Ejecutabilidad NO. Falta `'/content/GRAN_TABLA(SIN ID).xlsx'` (no existe en el clon; es una ruta de Colab).

¿Produce lo esperado? Correcto como EDA univariado: no hay errores metodológicos porque no realiza inferencia. El parsing defensivo de fechas es una buena práctica. El único riesgo es interpretar estadísticos de variables ordinales (NRO_ORDEN_FORM_PR) como continuas, pero el script no extrae conclusiones inferenciales.

Riesgo / Limitación

Bugs / problemas detectados:

- Doble lectura del Excel (desperdicio y riesgo de discrepancias entre lecturas)
- `value_counts` con `dropna=False` en tablas vs `dropna=True` en gráficos (conteos inconsistentes)
- Dependencia de parche: supone que ANO_CONVO viene mal tipada y la re-parsea

Recomendación de auditoría Mantener como referencia del EDA univariado de la gran tabla. No migrar: `src/analisis/calidad.py` y los scripts de sprint (`sprint2_duckdb.py`, `sprint2_genero_ocde.py`) ya cubren descriptivos y perfiles con datos tipados correctamente.

4.10.11 notebooks/tarea_gran_tabla/codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py

Rol: legacy · **Líneas:** 205

Descripción funcional Ensambla la gran tabla a partir del modelo estrella: lee todos los CSV/Excel de una carpeta, detecta la tabla de hechos por nombre (regex `'hecho|hechos|fact'`) o por máximo de filas, detecta claves de unión con heurísticas (columnas comunes únicas, llaves compuestas, columnas tipo ID) y hace merges left con guarda anti-1:N; exporta `gran_tabla.*` y un reporte de uniones, y luego elimina columnas ID para producir GRAN_TABLA(SIN ID). Corresponde a la tarea de creación de la gran tabla a partir del modelo estrella.

Análisis de la lógica `pandas + re + pathlib`: `read_any()` por archivo/hoja, `guess_fact_name()`, `candidate_keys_for_dim()` (`nunique == len`, combinaciones únicas, columnas `id_like`), `safe_left_join()` que revierte el merge si aumenta el número de filas (1:N), `merge(how='left')` con `rename` de columnas con prefijo de dimensión, `drop` de IDs y export a CSV/Excel (`xlsxwriter/openpyxl`).

Fortalezas

- Heurísticas de detección de claves razonables y defensivas: verifica unicidad y revierte joins 1:N en lugar de duplicar filas silenciosamente
- Genera un reporte de uniones (reporte uniones.csv/xlsx) que permite auditar qué dimensiones se integraron y con qué claves
- Separa la versión completa (con IDs) de la versión analítica sin IDs (GRAN_TABLA(SIN ID))
- Tolerante a formatos múltiples (CSV/Excel y múltiples hojas)

Debilidades

- Ruta hardcodeada `'/content/A123'` (placeholder del drive del autor), inexistente en el clon; y `claves_forzadas` apunta a `'MiArchivo__Dimension_municipios_de_naciminetos_R'`, nombre que nunca coincide con los archivos reales del repo: la única clave forzada no aplica y todo queda a merced de las heurísticas
- `!pip install xlswriter` y `display()` dentro del script: magias de Colab que fallan como script estándar
- Lee todo con `dtype='object'`: las columnas numéricas salen como texto y contaminan la gran tabla (por eso los análisis posteriores re-parsan fechas y números, p. ej., `codigo_analisis_univariado.py`)
- La heurística de clave puede unir por una columna común equivocada (p. ej., `NME_CONVOCATORIA`) sin verificación semántica de que sea clave en ambas tablas
- Renombra columnas con prefijos largos tipo `'Archivo__Hoja_col'`, generando nombres difíciles de usar
- No versiona inputs ni dimensiones: el resultado depende del contenido exacto de la carpeta
- Import de re duplicado (líneas 11 y 13) y variable `pat_fact` definida dos veces

Mejoras propuestas

- Eliminar magias de Colab (`pip/display`) y parametrizar la carpeta de entrada
- Leer con tipos inferidos o tipar explícitamente las columnas numéricas
- Verificar semánticamente las claves (unicidad en hechos y en dimensión) y agregar validación cruzada
- Persistir el reporte de uniones como artefacto del pipeline y versionar los inputs
- Usar prefijos de columna cortos y estables

Ejecutabilidad NO. Falta la carpeta `'/content/A123'` con las tablas del modelo estrella (y los nombres forzados `'MiArchivo_...'` no existen en el repo); además `!pip install` y `display()` son incompatibles con un script estándar.

¿Produce lo esperado? La lógica de ensamblaje (star schema -> tabla denormalizada) es correcta en concepto y tiene buenas salvaguardas (reversión de 1:N, reporte de uniones). El problema grave es `dtype='object'`, que degrada tipos numéricos/ordenados y obliga a parches aguas abajo, y la falta de versionado de inputs, que hace el resultado no reproducible con los datos actuales del repositorio.

Riesgo / Limitación

Bugs / problemas detectados:

- `dtype='object'` al leer Excel convierte todas las columnas en texto (pérdida de tipos numéricos)
- `claves_forzadas` apunta a un nombre inexistente ('MiArchivo__Dimension_municipios_de_naciminetos_R'): nunca se usa
- `!pip install` dentro de un archivo `.py` (falla fuera de notebook)
- La detección de la tabla de hechos por máximo de filas puede elegir la tabla equivocada si ninguna coincide con la regex

Recomendación de auditoría Mantener como referencia del proceso de ensamblaje (buena idea de reporte de uniones). No migrar tal cual: el modelo moderno (`src/modelo/dimensional.py` + `scripts/sprint2_duckdb.py`) ya construye la gran tabla con tipos correctos y control de calidad; este script solo aporta la idea de auditar las uniones.

4.10.12 notebooks/tarea_join/codigo_join.py

Rol: legacy · **Líneas:** 26

Descripción funcional Une verticalmente (concatenación) las bases por convocatoria 2017, 2019 y 2021 (`Investigadores_Reconocidos_por_convocatoria_20250829*.csv`) para crear `Investigadores_Consolidado.xlsx` y `.csv` y descargarlos. Corresponde a la tarea de join (unión de las bases por convocatoria en una base consolidada).

Análisis de la lógica pandas: `pd.concat([df.2017, df.2019, df.2021], ignore_index=True)`, `to_excel()` y `to_csv()`, y descarga con `google.colab.files.download()`. Operación de unión vertical (union all de registros persona x convocatoria).

Fortalezas

- Código simple y claro: hace exactamente lo que la tarea pide (unión vertical de las tres convocatorias)
- Genera la base en ambos formatos (Excel y CSV) para consumo posterior

Debilidades

- Rutas hardcodeadas de Colab `’/content/Investigadores_Reconocidos_por_convocatoria_20250829*.csv’`, inexistentes en el clon
- `from google.colab import files`: solo funciona en Colab; como script local falla
- `concat` sin verificar que las tres bases tengan el mismo esquema (mismas columnas, mismo orden, mismos tipos): un cambio de esquema genera columnas NaN o tipos mezclados silenciosamente
- Sin deduplicación ni control de calidad: los duplicados de `ID_PERSONA_PR` se manejan después en `arreglar_base_consultoria_7.R` (acoplamiento frágil entre scripts)
- Versiones de archivo inconsistentes entre tareas: aquí 20250829, pero `2019_codigo.py` usa 20250909 e `inv_17_codigo_graficas.R` usa 20250831: los análisis de distintas tareas NO se hicieron sobre el mismo snapshot de datos
- Sin documentación de la granularidad resultante (fila = investigador x convocatoria)

Mejoras propuestas

- Verificar igualdad de columnas y tipos antes del `concat` (`assert` o comparación de listas)
- Deduplicar con criterio documentado en el mismo script o validar en uno solo
- Parametrizar rutas y fijar una única versión de archivos fuente para todo el proyecto
- Registrar un manifiesto de versiones de inputs

Ejecutabilidad NO. Faltan los tres CSV en `’/content/’` (no existen en el clon) y `google.colab.files` no existe fuera de Colab.

¿Produce lo esperado? El `concat` vertical es la operación correcta para unir convocatorias (no hay error metodológico en la unión en sí). El riesgo es la ausencia de control de esquema y de deduplicación, que genera inconsistencias aguas abajo (duplicados, tipos mezclados) que luego otros scripts deben parchear.

Riesgo / Limitación

Bugs / problemas detectados:

- Import de `google.colab.files` fuera de Colab (`ImportError`)
- `concat` sin validación de esquema: si una convocatoria cambia de columnas, se crean NaN silenciosos
- Versiones de archivo fuente distintas entre tareas (20250829 vs 20250909 vs 20250831): datos no comparables entre scripts

Recomendación de auditoría Mantener como referencia histórica de la tarea de `join`. No migrar: `src/ingesta/minciencias.py` hace la ingesta consolidada moderna con validación de esquema y versionado de datos.

4.10.13 notebooks/tarea_tabla/arreglar_base_consultoria_7.R

Rol: legacy · **Líneas:** 125

Descripción funcional Depura Investigadores_Consolidado: diagnostica duplicados de ID_PERSONA_PR, deduplica conservando una fila por persona con prioridad de convocatoria ($21 > 20 > 19$) mediante `slice(1)`, y audita las diferencias de edad entre registros duplicados ($\text{dif_edad} > 4.1$) revisando casos anómalos de forma manual con View. Corresponde a la tarea de arreglo de la base (preparación para análisis a nivel de persona).

Análisis de la lógica `readxl + dplyr: duplicated()` y `sum(table(...)) > 1` para diagnóstico, `group_by(ID_PERSONA_PR) + mutate(prioridad = case_when(ID_CONVOCATORIA == 21 ~ 3, ...)) + arrange(desc(prioridad)) + slice(1) + ungroup()` para deduplicar, y `mutate(dif_edad = EDAD_ANOS_PR - dplyr::lag(EDAD_ANOS_PR))` dentro de `group_by` para auditar la coherencia temporal de la edad entre convocatorias.

Fortalezas

- Enfoque de calidad de datos correcto: detecta duplicados, elige el registro con una jerarquía de convocatoria explícita y verificable, y valida la unicidad después de `slice(1)`
- Control de calidad interesante: calcula la diferencia de edad entre registros duplicados de la misma persona (debería ser ~2-4 años entre convocatorias) y detecta inconsistencias ($\text{dif_edad} > 4.1$)
- Decisión documentada en el código (comentarios con los casos revisados y sus diferencias)
- Reporta la proporción de filas retenidas tras la deduplicación

Debilidades

- Ruta hardcodeada 'Downloads/Investigadores_Consolidado.xlsx', inexistente en el clon
- `write.xlsx()` sin cargar `library(openxlsx)` (solo `readxl` y `dplyr` están cargados) y sin extensión en el nombre ('Base_sin_duplicados'): error en ejecución y archivo sin formato
- La prioridad por convocatoria crea un snapshot heterogéneo: los investigadores que aparecen en varias convocatorias quedan representados con sus datos del año más reciente, mientras que los singletons conservan el único año que tienen; la edad, formación y clasificación de distintas personas provienen de años distintos (sesgo de selección de año)
- Los casos con $\text{dif_edad} > 4.1$ se inspeccionan visualmente pero NO se decide ni se registra su tratamiento (no se excluyen, no se corrigen): auditoría incompleta
- `slice(1)` sin desempate: si la misma persona tiene dos registros con la misma prioridad (duplicado dentro de la misma convocatoria), se elige una fila arbitraria (no determinista)

- No valida el resultado contra el total esperado ni contra la base original más allá de nrow ratio

Mejoras propuestas

- Cargar openxlsx y escribir 'Base_sin_duplicados.xlsx' con ruta explícita
- Documentar y aplicar una política para los casos con dif.edad >4.1 (excluir, imputar o marcar)
- Añadir un orden determinista dentro de slice(1) (p. ej., arrange por año o por nivel de formación)
- Evaluar el sesgo del snapshot mixto: reportar cuántos registros provienen de cada convocatoria tras la deduplicación
- Comparar la edad imputada vs la declarada entre convocatorias como control adicional

Ejecutabilidad NO. Falta 'Downloads/Investigadores_Consolidado.xlsx' en el clon y write.xlsx() falla por no tener cargado openxlsx (además el nombre de archivo carece de extensión).

¿Produce lo esperado? La deduplicación con jerarquía es una decisión válida para análisis a nivel persona, pero introduce un sesgo de selección de año (mezcla de snapshots temporales). El análisis de dif.edad es un buen control de calidad incompleto: identifica anomalías pero no las resuelve ni documenta la decisión. La base resultante solo se valida por conteo, no por contenido.

Riesgo / Limitación

Bugs / problemas detectados:

- write.xlsx() sin library(openxlsx) y con nombre sin extensión (error de ejecución y archivo sin formato)
- slice(1) sin orden determinista ante empates de prioridad (resultado no reproducible)
- Caso de auditoría (dif.edad >4.1) revisado manualmente sin decisión ni registro del tratamiento

Recomendación de auditoría Mantener como referencia del proceso de limpieza y deduplicación. No migrar: la limpieza moderna está en src/ingesta/minciencias.py y src/analisis/calidad.py; si se necesita la base persona-nivel, reimplementar con política de año explícita, desempate determinista y registro de decisiones de calidad.

5 Validación de la calidad de pruebas, tratamientos y lógica estadística

Esta sección evalúa críticamente el rigor estadístico de los análisis implementados, por eje metodológico.

5.1 Tratamiento de datos (preprocesamiento)

- **Normalización de texto y categorías** (`src/Transformacion.py`): correcta y consistente (mayúsculas, NO REPORTADO).
- **Imputación por mediana** en variables numéricas (`NRO_ORDEN_FORM_PR`, `ORDEN_CLAS_PR`, `EDAD_ANOS_PR`): **cuestionable**. Imputar la mediana reduce la varianza estimada y puede sesgar análisis de dispersión; además `ORDEN_*` son ordinales, no continuas. Se recomienda imputación múltiple o modelos que manejen faltantes (p. ej. `mice`), o al menos documentar el impacto.
- **Atípicos de edad** (`src/analisis/calidad.py`): decisión **correcta y bien documentada** (eliminación sobre imputación por trazabilidad; 22 casos = 0.028 % del padrón; sesgo despreciable). Se ofrece comparación con imputación por mediana del grupo.
- **Parsing de años**: manejo robusto de 3 formatos (fecha, serial de Excel, regex) — **bien resuelto**.

5.2 Análisis longitudinal

- **Matrices de transición de Markov** (`src/analisis/longitudinal.py`): implementación **correcta** (conteos normalizados por fila, categorías ordenadas por escala ordinal, estado “Desaparece” opcional).
- **Caveats no tratados**: la propiedad markoviana (la transición solo depende del estado actual) y la homogeneidad temporal no se testean; con intervalos irregulares entre convocatorias, las tasas no son directamente comparables entre periodos. La literatura recomienda matrices de orden superior o modelos de transición con covariables.
- **Tasas de retención**: definición correcta (presentes en t_0 que reaparecen en t_1); el hallazgo de los méritos vitalicios (estado absorbente por diseño) está bien documentado.
- **Ausencias**: no se usa análisis de supervivencia (Kaplan–Meier/Cox) para el tiempo hasta la salida, que es el complemento natural y de mayor valor inferencial.

5.3 Análisis territorial

- **Índice HHI**: **correctamente implementado** y calculado por convocatoria y por región (Tabla con HHI decreciente 0.190 → 0.146, lo que sugiere leve desconcentración).
- **Limitaciones**: HHI sin intervalos de confianza (bootstrap); no se complementa con Gini/Lorenz/Theil; la comparación per cápita usa el censo DANE 2018 como denominador fijo, lo que ignora la dinámica poblacional (2013–2021).

5.4 Análisis de género

- **Indicadores:** proporciones de participación femenina por gran área OCDE y evolución temporal — **correctos** y alineados con la literatura (Larivière et al., 2013).
- **Limitaciones:** no hay inferencia formal (intervalos de confianza, pruebas de diferencia de proporciones o modelos logísticos ajustados por área/cohorte); la variable género es auto-declarada y con 2.7 % de “NO REPORTADO”.

5.5 Análisis de diversidad

- **Representación relativa vs DANE 2018 / RUV 2021:** **metodología correcta** (razón observado/esperado); hallazgos coherentes (afro 3×, indígena 7.9×, discapacidad 8× subrepresentados).
- **Caveats:** variables auto-declaradas y solo disponibles desde 2021 (0 % de cobertura antes), por lo que no hay serie temporal; los conteos de subgrupos pequeños requieren IC bootstrap; la comparación con DANE 2018 mezcla años de referencia.

5.6 Análisis de redes

- **Construcción del grafo** (co-filiación a partir de INST_FILIA con separador |): **lógica correcta**, con normalización de nombres de institución y métricas por convocatoria.
- **Limitaciones:** no se contrastan las métricas de centralidad contra modelos nulos (grafos aleatorios de configuración); la captura selectiva (solo 2019 con co-filiación múltiple) hace que la red sea un corte transversal; falta análisis de robustez.

5.7 Análisis de productividad (cruce con producción)

- **Cobertura:** 36.9 % de autores únicos en producción son investigadores reconocidos — hallazgo relevante y correctamente calculado.
- **Productividad por categoría:** conteos medios (Junior 30 / Asociado 65 / Senior 121) — **descriptivo correcto**; la literatura recomienda contrastar la concentración con la ley de Lotka y normalizar por área y ventana.
- **Brecha de género en outputs:** 5 de 6 grandes áreas con brecha — hallazgo válido; falta ajuste por composición de área.

5.8 Clustering y análisis multivariado (legacy R)

- **CA/MCA** (`analisis_multivariado_base_gran_table.R`): uso de correspondencia simple (CA) sobre tablas de contingencia (área × género) — **adecuado para 2 variables**; para más variables habría requerido MCA/FAMD.
- **k-prototypes** (`analisis_de_clusters.R`): **algoritmo apropiado para datos mixtos**; pero se fija $k = 3$ sin validación (silhouette/Calinski-Harabasz) y se eliminan casos con NA (`cc()`), perdiendo 8.4 % de la muestra. No reproducible por rutas hardcodeadas.

- **Verificación de unicidad de IDs** (heurística edad ± 4 años + género): razonable como primer filtro, pero sin formalización (no se reportan tasas de conflicto ni revisiones manuales).

5.9 Veredicto estadístico

Hallazgo

La **lógica estadística descriptiva y exploratoria es sólida** y los hallazgos principales son válidos para el alcance planteado. La **inferencia formal es la principal debilidad**: faltan intervalos de confianza, pruebas de supuestos (markovianidad, independencia en paneles/redes), modelos de supervivencia y validación de clusters. Ninguno de estos vacíos invalida los hallazgos ya producidos, pero limita su generalización y su uso en decisiones de política.

6 Lógica de construcción del repositorio y cumplimiento del objetivo

6.1 Cómo se construyó el repositorio

El repositorio evolucionó en dos grandes etapas (ver línea de tiempo en la Sección 7):

1. **Etapla académica inicial (ago–oct 2025)**: carpetas `notebooks/tarea_*` con análisis por convocatoria (EDA en R y Python), unión de bases (`codigo_join.py`), verificación de unicidad de IDs, construcción de dimensiones/hechos y “gran tabla”, análisis univariado/multivariado y clustering. En esta etapa el proyecto se gestionaba como tareas independientes, sin pipeline unificado.
2. **Etapla de ingeniería de datos (oct 2025 – may 2026)**: adopción de CRISP-DM, creación de `src/` (ingesta, transformación, análisis, modelo dimensional), `scripts/sprint*_*.py` como orquestadores, DuckDB como almacén, dashboard Streamlit, documentación (LaTeX, Reveal.js, manual) y CI/CD. Se amplió el alcance de 3 a 6 convocatorias y se integró el dataset de producción (Sprint 5) y los ajustes del director (Sprint 6).

6.2 ¿La lógica de construcción es correcta?

Hallazgo

La **arquitectura final es correcta y apropiada** para el objetivo: el flujo ingesta $\rightarrow$ transformación $\rightarrow$ modelo dimensional $\rightarrow$ análisis por sprint $\rightarrow$ visualización sigue el estándar de un observatorio de datos (Kimball; CRISP-DM). La separación de responsabilidades en `src/` es limpia y la orquestación por sprints es trazable.

6.3 Flujos, herramientas y pasos incorrectos o no óptimos

1. **CI/CD rota**: el workflow de GitHub Actions referencia módulos inexistentes; la ingesta automatizada mensual nunca podría completarse.
2. **Dockerfile con placeholder**: el contenedor desplegaría la app vacía `app/streamlit_app.py` en lugar del dashboard real.
3. **Duplicación de apps**: `streamlit_app.py` (raíz, 983 líneas, dashboard real) vs `app/streamlit_app.py` (17 líneas, placeholder) — fuente de confusión en despliegues.
4. **Legacy no integrado**: el clustering y el análisis multivariado viven en R fuera del pipeline; scikit-learn está declarado en `pyproject.toml` pero no se usa.
5. **Versionado de datos incompleto**: los datos crudos y procesados están gitignored sin DVC; la reproducibilidad end-to-end depende de la disponibilidad de la API Socrata.
6. **Testing insuficiente**: un solo test; `pandera` sin uso; sin tests de contrato ni de funciones analíticas.
7. **Codificación**: el CSV canónico con mojibake no fue detectado por ningún control automático.
8. **Artefactos duplicados**: variantes de matrices de transición sin documentación de diferencias.
9. **Referencias desactualizadas**: Grupo7/Grupo8, ramas y repos antiguos en el README.
10. **Rutas hardcodeadas** en los scripts legacy (`Downloads/...`) que impiden su re-ejecución.

6.4 ¿La ejecución cumplió el objetivo principal?

Hallazgo

Sí, en lo esencial. Los entregables declarados existen y son consistentes entre sí: 14 hallazgos críticos documentados, 79 CSVs de evidencias, 56 figuras, dashboard de 7 capítulos, informe LaTeX (30 págs.), presentación y manual ejecutable. Los hallazgos responden las 7 preguntas de investigación y su narrativa es coherente (patrón Pregunta → Hallazgo → Caveat). La ejecución fue **correcta en la lógica analítica** pero **no óptima en la industrialización** (reproducibilidad, automatización, pruebas), lo que se recomienda corregir en la fase de entrega.

6.5 Verificación empírica de ejecutabilidad (auditoría 2026)

Como parte de esta consultoría se ejecutaron los 16 scripts de orquestación del repositorio en un entorno limpio (venv Python, clon fresco con el XLSX versionado). Resultados:

Tabla 4. Ejecución real de los scripts del repositorio

Script	Exit	Tiempo	Resultado
<code>sprint2_panel_longitudinal.py</code>	0	108 s	OK
<code>sprint2_matrices_transicion.py</code>	0	74 s	OK

Script	Exit	Tiempo	Resultado
sprint2_territorial.py	0	0.4 s	OK
sprint2_genero_ocde.py	0	61 s	OK
sprint2_duckdb.py	0	68 s	OK
sprint3_redes.py	0	64 s	OK
sprint3_grafo_interactivo.py	0	54 s	OK
sprint4_diversidad.py	0	82 s	OK
sprint6_validacion_calidad.py	0	62 s	OK
sprint6_tabla_maestra_ies.py	0	1.9 s	OK
sprint6_ocde_composicion.py	1	58 s	ERROR — falta datos/raw/produccion_grupos.csv (gitignored)
sprint6_sankey_categoria.py	1	56 s	ERROR — kaleido no declarado (requisito de write_image)
sprint6_geografia_institucional.py	1	38 s	ERROR — UnicodeEncodeError (→ en print, Windows cp1252)
sprint6_sankey_territorial.py	1	2.7 s	ERROR — kaleido no declarado
sprint5_produccion.py	1	50 s	ERROR — falta dataset de producción (gitignored)
sprint5_duckdb.py	1	2.2 s	ERROR — falta dataset de producción (gitignored)

Hallazgo

10 de 16 scripts (62 %) se ejecutan correctamente en un clon fresco. Los 6 fallos se explican por: (i) datos de producción gitignored (4 scripts), (ii) dependencia no declarada `kaleido` (2 scripts) y (iii) un `print` con el carácter `→` que falla en consolas Windows (1 script). Además, los 10 que corren lo hacen sobre el XLSX de 3 convocatorias (50 891 filas), por lo que **producen resultados distintos a las evidencias versionadas**: p. ej. el panel longitudinal regenerado solo tiene 2 periodos (2017–2019, 2019–2021) frente a los 5 periodos (2013–2021) del archivo versionado. Esto demuestra empíricamente que **un clon fresco no reproduce los hallazgos del README**.

6.6 Recomendaciones de mejora de la lógica

1. Corregir CI/CD y Dockerfile; alinear el despliegue al dashboard real.
2. Normalizar codificación UTF-8 en el pipeline y añadir contratos de esquema.
3. Migrar el análisis multivariado/clustering a `src/` con validación formal (silhouette, Calinski–Harabasz) o documentarlo como histórico.
4. Versionar datos críticos (DVC o Git LFS) y documentar la ingesta de producción.
5. Ampliar tests a funciones de transformación y análisis.
6. Unificar nomenclatura (Grupo 8, rama `main`) y eliminar artefactos duplicados.

7 Línea de tiempo de la construcción del repositorio

7.1 Resumen general

El repositorio fue construido con **133 commits** entre **2025-08-24** y **2026-05-14** por **12** identidades de autor (12 personas/roles distintos). La construcción muestra dos etapas claras: una fase académica inicial (tareas de análisis por convocatoria, ago–oct 2025) y una fase de ingeniería de datos (pipelines, sprints 2–6, oct 2025 – may 2026).

7.2 Distribución de autoría

Autor	Correo	Commits	%
camiloacr1322	camilocast2213@gmail.com	46	34.6 %
Victor-Diaz-Usta	darksite65@gmail.com	34	25.6 %
Maria Amaya	paulaamaya012@gmail.com	13	9.8 %
Paula	64102754+PaulaGuevara@users.noreply.github.com	11	8.3 %
JulianMendez27	julianmendez0420@gmail.com	11	8.3 %
Victor-Diaz-Usta	Victordiazb@usantotomas.edu.co	5	3.8 %
PaulaGuevara	paulaximenag2@gmail.com	4	3.0 %
Julian Mendez	julianmendez0420@gmail.com	3	2.3 %
copilot-swe-agent[bot]	198982749+Copilot@users.noreply.github.com	3	2.3 %
Camilo Castillo	camilocast2213@gmail.com	1	0.8 %
Ustadistica Bot	ustadistica@usantotomas.edu.co	1	0.8 %
Paula Betancour	paulagonzalezoh@gmail.com	1	0.8 %

Tabla 5. Commits por autor

7.3 Actividad por mes

Mes	Commits
2025-08	9
2025-09	46
2025-10	15
2025-11	6
2026-02	1
2026-03	17
2026-04	35
2026-05	4

Tabla 6. Commits por mes

7.4 Tabla completa de commits

Tabla 7. Línea de tiempo completa de commits (orden cronológico)

#	Fecha	Autor	Hash	Asunto
1	2025-08-24	Paula	1bc40e1b	Initial commit
2	2025-08-24	JulianMendez27	ed99fc3c	Update README.md
3	2025-08-25	Paula	be573fb5	Update README.md
4	2025-08-25	Paula	938bc32f	Update README.md
5	2025-08-25	JulianMendez27	f0d30120	README.md
6	2025-08-29	Julian Mendez	1cd5ace8	Agrego resultados de tarea Join (bases unidas)
7	2025-08-31	camiloacr1322	fb053b8d	Create README.md
8	2025-08-31	camiloacr1322	03b3f0bb	Add files via upload
9	2025-08-31	camiloacr1322	97c4d987	Add files via upload
10	2025-09-01	camiloacr1322	22bd8716	agrego el md de mi analisis Analisis_17.md
11	2025-09-01	camiloacr1322	a7134584	introduccion de Analisis_17.md
12	2025-09-01	camiloacr1322	c6c01a84	introducción a Analisis_17.md
13	2025-09-01	camiloacr1322	c1ed5862	imagen Analisis_17.md
14	2025-09-01	camiloacr1322	d4ec3e17	Update Analisis_17.md
15	2025-09-01	camiloacr1322	20d47fd8	Update Analisis_17.md
16	2025-09-01	camiloacr1322	4eca72d6	Update Analisis_17.md
17	2025-09-02	camiloacr1322	62881b89	Update Analisis_17.md
18	2025-09-02	camiloacr1322	6207fce6	Update Analisis_17.md
19	2025-09-02	camiloacr1322	e0d80901	Update Analisis_17.md
20	2025-09-02	camiloacr1322	567cf0e9	Update Analisis_17.md
21	2025-09-02	camiloacr1322	8a36f539	Update Analisis_17.md
22	2025-09-02	camiloacr1322	f412ac70	termino análisis Analisis_17.md
23	2025-09-02	camiloacr1322	7306368b	Update Análisis_17.md
24	2025-09-02	camiloacr1322	0024e370	termino análisis Analisis_17.md
25	2025-09-02	Paula	297b0633	Create 2021
26	2025-09-02	Paula	09e9b978	Delete 2021
27	2025-09-02	Paula	593e4a95	datos_2021
28	2025-09-02	camiloacr1322	e37322bb	agrego R
29	2025-09-03	Camilo Castillo	5fda9af6	qito base de datos subidad anteriormente
30	2025-09-03	camiloacr1322	72446079	codigo de la creación y verificación de bases de deatosarreglar_base_consultoria_7.R
31	2025-09-03	camiloacr1322	71960bf6	Update README.md
32	2025-09-03	camiloacr1322	4b175f5c	Update README.md
33	2025-09-03	camiloacr1322	2363a8c5	Update README.md
34	2025-09-04	camiloacr1322	b0a784eb	Update README.md
35	2025-09-08	PaulaGuevara	3d63c1a8	update 2021
36	2025-09-08	PaulaGuevara	37bee23c	upd_2021
37	2025-09-09	Julian Mendez	926cc271	Añadir análisis 2019, código y gráficos
38	2025-09-09	Julian Mendez	08df1711	Añadir análisis 2019, código y gráficos
39	2025-09-09	JulianMendez27	1498dd5d	Update 2019_Analisis.md
40	2025-09-09	JulianMendez27	23be3f36	1
41	2025-09-17	camiloacr1322	2f8f09e4	Create README.md
42	2025-09-17	camiloacr1322	55df6fb9	Create codigo dimensiones_C.R
43	2025-09-17	camiloacr1322	24598bfa	Update README.md
44	2025-09-17	PaulaGuevara	02a9fb81	Agrego codigo de dimensiones en Colab
45	2025-09-17	Paula	22d49828	Upd README.md
46	2025-09-17	Paula	51d16bb6	Upd README.md
47	2025-09-17	camiloacr1322	a1329029	Update README.md
48	2025-09-17	camiloacr1322	ad9b51f8	Update README.md
49	2025-09-17	camiloacr1322	777795c3	Update README.md
50	2025-09-17	camiloacr1322	fb3d2d0b	Update README.md
51	2025-09-17	camiloacr1322	e468c108	Update README.md
52	2025-09-17	camiloacr1322	2869c6e8	Update codigo dimensiones_C.R
53	2025-09-17	camiloacr1322	887129b0	Update README.md
54	2025-09-18	camiloacr1322	e912b87f	codigo de la tabla Tabla_hechos_codigo.R
55	2025-09-30	Paula	14f9cb19	Create analisis2.ipynb
56	2025-10-01	Paula	7482554d	analisisB
57	2025-10-02	camiloacr1322	e923d163	Create Informe_multivariado.md

7.4 Tabla completa de eventos TIEMPO DE LA CONSTRUCCIÓN DEL REPOSITORIO

#	Fecha	Autor	Hash	Asunto
58	2025-10-07	camiloacr1322	963b16c4	Update Informe_multivariado.md
59	2025-10-07	camiloacr1322	d3c8bac5	Update Informe_multivariado.md
60	2025-10-07	camiloacr1322	7618a9a6	Update Informe_multivariado.md
61	2025-10-07	camiloacr1322	95558e48	Update Informe_multivariado.md
62	2025-10-07	camiloacr1322	bfcdb3a8	Update Informe_multivariado.md
63	2025-10-07	camiloacr1322	57f7d1ac	Add files via upload
64	2025-10-08	JulianMendez27	69567165	Análisis Univariado
65	2025-10-08	JulianMendez27	570728e5	Add imagenes
66	2025-10-08	JulianMendez27	8149bbdc	Codigo Univariado
67	2025-10-08	JulianMendez27	c09343a7	Update Analisis_Univariado.md
68	2025-10-08	JulianMendez27	20030ac9	a
69	2025-10-08	JulianMendez27	3f6843f1	Codigo Univariado
70	2025-10-08	JulianMendez27	c662cef7	Creacion gran tabla a partir del modelo estrella
71	2025-11-07	camiloacr1322	b621e2a2	Se sube el código de el análisis de grupos
72	2025-11-12	camiloacr1322	0afa6315	Crear el Informe_clusters.md, para la exposición
73	2025-11-12	camiloacr1322	fb1fa6ad	Documentar análisis y resultados del proyecto de investigadores
74	2025-11-18	Paula	7e35cd67	cambio snakecase
75	2025-11-18	PaulaGuevara	ad506acf	Reorganización de archivos y carpetas
76	2025-11-18	camiloacr1322	d7f9eba7	Documentar análisis y resultados del proyecto de investigadores, informe final
77	2026-02-26	Ustadistica Bot	c7003149	feat: aplicar plantilla Ustadistica 2026-I\{\n\}nEstructura estandar: src/, notebooks/, app/, datos/, docs/, tests/\{\nBasado en Kickoff Meeting +MLOps template +Plan de Continuidad
78	2026-03-12	Maria Amaya	0b7b4244	feat(catalogo): agregar catalogo analitico de investigadores Minciencias
79	2026-03-13	Maria Amaya	dd937c64	docs: agregar evidencia de verificacion de convocatoria 2023
80	2026-03-13	Maria Amaya	344b6a78	docs(diccionario): renombrar catalogo y documentar variables de Minciencias
81	2026-03-13	copilot-swe-agent[bot]	6a4205a6	Initial plan
82	2026-03-13	copilot-swe-agent[bot]	3a852c5b	Sprint 0 onboarding: add project structure (pyproject.toml, poetry.lock, src/, notebooks, catalogo, MARCO.TEORICO, streamlit.app)
83	2026-03-13	copilot-swe-agent[bot]	43abb55b	Add .gitignore and remove committed pycache files
84	2026-03-13	Paula Betancour	e3569226	Merge pull request #26 from ustadistica/copilot/clone-repo-checkout-develop
85	2026-03-17	Maria Amaya	79ddd164	Agrega notebook del Sprint 2 con analisis longitudinal
86	2026-03-17	Maria Amaya	82e3befc	Documenta evidencia de la tarea 1 de tracking longitudinal
87	2026-03-17	Maria Amaya	cd4ba250	Documenta evidencia de la tarea 2 de transiciones de categoria
88	2026-03-17	Maria Amaya	12eaced8	Agrega resultados exportados y excluye entorno virtual del repositorio
89	2026-03-18	Maria Amaya	1896cd7f	Merge branch 'develop' into develop_mariap
90	2026-03-24	Maria Amaya	5e5e4f9e	feat: reorganizar notebook del Sprint 2 y agregar version comentada
91	2026-03-25	Maria Amaya	9517143c	docs: mejorar evidencias del Sprint 2 con cuadros resumen
92	2026-03-24	Maria Amaya	7962978a	Merge pull request #25 from ustadistica/develop_mariap
93	2026-03-24	Maria Amaya	8fc088b0	Merge branch 'main' into develop
94	2026-03-24	Maria Amaya	c70b05fb	Merge pull request #35 from ustadistica/develop
95	2026-04-23	Victor-Diaz-Usta	d48ef81a	feat: ingesta MinCiencias desde Socrata ---cierra #6
96	2026-04-23	Victor-Diaz-Usta	7eef8805	docs: completar catalogo.yaml con 30 variables y metadatos ---cierra #7
97	2026-04-23	Victor-Diaz-Usta	485d4969	docs: inventario de notebooks existentes ---cierra #8
98	2026-04-23	Victor-Diaz-Usta	8ef5b56c	refactor: reorganizar notebooks y corregir pipeline de ingesta ---cierra #9
99	2026-04-23	Victor-Diaz-Usta	007061b0	chore: eliminar catalogo.yaml duplicado de la raíz (obsoleto)

7.4 Tabla completa de actividades y tiempo de la construcción del repositorio

#	Fecha	Autor	Hash	Asunto
100	2026-04-23	Victor-Diaz-Usta	521256bb	chore: fusionar diccionario_minciencias.yaml en catalogo. yaml y eliminar duplicado
101	2026-04-26	Victor-Diaz-Usta	b6cabc1f	feat: Issue #11 ---Panel longitudinal de investigadores (6 convocatorias)
102	2026-04-26	Victor-Diaz-Usta	3ea5ae1e	docs: actualizar README con estado Sprint 2 ---Issue #11 completado
103	2026-04-26	Victor-Diaz-Usta	3b281ceb	feat: Issue #12 ---Matrices de transición de categoría (6 convocatorias)
104	2026-04-26	Victor-Diaz-Usta	b8e3b35c	docs: actualizar README con estado Sprint 2 ---Issue #12 completado
105	2026-04-26	Victor-Diaz-Usta	88ec6277	feat: Issue #13 ---Análisis de concentración territorial (HHI)
106	2026-04-26	Victor-Diaz-Usta	2e0094ef	docs: actualizar README con estado Sprint 2 ---Issue #13 completado
107	2026-04-26	Victor-Diaz-Usta	57ead679	refactor: convertir notebook 02 a scripts Python por sprint
108	2026-04-26	Victor-Diaz-Usta	9da1259f	feat: Issue #14 ---Modelo dimensional en DuckDB
109	2026-04-26	Victor-Diaz-Usta	ff3d7598	docs: actualizar README con estado Sprint 2 ---Issue #14 completado
110	2026-04-26	Victor-Diaz-Usta	9e0cfd4d	feat: Issue #22 ---Análisis de genero por area OCDE
111	2026-04-26	Victor-Diaz-Usta	95acde4e	docs: Sprint 2 completado ---Issue #22 cerrado, README actualizado
112	2026-04-26	Victor-Diaz-Usta	bb0a6f5c	chore: limpieza de codigo muerto post-Sprint 2
113	2026-04-26	Victor-Diaz-Usta	36591b2e	docs: actualiza README con estructura post-limpieza
114	2026-04-26	Victor-Diaz-Usta	b681920d	feat: Issue #15 ---Network analysis de co-filiacion institucional
115	2026-04-26	Victor-Diaz-Usta	92eb8bcc	docs: Sprint 3 iniciado ---Issue #15 completado, README actualizado
116	2026-04-26	Victor-Diaz-Usta	6aae6156	feat: Issue #16 ---Visualizacion interactiva Pyvis del grafo de co-filiacion
117	2026-04-26	Victor-Diaz-Usta	d156573c	docs: Issue #16 completado ---README y CLAUDE.md actualizados
118	2026-04-26	Victor-Diaz-Usta	df606309	feat: Issue #17 ---Mapa interactivo de investigadores por departamento
119	2026-04-26	Victor-Diaz-Usta	d26bca71	docs: Issue #17 completado ---README y CLAUDE.md actualizados
120	2026-04-26	Victor-Diaz-Usta	006dbf7a	feat: Issue #18 ---Tabla de instituciones en dashboard
121	2026-04-26	Victor-Diaz-Usta	0f7b3179	docs: Issue #18 completado ---README y CLAUDE.md actualizados
122	2026-04-26	Victor-Diaz-Usta	e0ea973d	feat: Issue #19 ---Configuracion para deploy en Streamlit Cloud
123	2026-04-26	Victor-Diaz-Usta	7e15d248	feat: Issue #20 ---Análisis de diversidad (conflicto, etnia, discapacidad)
124	2026-04-26	Victor-Diaz-Usta	5b84cf99	docs: Sprint 4 iniciado ---Issue #20 completado, README actualizado
125	2026-04-26	Victor-Diaz-Usta	72f7a5eb	refactor: dashboard con narrativa critica en 6 secciones tipo capitulo de informe
126	2026-04-26	Victor-Diaz-Usta	52f10df3	fix: dashboard ---heatmap genero legible, normalizacion de instituciones, metodologia DANE
127	2026-04-26	Victor-Diaz-Usta	a59482d3	fix: normalizacion en grafo +bug 'estado' en analisis longitudinal
128	2026-04-26	Victor-Diaz-Usta	f203fff1	docs: actualiza README con refactor del dashboard y hallazgos criticos
129	2026-04-30	Victor-Diaz-Usta	ce8a4a0e	feat: Sprint 5 ---analisis de produccion de grupos cruzado con investigadores
130	2026-05-01	Victor-Diaz-Usta	9cbfc68b	docs: actualiza README con Sprint 5 ---produccion de grupos
131	2026-05-13	Victor-Diaz-Usta	fad1e803	feat: Sprint 6 ---ajustes del director (calidad, tabla maestra, geografia, sankeys, manual)
132	2026-05-14	Victor-Diaz-Usta	0a793453	chore: agrega evidencias y artifacts pendientes de sprints 2-4
133	2026-05-14	Victor-Diaz-Usta	15289391	Merge pull request #23 from Victor-Diaz-Usta/main_ VictorD

7.5 Lectura de la línea de tiempo

- El commit inicial (2025-08-24) y los primeros mensajes son de documentación y carga de tareas (README, Add files via upload), característicos de la fase académica.
- A partir de oct/nov 2025 se observa la consolidación del pipeline: ingesta, transformación, modelo dimensional y sprints de análisis.
- Los sprints 5 y 6 (abr–may 2026), liderados por Victor-Diaz-Usta, incorporan producción, calidad, tabla maestra IES y Sankeys, cerrando los ajustes del director.

A Apéndice A: Inventario de archivos de código

Tabla 8. Inventario de los 51 archivos de código del repositorio

Ruta	Lenguaje	Líneas	KB	Rol
app\{}\streamlit.app.py	Python	17	0,4	visualizacion
docs\{}\manual.ipynb	Notebook	805	27	documentacion
notebooks\{}\01.eda.ipynb	Notebook	240	6,7	eda
notebooks\{}\tarea.anlisis_bases_de_datos\{}\2019_codigo.py	Python	174	6,7	legacy
notebooks\{}\tarea.anlisis_bases_de_datos\{}\datos_2021.ipynb	Notebook	2338	957,3	legacy
notebooks\{}\tarea.anlisis_bases_de_datos\{}\inv.17_codigo_graficas.R	R	614	18,4	legacy
notebooks\{}\tarea.dimensiones_y_hechos\{}\codigo.dimensiones.C.R	R	40	0,9	legacy
notebooks\{}\tarea.dimensiones_y_hechos\{}\dimensiones.ipynb	Notebook	289	11,8	legacy
notebooks\{}\tarea.dimensiones_y_hechos\{}\tabla.hechos_codigo.R	R	20	0,6	legacy
notebooks\{}\tarea_gran_tabla\{}\analisis_de_clusters.R	R	94	2,6	legacy
notebooks\{}\tarea_gran_tabla\{}\analisis_multivariado_base_gran_table.R	R	405	13,1	legacy
notebooks\{}\tarea_gran_tabla\{}\analisis2.ipynb	Notebook	294	11,9	legacy
notebooks\{}\tarea_gran_tabla\{}\codigo_analisis_univariado.py	Python	123	4,2	legacy
notebooks\{}\tarea_gran_tabla\{}\codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py	Python	205	7,3	legacy
notebooks\{}\tarea_join\{}\codigo.join.py	Python	26	0,8	legacy
notebooks\{}\tarea_tabla\{}\arreglar_base_consultoria.7.R	R	125	3,4	legacy
scripts\{}\generar_manual.py	Python	610	22,1	documentacion
scripts\{}\sprint2.duckdb.py	Python	127	4,6	orquestacion
scripts\{}\sprint2.genero.ocde.py	Python	182	6,8	orquestacion
scripts\{}\sprint2.matrices_transicion.py	Python	125	4	orquestacion
scripts\{}\sprint2.panel_longitudinal.py	Python	229	8,6	orquestacion
scripts\{}\sprint2.territorial.py	Python	142	4,7	orquestacion
scripts\{}\sprint3.grafo_interactivo.py	Python	72	2,1	orquestacion
scripts\{}\sprint3.redes.py	Python	134	4,7	orquestacion
scripts\{}\sprint4.diversidad.py	Python	172	6,8	orquestacion
scripts\{}\sprint5.duckdb.py	Python	193	6,4	orquestacion
scripts\{}\sprint5.produccion.py	Python	251	10,1	orquestacion

Ruta	Lenguaje	Líneas	KB	Rol
scripts\{}\sprint6_geografia_institucional.py	Python	302	12,3	orquestacion
scripts\{}\sprint6_ocde_composicion.py	Python	183	7,6	orquestacion
scripts\{}\sprint6_sankey_categoria.py	Python	184	6,2	orquestacion
scripts\{}\sprint6_sankey_territorial.py	Python	135	4,9	orquestacion
scripts\{}\sprint6_tabla_maestra_ies.py	Python	635	39,6	orquestacion
scripts\{}\sprint6_validacion_calidad.py	Python	121	4,3	orquestacion
src\{}__init__.py	Python	2	0,1	orquestacion
src\{}\analisis\{}__init__.py	Python	80	2,4	orquestacion
src\{}\analisis\{}\calidad.py	Python	105	4,3	analisis
src\{}\analisis\{}\diversidad.py	Python	162	6,1	analisis
src\{}\analisis\{}\genero.py	Python	85	2,9	analisis
src\{}\analisis\{}\longitudinal.py	Python	207	7,2	analisis
src\{}\analisis\{}\produccion.py	Python	340	12,5	analisis
src\{}\analisis\{}\redes.py	Python	225	8,1	analisis
src\{}\analisis\{}\territorial.py	Python	107	3,3	analisis
src\{}\ingesta\{}__init__.py	Python	53	1,9	orquestacion
src\{}\ingesta\{}\minciencias.py	Python	70	2,7	ingesta
src\{}\ingesta\{}\produccion.py	Python	106	4	analisis
src\{}\modelo\{}__init__.py	Python	2	0,1	orquestacion
src\{}\modelo\{}\dimensional.py	Python	286	9,2	modelado
src\{}\Transformacion.py	Python	222	5,9	transformacion
streamlit_app.py	Python	983	45,9	visualizacion
tests\{}__init__.py	Python	0	0	orquestacion
tests\{}\test_ingesta.py	Python	8	0,2	testing

Referencias